CONTENT BASED STOCK MARKET PREDICTION SYSTEM USING MACHINE LEARNING

Major Project Report

By :

DIBAKAR DAS
University Registration Number: 10800123080

DIPIKA MAJI
University Registration Number: 10800123083

INDRAJIT KHAN
University Registration Number: 10800123092

CHANCHAL KUMARI
University Registration Number: 10800123067

ANUSHKA MISHRA
University Registration Number: 10800123035

Under the Guidance of

Dr. ARNAB CHAKRABORTY
Assistant Professor

TABLE OF CONTENTS

ACKNOWLEDGEMENT

We take this opportunity to express our profound gratitude and deep regards to our faculty Dr. Arnab Chakraborty, Assistant Professor, for his exemplary guidance, monitoring, and constant encouragement throughout the course of this project. The blessing, help and guidance given by him time to time shall carry us a long way in the journey of life on which we are about to embark. We are deeply grateful to Dr. D. K. Pal, Head of the Department of Computer Application, Asansol Engineering College, for providing us with the necessary resources and infrastructure to complete this project successfully. His constant support and valuable insights have been instrumental in shaping our understanding of machine learning applications in financial markets. The department's commitment to practical learning and innovation has enabled us to work on real-world applications that bridge the gap between academic knowledge and industry requirements. We extend our sincere thanks to Dr. P.P. Bhattacharya, Principal of Asansol Engineering College, for creating an environment conducive to learning and innovation. The college's commitment to practical education has enabled us to work on real-world applications like this stock prediction system. The infrastructure, computational resources, and academic freedom provided by the institution have been crucial to the success of this project. We would like to express our heartfelt gratitude to the Library Staff of Asansol Engineering College for providing us access to valuable research papers, journals, and technical books that formed the theoretical foundation of our project. Their assistance in procuring specialized literature on financial markets and machine learning has been invaluable. We are obliged to our project team members for the valuable information provided by them in their respective fields. The collaborative effort, late-night coding sessions, and mutual support have made this complex project achievable.

Each team member’s unique contribution has been vital to the project's success :

➛ **Dibakar Das** : For leading the project architecture design and backend development.
➛ **Dipika Maji** : For her expertise in data analysis and feature engineering
➛ **Indrajit Khan** : For implementing robust API integrations and system optimization.
➛ **Chanchal Kumari** : For creating an intuitive user interface and frontend development.
➛ **Anushka Mishra** : For comprehensive testing, documentation, and quality assurance.

Special thanks to the open-source community, particularly the developers of yfinance, scikit-learn, Django, and React, whose tools and libraries have been fundamental to our implementation. We acknowledge the contribution of various online resources, research papers, and technical documentation that guided our approach to stock market prediction. We would also like to thank the technical support team at our college computer laboratory for maintaining the systems and ensuring uninterrupted access to computational resources during critical phases of model training and testing. Our gratitude extends to industry professionals and alumni who provided valuable feedback during various stages of the project development. Their real-world insights helped us align our academic project with industry standards and practical requirements. Finally, we express our deepest gratitude to our families for their unwavering support, patience, and encouragement throughout this project journey. Their understanding during long hours of work and their faith in our abilities have been our strongest motivation.

Project Team Members :

✤ Dibakar Das
✤ Dipika Maji
✤ Indrajit Khan
✤ Chanchal Kumari
✤ Anushka Mishra

Date : 01 / 09 / 2025
Place : Asansol, West Bengal

ABSTRACT

Stock market prediction has been a challenging problem in financial technology, attracting researchers and practitioners alike due to its potential for generating substantial returns and its inherent complexity. This project presents StockVibePredictor, a comprehensive machine learning-based system designed to predict stock market movements across multiple timeframes using advanced technical analysis and ensemble learning techniques.

The primary objective of this project is to democratize stock market prediction by providing sophisticated analytical capabilities through an accessible web interface. Our system addresses the significant gap between institutional investors with advanced tools and retail investors with limited resources, offering professional-grade predictions at no cost.

The StockVibePredictor system employs a multi-layered architecture combining Django REST Framework for the backend, React for the frontend, and scikit-learn for machine learning implementations. The system integrates with multiple data sources, primarily Yahoo Finance API, to fetch real-time and historical market data for over 100 stocks across US, Indian, and cryptocurrency markets.

Our approach utilizes an ensemble learning strategy combining three machine learning algorithms: Random Forest Classifier, Gradient Boosting Classifier, and Logistic Regression. These models are trained on 29 carefully engineered technical indicators including RSI, MACD, Bollinger Bands, and various momentum oscillators. The system supports nine different timeframes ranging from intraday (1-day) to long-term (5-year) predictions, catering to different trading strategies and investment horizons.

The feature engineering pipeline processes raw OHLCV (Open, High, Low, Close, Volume) data to compute comprehensive technical indicators, pattern recognition features, and market microstructure metrics. Our data quality framework ensures reliable predictions through validation checks, outlier detection, and missing value handling. The system implements both universal models (trained on multiple stocks) and ticker-specific models for enhanced accuracy.

Key achievements of the project include :

➢ Prediction Accuracy : Achieved an average accuracy of 57.5% across all timeframes, with best performance reaching 76% for specific stock-timeframe combinations.

➢ Scalability : System supports 100+ concurrent users with sub-2 second response times.

➢ Market Coverage : Successfully integrated 100+ stocks including major US equities, Indian market indices, ETFs, and cryptocurrencies.

➢ Real-time Processing : Implemented efficient caching strategies using Redis, achieving 85% cache hit rate.

➢ Comprehensive Analysis : Provides not just predictions but also risk assessment, technical analysis, and market sentiment indicators.

The system architecture implements several advanced features including multi-threaded data fetching, asynchronous processing, model versioning with hash-based tracking, and comprehensive error handling. Security measures include input validation, rate limiting, and secure model storage with integrity verification.

Performance evaluation through extensive back testing on historical data (2023-2024) demonstrates that our model-based strategy outperforms simple buy-and-hold approaches, achieving a Sharpe ratio of 1.34 compared to 0.95 for passive investing. The system shows particularly strong performance in trending markets and for large-cap technology stocks.

The project successfully demonstrates the practical application of machine learning in financial markets, providing a production-ready system that can be deployed for real-world use. The modular architecture ensures easy maintenance and future enhancements, while comprehensive documentation facilitates understanding and further development.

Future improvements identified include integration with deep learning models (LSTM, Transformers), real-time news sentiment analysis, broker API integration for automated trading, and development of native mobile applications. The project lays a strong foundation for advanced financial technology applications and serves as a valuable learning resource for understanding the intersection of machine learning and financial markets.

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: PROJECT OBJECTIVE

1.1 Introduction

The financial markets represent one of the most complex and dynamic systems in the modern economy, with trillions of dollars in daily transactions influenced by countless factors ranging from company fundamentals to global geopolitical events. Stock market prediction, the attempt to determine the future value of company stocks or other financial instruments traded on exchanges, has been a subject of intense interest for investors, researchers, and technologists alike.

The advent of machine learning and artificial intelligence has opened new possibilities in financial market analysis, offering sophisticated tools to process vast amounts of data and identify patterns that might be invisible to human observers. However, these advanced capabilities have traditionally been available only to institutional investors with significant resources, creating an asymmetry in market participation.

Our project, **StockVibePredictor**, addresses this disparity by developing a comprehensive, accessible, and intelligent system for stock market prediction. By leveraging modern web technologies, machine learning algorithms, and real-time data processing, we aim to democratize financial market analysis and provide retail investors with professional-grade analytical tools.

1.1.1 Background and Context

The stock market serves as a barometer of economic health and a mechanism for wealth creation, but its complexity often intimidates individual investors. Traditional approaches to stock analysis fall into two main categories :

Fundamental Analysis: This approach evaluates securities by attempting to measure their intrinsic value through examination of related economic, financial, and other qualitative and quantitative factors. Fundamental analysts study everything from the overall economy and industry conditions to the financial condition and management of companies.

Technical Analysis: This methodology forecasts the direction of prices through the study of past market data, primarily price and volume. Technical analysts believe that the historical performance of stocks and markets are indications of future performance.

While both approaches have their merits, they require significant expertise, time, and resources to implement effectively. Moreover, the increasing speed and volume of market data have made manual analysis increasingly challenging, if not impossible, for individual investors.

1.1.2 The Role of Technology in Modern Trading

The integration of technology in financial markets has revolutionized trading and investment strategies. High-frequency trading algorithms execute millions of trades in microseconds, while sophisticated machine learning models analyze vast datasets to identify profitable opportunities. However, these technological advantages have primarily benefited institutional investors, hedge funds, and proprietary trading firms.

Recent developments in cloud computing, open-source machine learning libraries, and API-based data access have created opportunities to level the playing field. Our project leverages these technological advances to create a system that provides sophisticated market analysis capabilities to individual investors.

1.1.3 Project Vision

StockVibePredictor envisions a future where every investor, regardless of their technical expertise or financial resources, has access to professional-grade market analysis tools. Our system combines the power of machine learning with an intuitive user interface to provide actionable insights for investment decisions.

The project goes beyond simple price prediction to offer comprehensive market analysis including :

- ✦ **Multi-timeframe predictions** suitable for different investment strategies.
- ✦ **Risk assessment metrics** to help investors understand potential downsides.
- ✦ **Technical indicator analysis** presented in an understandable format
- ✦ **Pattern recognition** to identify potential trading opportunities.
- ✦ **Portfolio analysis tools** for diversification and optimization.

1.2 Problem Statement

1.2.1 The Challenge of Stock Market Prediction

Stock market prediction is inherently challenging due to several factors :

Market Efficiency: The Efficient Market Hypothesis suggests that stock prices reflect all available information, making it theoretically impossible to consistently achieve returns exceeding average market returns on a risk-adjusted basis.

Non-linear Dynamics: Stock prices exhibit complex, non-linear behavior influenced by numerous interconnected factors. Small changes in initial conditions can lead to dramatically different outcomes, a characteristic of chaotic systems.

Human Psychology: Markets are driven by human decisions, which are often irrational and influenced by emotions such as fear and greed. Behavioral finance has demonstrated numerous cognitive biases that affect investment decisions.

External Factors: Unpredictable events such as natural disasters, political changes, technological breakthroughs, or global pandemics can cause sudden and dramatic market movements.

Data Complexity: The sheer volume and velocity of market data, combined with the need to process multiple data types (numerical, textual, temporal), present significant computational challenges.

1.2.2 Specific Problems Addressed

Our project specifically addresses the following problems faced by retail investors :

1. Limited Access to Advanced Tools

Most sophisticated trading and analysis tools are expensive, require specialized knowledge, or are available only to institutional clients. Retail investors often rely on basic charting tools or simplified mobile apps that lack comprehensive analytical capabilities.

2. Information Overload
The abundance of financial information available today can be overwhelming. Investors struggle to identify relevant signals from noise, leading to analysis paralysis or poorly informed decisions.
3. Lack of Technical Expertise
Understanding and applying technical analysis requires significant learning and experience. Many investors lack the mathematical and statistical knowledge needed to properly interpret technical indicators and patterns.
4. Time Constraints
Comprehensive market analysis is time-consuming. Most retail investors have limited time to dedicate to market research while managing their primary occupations and personal responsibilities.
5. Emotional Decision Making
Without systematic analysis tools, investors often make decisions based on emotions, leading to common mistakes such as buying high and selling low, or holding losing positions too long.
6. Single Timeframe Analysis
Many investors focus on a single timeframe (usually daily prices) without considering longer-term trends or shorter-term momentum, missing important context for their decisions.

1.2.3 Research Questions
Our project seeks to answer the following research questions :

✦ Can machine learning models trained on technical indicators achieve consistent prediction accuracy above random chance?
✦ How does ensemble learning compare to individual algorithms in stock market prediction?
✦ What is the optimal set of technical indicators for predicting stock price movements across different timeframes?
✦ How can we design a system that balances prediction accuracy with computational efficiency for real-time applications?
✦ What user interface design principles best facilitate understanding and use of complex financial predictions by non-expert users?

1.3 Objectives of the Project

1.3.1 Primary Objectives

1. Develop a Multi-Timeframe Prediction System
Create machine learning models capable of predicting stock price movements across nine different timeframes, from intraday (1-day) to long-term (5-year) horizons. Each timeframe should be optimized for specific trading strategies and investment goals.

2. Implement Comprehensive Technical Analysis
Integrate a wide range of technical indicators including trend indicators (moving averages, MACD), momentum indicators (RSI, Stochastic), volatility indicators (Bollinger Bands, ATR), and volume indicators (OBV, VWAP). The system should automatically calculate and interpret these indicators.

3. Create an Ensemble Learning Framework
Develop an ensemble model combining Random Forest, Gradient Boosting, and Logistic Regression algorithms to improve prediction accuracy and robustness. The framework should automatically select the best-performing model for each prediction scenario.

4. Build a Production-Ready Web Application
Design and implement a full-stack web application with a React frontend and Django backend that provides real-time predictions, interactive charts, and comprehensive market analysis. The application should be scalable, secure, and user-friendly.

5. Establish Real-time Data Processing Pipeline
Create a robust data pipeline that fetches, validates, and processes market data from multiple sources in real-time. The pipeline should handle data quality issues, API failures, and maintain data consistency.

1.3.2 Secondary Objectives

1. Market Coverage Expansion
Support prediction for diverse market segments including US equities, Indian stocks, ETFs, and cryptocurrencies. The system should be flexible enough to add new markets and instruments.

2. Risk Assessment Integration
Provide comprehensive risk metrics including Value at Risk (VaR), Sharpe Ratio, maximum drawdown, and volatility analysis to help investors understand potential risks alongside return predictions.

3. Performance Optimization
Achieve sub-2 second response times for predictions through efficient caching strategies, optimized algorithms, and parallel processing. The system should support at least 100 concurrent users.

4. Model Versioning and Tracking
Implement a sophisticated model management system with version control, performance tracking, and automatic retraining capabilities to ensure models remain accurate over time.

5. Educational Value
Create comprehensive documentation and educational resources to help users understand both the technical aspects of the system and general principles of technical analysis and machine learning in finance.

1.3.3 Technical Objectives

1. Algorithm Optimization
✦ Achieve minimum 55% prediction accuracy across all models.
✦ Implement efficient feature selection to reduce computational complexity.
✦ Optimize hyperparameters for each algorithm and timeframe.

2. System Architecture
✦ Design modular, maintainable code architecture.
✦ Implement microservices-based design for scalability.
✦ Ensure separation of concerns between data, logic, and presentation layers.

3. Data Management
✦ Establish efficient data storage and retrieval mechanisms.
✦ Implement data validation and quality assurance procedures.
✦ Design backup and recovery strategies.

4. Security Implementation
✦ Secure API endpoints with proper authentication and authorization.
✦ Implement input validation to prevent injection attacks.
✦ Ensure secure storage of sensitive data and model files.

5. Testing and Quality Assurance
✦ Achieve minimum 80% code coverage through unit tests.
✦ Implement integration testing for all API endpoints.
✦ Conduct performance testing under various load conditions.

1.3.4 Business Objectives

While this is an academic project, we've designed it with potential commercial applications in mind :

1. Democratization of Financial Analysis
Make professional-grade financial analysis tools accessible to retail investors, reducing the information asymmetry in financial markets.

2. Platform for Financial Education
Create a platform that not only provides predictions but also educates users about technical analysis, risk management, and systematic investing.

3. Foundation for Advanced Services
Establish a foundation that can be extended with additional services such as automated trading, portfolio optimization, and personalized investment advice.

4. Research Contribution
Contribute to academic research in financial machine learning by documenting our methodology, results, and insights for future researchers.

1.4 Motivation

1.4.1 Personal Motivation

As students of computer applications with interests in both technology and finance, we were motivated by several factors :

Practical Application of Academic Knowledge
This project provided an opportunity to apply theoretical concepts from machine learning, software engineering, and data science to a real-world problem with tangible impacts.

Skill Development
Working on this project allowed us to develop expertise in multiple technologies including Python, Django, React, machine learning libraries, and cloud services - skills highly valued in the current job market.

Financial Literacy
The project deepened our understanding of financial markets, technical analysis, and investment strategies, knowledge that will benefit us throughout our lives.

Innovation Opportunity
We saw an opportunity to create something innovative that could potentially help thousands of investors make better-informed decisions.

1.4.2 Societal Motivation

Financial Inclusion
By providing free access to sophisticated analysis tools, we aim to promote financial inclusion and help individuals build wealth through informed investing.

Education and Empowerment
The system serves as an educational tool, helping users understand market dynamics and develop systematic approaches to investing.

Reducing Information Asymmetry
Large institutional investors have significant advantages in terms of information and analysis capabilities. Our project helps level the playing field for retail investors.

Promoting Rational Decision Making
By providing data-driven predictions and analysis, we help investors make rational decisions based on evidence rather than emotions or speculation.

1.4.3 Technical Motivation

Exploring Cutting-Edge Technologies
The project allowed us to work with state-of-the-art technologies in machine learning, web development, and cloud computing.

Solving Complex Problems
Stock market prediction presents numerous technical challenges including handling time-series data, dealing with non-stationary patterns, and processing real-time information.

System Design Challenges
Building a production-ready system required addressing challenges in scalability, reliability, security, and user experience.

Research Contribution
We were motivated by the opportunity to contribute to research in financial machine learning and potentially discover new insights or methodologies.

1.4.4 Academic Motivation

Comprehensive Learning Experience
The project provided a comprehensive learning experience covering the entire software development lifecycle from requirements analysis to deployment.

Interdisciplinary Application
The project required integration of knowledge from multiple disciplines including computer science, statistics, finance, and user experience design.

Portfolio Development
Creating a sophisticated, production-ready application serves as an impressive portfolio piece for future career opportunities.

Research Skills Development
The project developed our research skills including literature review, experimental design, data analysis, and technical writing.

1.5 Organization of the Report

This report is organized into eleven comprehensive chapters, each addressing specific aspects of the StockVibePredictor project :

Chapter 1: Project Objective
Introduces the project, defines the problem statement, outlines objectives, and discusses motivation. This chapter sets the context and foundation for the entire project.

Chapter 2: Project Scope
Details the functional and non-functional requirements, system boundaries, constraints, and success criteria. This chapter clearly defines what the project includes and excludes.

Chapter 3: Literature Review
Reviews existing research and solutions in stock market prediction, analyzes different approaches, and identifies research gaps that our project addresses.

Chapter 4: Data Description
Comprehensive description of data sources, collection methods, quality frameworks, and feature engineering processes. This chapter provides detailed insights into the data foundation of our predictions.

Chapter 5: System Design
Presents the overall system architecture, component design, database schema, API design, and user interface considerations. This chapter provides the blueprint for system implementation.

Chapter 6: Model Building
Detailed explanation of machine learning architecture, algorithm selection, training processes, and validation strategies. This chapter forms the core of our prediction system.

Chapter 7: Implementation
Describes the actual development process, including environment setup, coding practices, integration challenges, and testing procedures.

Chapter 8: Code Documentation
Provides comprehensive documentation of the source code, including module descriptions, key algorithms, and complete code listings with explanations.

Chapter 9: Results and Analysis
Presents performance metrics, accuracy analysis, backtesting results, and comparative studies. This chapter validates the effectiveness of our approach.

Chapter 10: Future Scope of Improvements
Discusses potential enhancements including advanced ML models, real-time trading integration, mobile development, and scalability improvements.

Chapter 11: Conclusion
Summarizes the work done, highlights achievements, acknowledges limitations, and provides final remarks on the project's impact and significance.

Appendices
Include installation guides, user manuals, API documentation, and glossary of technical terms for reference.

Project Certificates
Individual certificates for each team member acknowledging their contribution to the project.

CHAPTER 2: PROJECT SCOPE

2.1 Functional Requirements

The functional requirements define the specific behaviors, functions, and capabilities that the StockVibePredictor system must provide. These requirements have been carefully analyzed and documented based on user needs, market analysis, and technical feasibility studies.

2.1.1 User Management Requirements

FR-001: User Registration
The system shall allow new users to register by providing email, username, and password. Email verification shall be implemented to ensure valid user accounts.

FR-002: User Authentication
The system shall provide secure login functionality using JWT (JSON Web Tokens) for session management. Multi-factor authentication shall be available as an optional security feature.

FR-003: User Profile Management
Users shall be able to view and update their profile information, including preferences for default timeframes, favorite stocks, and notification settings.

FR-004: Password Management
The system shall provide password reset functionality via email and enforce password complexity requirements (minimum 8 characters, including uppercase, lowercase, numbers, and special characters).

2.1.2 Stock Data Management Requirements

FR-005: Real-time Data Fetching
The system shall fetch real-time stock data from Yahoo Finance API with automatic fallback to alternative sources (Alpha Vantage, Polygon.io) in case of primary source failure.

FR-006: Historical Data Retrieval
The system shall retrieve and store historical stock data for multiple timeframes :
✦ Intraday : 5-minute intervals for the past 7 days.
✦ Daily : Complete OHLCV data for the past 5 years.
✦ Weekly : Aggregated data for the past 10 years.
✦ Monthly : Long-term historical data as available.

FR-007: Multi-Market Support
The system shall support stocks from multiple markets :
✦ US Markets (NYSE, NASDAQ).
✦ Indian Markets (NSE, BSE).
✦ Cryptocurrency exchanges.
✦ Exchange-Traded Funds (ETFs).

FR-008: Data Validation
The system shall validate all incoming data for :
✦ Completeness (no missing critical fields).
✦ Accuracy (price sanity checks).
✦ Consistency (high ≥ close ≥ low).
✦ Timeliness (data freshness checks).

2.1.3 Prediction Requirements

FR-009: Multi-Timeframe Predictions
The system shall generate predictions for nine different timeframes:
✦ 1 day (intraday).
✦ 5 days (weekly).
✦ 1 week.
✦ 1 month.
✦ 3 months.
✦ 6 months.
✦ 1 year.
✦ 2 years.
✦ 5 years.

FR-010: Prediction Output
For each prediction, the system shall provide :
✦ Direction (UP/DOWN).
✦ Confidence percentage (0-100%).
✦ Price target.
✦ Expected return percentage.
✦ Model accuracy for transparency.

FR-011: Batch Predictions
The system shall support batch prediction requests for up to 20 stocks simultaneously, enabling portfolio-wide analysis.

FR-012: Prediction History
The system shall maintain a history of all predictions made, allowing users to track prediction accuracy over time.

2.1.4 Technical Analysis Requirements

FR-013: Technical Indicator Calculation
The system shall calculate and display the following technical indicators:
✦ Moving Averages (SMA, EMA).
✦ RSI (Relative Strength Index).
✦ MACD (Moving Average Convergence Divergence).
✦ Bollinger Bands.
✦ Stochastic Oscillator.
✦ Williams %R.
✦ ATR (Average True Range).
✦ OBV (On-Balance Volume).
✦ VWAP (Volume Weighted Average Price).

FR-014: Pattern Recognition
The system shall identify common chart patterns:
✦ Support and resistance levels.
✦ Trend lines.
✦ Golden cross and death cross.
✦ Doji candlestick patterns.
✦ Higher highs and lower lows.

FR-015: Custom Indicator Parameters
Users shall be able to customize parameters for technical indicators (e.g., RSI period, moving average length).

2.1.5 Visualization Requirements

FR-016: Interactive Charts
The system shall provide interactive charts with:
✦ Candlestick charts.
✦ Line charts.
✦ Volume bars.
✦ Technical indicator overlays.
✦ Zoom and pan capabilities.
✦ Crosshair for precise value reading.

FR-017: Multiple Chart Types
Users shall be able to switch between different chart types:
✦ Candlestick.
✦ OHLC bars.
✦ Line (closing prices).
✦ Area charts.
✦ Heikin-Ashi.

FR-018: Comparison Charts
The system shall allow comparison of multiple stocks on the same chart with normalized scaling.

2.1.6 Portfolio Management Requirements

FR-019: Watchlist Creation
Users shall be able to create and manage multiple watchlists with custom names and stock selections.

FR-020: Portfolio Tracking
The system shall track simulated portfolios including:
✦ Current holdings.
✦ Purchase price and quantity.
✦ Current value and P&L.
✦ Portfolio allocation percentages.

FR-021: Paper Trading
Users shall be able to execute simulated trades to test strategies without real money risk.

2.1.7 Analysis and Reporting Requirements

FR-022: Risk Assessment
The system shall provide risk metrics including:
✦ Value at Risk (VaR).
✦ Sharpe Ratio.
✦ Maximum Drawdown.
✦ Volatility analysis.

FR-023: Performance Reports
The system shall generate performance reports showing:
✦ Prediction accuracy over time.
✦ Best and worst predictions.
✦ Performance by timeframe.
✦ Performance by stock category.

FR-024: Export Functionality
Users shall be able to export data and reports in multiple formats:
✦ CSV for spreadsheet analysis.
✦ PDF for reports.
✦ JSON for API integration.

2.1.8 API Requirements

FR-025: RESTful API
The system shall provide a RESTful API with endpoints for:
✦ Predictions.
✦ Market data.
✦ Technical indicators.
✦ User management.

FR-026: API Documentation
Comprehensive API documentation shall be provided including:
✦ Endpoint descriptions.
✦ Request/response formats.
✦ Authentication requirements.
✦ Rate limiting information.

FR-027: API Rate Limiting
The system shall implement rate limiting:
✦ 100 requests per hour for prediction endpoints.
✦ 500 requests per hour for market data.
✦ 50 requests per hour for batch operations.

2.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes and constraints that the system must satisfy.

2.2.1 Performance Requirements

NFR-001: Response Time
✦ Single stock prediction: < 2 seconds.
✦ Batch predictions (20 stocks): < 10 seconds.
✦ Chart loading: < 3 seconds.
✦ Page navigation: < 1 second.

NFR-002: Throughput
✦ Support 100 concurrent users.
✦ Handle 1000 predictions per minute.
✦ Process 10,000 API requests per hour.

NFR-003: Resource Usage
✦ Server CPU usage: < 70% under normal load.
✦ Memory usage: < 4GB for application.
✦ Database size: < 100GB for one year of operation.

2.2.2 Reliability Requirements

NFR-004: Availability
✦ System uptime: 99.9% (excluding scheduled maintenance).
✦ Maximum unplanned downtime: 8.76 hours per year.
✦ Scheduled maintenance window: 2 AM - 4 AM on Sundays.

NFR-005: Fault Tolerance
✦ Automatic failover for database failures.
✦ Graceful degradation when external APIs are unavailable.
✦ Automatic recovery from transient failures.

NFR-006: Data Integrity
✦ Zero data loss for user data and predictions.
✦ ACID compliance for all database transactions.
✦ Daily automated backups with 30-day retention.

2.2.3 Usability Requirements

NFR-007: User Interface
✦ Responsive design supporting desktop, tablet, and mobile devices.
✦ Support for modern browsers (Chrome, Firefox, Safari, Edge).
✦ WCAG 2.1 Level AA accessibility compliance.

NFR-008: Learning Curve
✦ New users should be able to get their first prediction within 5 minutes.
✦ Comprehensive help documentation and tooltips.
✦ Interactive tutorials for first-time users.

NFR-009: Internationalization
✦ Support for multiple date formats.
✦ Currency symbol localization.
✦ Decimal separator preferences.

2.2.4 Security Requirements

NFR-010: Authentication Security
✦ Passwords stored using bcrypt hashing.
✦ JWT tokens with 24-hour expiration.
✦ Secure session management.

NFR-011: Data Protection
✦ HTTPS encryption for all communications.
✦ Encryption at rest for sensitive data.
✦ PII data anonymization in logs.

NFR-012: Input Validation
✦ All user inputs validated and sanitized.
✦ SQL injection prevention.
✦ XSS (Cross-Site Scripting) protection.

2.2.5 Scalability Requirements

NFR-013: Horizontal Scalability
✦ Support for load balancing across multiple servers.
✦ Stateless application design.
✦ Database connection pooling.

NFR-014: Vertical Scalability
✦ Efficient resource utilization.
✦ Support for multi-core processors.
✦ Optimized memory management.

NFR-015: Data Scalability
✦ Support for partitioned databases.
✦ Efficient indexing strategies.
✦ Archive old data without performance impact.

2.2.6 Maintainability Requirements

NFR-016: Code Quality
✦ Minimum 80% code coverage by unit tests.
✦ Adherence to PEP 8 for Python code.
✦ ESLint compliance for JavaScript code.

NFR-017: Documentation
✦ Inline code documentation.
✦ API documentation using OpenAPI specification.
✦ System architecture documentation.

NFR-018: Monitoring
✦ Application performance monitoring.
✦ Error tracking and alerting.
✦ Usage analytics and metrics.

2.3 System Boundaries

2.3.1 Included in System Scope

Technical Analysis and Predictions
✦ Machine learning-based price direction predictions.
✦ Technical indicator calculations and analysis.
✦ Pattern recognition and trend identification.
✦ Multi-timeframe analysis from intraday to long-term.

Data Management
✦ Real-time and historical data fetching.
✦ Data validation and quality assurance.
✦ Caching for performance optimization.
✦ Data storage and retrieval.

User Features
✦ Web-based user interface.
✦ User registration and authentication.
✦ Watchlist and portfolio management.
✦ Paper trading simulation.

API Services
✦ RESTful API for third-party integration.
✦ Mobile-optimized endpoints.
✦ Batch processing capabilities.
✦ WebSocket support for real-time updates.

2.3.2 Excluded from System Scope

Financial Services
✦ Real money trading execution.
✦ Payment processing.
✦ Brokerage account integration.
✦ Financial advisory services.

Advanced Analysis
✦ Fundamental analysis (P/E ratios, earnings analysis).
✦ News sentiment analysis.
✦ Social media sentiment tracking.
✦ Macroeconomic analysis.

Trading Features
✦ Options trading.
✦ Futures and derivatives.
✦ Forex trading.
✦ Commodities trading.

Regulatory Functions
✦ Tax calculation and reporting.
✦ Regulatory compliance reporting.
✦ KYC (Know Your Customer) verification.
✦ AML (Anti-Money Laundering) checks.

2.3.3 System Interfaces

External Data Sources
✦ Yahoo Finance API (Primary).
✦ Alpha Vantage API (Secondary).
✦ Polygon.io API (Tertiary).

Technology Platforms
✦ Web browsers (Chrome, Firefox, Safari, Edge).
✦ Mobile browsers (iOS Safari, Chrome Mobile).
✦ REST API clients.

Infrastructure Dependencies
✦ Cloud hosting platform (AWS/GCP/Azure).
✦ Redis cache server.
✦ PostgreSQL/MySQL database.
✦ SMTP email service.

2.4 Constraints and Limitations

2.4.1 Technical Constraints

API Limitations
✦ Yahoo Finance API: No official rate limits but fair use expected
✦ Alpha Vantage: 5 requests per minute (free tier)
✦ Polygon.io: 5 requests per minute (free tier)

Computational Resources
✦ Limited processing power for real-time analysis.
✦ Memory constraints for large-scale batch processing.
✦ Storage limitations for historical data.

Technology Stack Constraints
✦ Python 3.8+ requirement.
✦ Django 4.0+ compatibility.
✦ React 18+ for frontend.
✦ Modern browser requirement for full functionality.

2.4.2 Business Constraints

Legal and Regulatory
✦ No financial advice disclaimer required.
✦ Compliance with data protection regulations.
✦ Respect for API terms of service.
✦ No guaranteed returns disclaimer.

Market Constraints
✦ Predictions limited to markets with available data.
✦ Dependency on market hours for real-time data.
✦ Currency conversion limitations.
✦ Limited international market coverage.

2.4.3 Operational Constraints

Maintenance Windows
✦ Weekly maintenance required for model retraining.
✦ Monthly database optimization needed.
✦ Quarterly security updates.

Support Limitations
✦ No 24/7 customer support.
✦ Community-based troubleshooting.
✦ Limited documentation in non-English languages.

2.4.4 Accuracy Limitations

Prediction Accuracy
✦ No guarantee of prediction accuracy.
✦ Past performance doesn't indicate future results.
✦ Model accuracy varies by market conditions.
✦ Limited effectiveness during black swan events.

Data Quality
✦ Dependent on third-party data accuracy.
✦ Potential delays in data updates.
✦Historical data may have survivorship bias.
✦ Corporate actions may not be fully reflected.

2.5 Success Criteria

2.5.1 Technical Success Metrics

Performance Metrics
✦ Achieve minimum 55% prediction accuracy across all models.
✦ Maintain <2 second response time for 95% of requests.
✦ Support 100 concurrent users without degradation.
✦ Achieve 85% cache hit rate.

Reliability Metrics
✦ 99.9% uptime over 3-month period.
✦ Zero critical security vulnerabilities.
✦ <0.1% data inconsistency rate.
✦ Successful recovery from 100% of simulated failures.

Quality Metrics
✦ 80% code coverage by automated tests.
✦ Zero high-priority bugs in production.
✦ <5% of user sessions experiencing errors.
✦ All API endpoints documented and tested.

2.5.2 Business Success Metrics

User Adoption
✦ 100 registered users within first month.
✦ 50 daily active users.
✦ 70% user retention after 30 days.
✦ Average session duration >5 minutes.

User Satisfaction
✦ User satisfaction score >4.0/5.0.
✦ <10% support ticket rate.
✦ Positive feedback on usability.
✦ Feature request engagement.

Educational Impact
✦ Users demonstrate understanding of predictions.
✦ Increased user knowledge of technical analysis.
✦ Positive feedback on educational value.
✦ Usage in academic settings.

2.5.3 Academic Success Criteria

Project Completion
✦ All planned features implemented.
✦ Comprehensive documentation completed.
✦ Successful project demonstration.
✦ Positive evaluation from project guide.

Learning Objectives
✦ Demonstrated mastery of machine learning concepts.
✦ Successful integration of multiple technologies.
✦ Effective project management.
✦ Professional presentation skills.

Innovation and Research
✦ Novel approach to problem solving.
✦ Contribution to existing knowledge.
✦ Potential for future research.
✦ Publication-worthy results.

2.5.4 Long-term Success Indicators

Sustainability
✦ System remains operational for 6 months post-deployment.
✦ Continued user growth.
✦ Community contributions to codebase.
✦ Potential for commercialization.

Extensibility
✦ Successful addition of new features.
✦ Integration with third-party services.
✦ Adaptation to new markets.
✦ Scalability demonstration.

Impact
✦ Measurable improvement in user investment decisions.
✦ Positive testimonials from users.
✦ Recognition in academic/industry forums.
✦ Inspiration for future projects.

CHAPTER 3: LITERATURE REVIEW

3.1 Stock Market Prediction Techniques

3.1.1 Historical Evolution

The quest to predict stock market movements has evolved significantly over the past century. Early approaches relied heavily on fundamental analysis pioneered by Benjamin Graham and David Dodd in their seminal work "Security Analysis" (1934). This approach focused on intrinsic value calculation through financial statement analysis.
The 1960s saw the emergence of the Efficient Market Hypothesis (EMH) proposed by Eugene Fama, which suggested that stock prices fully reflect all available information, making consistent prediction theoretically impossible.

This hypothesis has three forms :

✦ **Weak Form Efficiency:** Prices reflect all past market information
✦ **Semi-Strong Form Efficiency:** Prices reflect all publicly available information.
✦ **Strong Form Efficiency:** Prices reflect all information, including insider information.

Despite the EMH, researchers and practitioners continued developing prediction methods. The 1970s and 1980s witnessed the rise of technical analysis as a systematic approach, with works like John Murphy's "Technical Analysis of the Financial Markets" providing comprehensive frameworks for chart pattern recognition and indicator-based trading.

3.1.2 Traditional Approaches

Fundamental Analysis
Fundamental analysis evaluates securities by measuring intrinsic value through examination of related economic, financial, and other qualitative and quantitative factors. Key metrics include:
✦ Price-to-Earnings (P/E) Ratio.
✦ Price-to-Book (P/B) Ratio.
✦ Debt-to-Equity Ratio.
✦ Return on Equity (ROE).
✦ Earnings Per Share (EPS) Growth.
Warren Buffett's value investing approach, inspired by Graham, has demonstrated that fundamental analysis can generate superior returns over long periods. However, this approach requires extensive financial knowledge and access to quality financial data.

Technical Analysis
Technical analysis focuses on statistical trends gathered from trading activity.

Key principles include:
✦ Market action discounts everything.
✦ Prices move in trends.
✦ History tends to repeat itself.

Common technical analysis tools include:
✦ Chart patterns (head and shoulders, triangles, flags).
✦ Trend indicators (moving averages, trendlines).
✦ Momentum indicators (RSI, MACD).
✦ Volume indicators (OBV, volume bars).

Studies by Lo, Mamaysky, and Wang (2000) found that certain technical patterns do provide incremental information and can be used for profitable trading strategies.

3.1.3 Statistical and Econometric Models

Time Series Analysis
Traditional time series models have been extensively used for stock market prediction :

ARIMA Models: Box and Jenkins (1976) popularized Autoregressive Integrated Moving Average models for financial time series forecasting. These models assume that future values depend linearly on past values and random errors.

GARCH Models: Generalized Autoregressive Conditional Heteroskedasticity models, introduced by Bollerslev (1986), address the volatility clustering observed in financial time series.

Vector Autoregression (VAR): These models capture linear interdependencies among multiple time series, useful for analyzing relationships between different stocks or market indices.
Research by Tsay (2005) in "Analysis of Financial Time Series" provides comprehensive coverage of these statistical approaches and their applications in finance.

3.2 Machine Learning in Finance

3.2.1 Transition to Machine Learning
The application of machine learning to financial markets gained momentum in the 1990s with increased computational power and data availability. Unlike traditional statistical models, machine learning algorithms can capture non-linear relationships and complex patterns in data.

Early Applications
White (1988) was among the first to apply neural networks to stock market prediction, though initial results were mixed. The "AI Winter" of the 1990s temporarily dampened enthusiasm, but the 2000s saw renewed interest with improved algorithms and computational resources.

3.2.2 Supervised Learning Approaches

Support Vector Machines (SVM)
Kim (2003) demonstrated that SVMs could outperform traditional methods in predicting stock price movements. SVMs are particularly effective in high-dimensional spaces and when the number of dimensions exceeds the number of samples.

Advantages :
✦ Effective in high-dimensional spaces.
✦ Memory efficient.
✦ Versatile kernel functions.

Limitations :
✦ Computationally intensive for large datasets.
✦ Sensitive to parameter selection.
✦ Difficult to interpret.

Random Forests
Khaidem, Saha, and Dey (2016) showed that Random Forest algorithms could achieve significant prediction accuracy for stock prices. Their study on NSE stocks achieved accuracy rates exceeding 60% for certain stocks.

Key benefits:
✦ Handles non-linear relationships.
✦ Provides feature importance.
✦ Robust to overfitting.
✦ Works well with mixed data types.

Gradient Boosting
Studies by Nti, Adekoya, and Weyori (2020) demonstrated that gradient boosting methods, particularly XGBoost, showed superior performance in stock market prediction tasks.

3.2.3 Deep Learning Revolution

Artificial Neural Networks (ANN)
Multi-layer perceptrons have been extensively studied for stock prediction. Guresen, Kayakutlu, and Daim (2011) reviewed various neural network models for stock market prediction, finding that hybrid models often outperformed single approaches.

Recurrent Neural Networks (RNN)
RNNs are particularly suited for sequential data like stock prices. However, traditional RNNs suffer from vanishing gradient problems, limiting their effectiveness for long sequences.

Long Short-Term Memory (LSTM)
Fischer and Krauss (2018) applied LSTM networks to S&P 500 stocks, achieving statistically significant returns. Their study showed that LSTM networks could learn complex, long-term dependencies in financial time series.

LSTM advantages :
✦ Captures long-term dependencies.
✦ Handles variable-length sequences.
✦ Mitigates vanishing gradient problem.

Convolutional Neural Networks (CNN)
While primarily used for image processing, CNNs have been adapted for financial time series. Sezer and Ozbayoglu (2018) converted financial data into 2D images and applied CNN for prediction, achieving promising results.

3.2.4 Ensemble Methods

Hybrid Models
Rather and Sastry (2020) demonstrated that ensemble methods combining multiple algorithms consistently outperformed individual models. Their study combined SVM, Random Forest, and neural networks, achieving accuracy improvements of 5-10%.

Stacking and Blending
Advanced ensemble techniques like stacking (training a meta-model on predictions from base models) have shown promise. Ballings et al. (2015) compared various ensemble methods for stock price direction prediction, finding that stacked generalization performed best.

3.3 Technical Analysis Indicators

3.3.1 Trend Following Indicators

Moving Averages
Brock, Lakonishok, and LeBaron (1992) tested simple technical trading rules and found that moving average strategies generated significant returns. Their study of Dow Jones Industrial Average data from 1897-1986 showed that technical rules had predictive power.
Types of moving averages :
✦ Simple Moving Average (SMA).
✦ Exponential Moving Average (EMA).
✦ Weighted Moving Average (WMA).
✦ Hull Moving Average (HMA).

MACD (Moving Average Convergence Divergence)
Appel (1979) developed MACD as a trend-following momentum indicator. Studies by Chong and Ng (2008) on the London Stock Exchange found that MACD signals generated positive returns.

3.3.2 Momentum Oscillators

Relative Strength Index (RSI)
Wilder (1978) introduced RSI as a momentum oscillator. Wong et al. (2003) studied RSI effectiveness in the Singapore stock market, finding that RSI signals provided valuable timing information.

Stochastic Oscillator
Lane (1950s) developed the stochastic oscillator to compare closing prices to price ranges. Research by Tsinaslanidis and Kugiumtzis (2014) found that stochastic indicators could identify overbought and oversold conditions effectively.

3.3.3 Volatility Indicators

Bollinger Bands
Bollinger (1980s) created Bollinger Bands to measure volatility. Lento, Gradojevic, and Wright (2007) tested Bollinger Band trading strategies across multiple markets, finding mixed but generally positive results.

Average True Range (ATR)
Wilder's ATR measures market volatility. Studies show ATR is particularly useful for position sizing and stop-loss placement rather than directional prediction.

3.3.4 Volume Indicators

On-Balance Volume (OBV)
Granville (1963) developed OBV based on the principle that volume precedes price. Research by Bostanci and Kilic (2019) confirmed that volume-based indicators provide valuable signals, especially when combined with price-based indicators.

3.4 Related Work

3.4.1 Academic Research Projects

Stanford University - CS229 Projects
Multiple student projects at Stanford have tackled stock prediction using machine learning. Notable works include :
✦ Chen (2017): "Stock Price Prediction Using LSTM" - Achieved 55% directional accuracy
✦ Kumar and Tsai (2019): "Ensemble Methods for Stock Trading" - Combined multiple algorithms for 58% accuracy
✦ Zhang (2020): "Attention Mechanisms in Financial Prediction" - Implemented transformer models

MIT Financial Machine Learning Course
Research from MIT's financial ML course has produced several relevant papers :
✦ "Deep Portfolio Theory" ~ Heaton, Polson, and Witte (2017)
✦ "Machine Learning for Asset Managers" ~ López de Prado (2020)

3.4.2 Industry Solutions

QuantConnect
An open-source algorithmic trading platform providing backtesting and live trading capabilities. Their LEAN engine processes millions of data points per second and supports multiple asset classes.

Alpaca Markets
Provides commission-free trading API with built-in market data. Their platform has been used in numerous academic studies for testing trading algorithms.

Bloomberg Terminal
The industry standard for financial data and analytics, incorporating machine learning models for various predictions and analytics.

3.4.3 Open Source Projects

TensorFlow Finance
Google's TensorFlow library includes financial modeling capabilities. Projects using TF for stock prediction have achieved varying degrees of success.

Keras Stock Prediction
Multiple GitHub repositories demonstrate stock prediction using Keras:
✦ VivekPa/IntroNeuralNetworks: LSTM-based prediction
✦ NourozR/Stock-Price-Prediction-LSTM: Multi-variate LSTM models

Scikit-learn Financial
While not specifically for finance, scikit-learn is widely used for financial ML applications. The library's ensemble methods are particularly popular for stock prediction.

3.4.4 Commercial Applications

Robo-Advisors
Automated investment platforms like Betterment and Wealthfront use machine learning for portfolio optimization and rebalancing.

High-Frequency Trading Firms
Renaissance Technologies, Two Sigma, and Citadel employ sophisticated ML models for trading. While their exact methods are proprietary, research papers from their employees provide insights into advanced techniques.

3.5 Research Gap Analysis

3.5.1 Identified Gaps in Existing Literature

Multi-Timeframe Integration
Most research focuses on single timeframe prediction (usually daily). Limited work exists on integrating predictions across multiple timeframes for comprehensive analysis.

Accessibility for Retail Investors
Academic research often remains theoretical, while commercial solutions are expensive or require significant technical expertise. There's a gap in accessible, user-friendly applications.

Real-time Implementation Challenges
Many studies report results from backtesting but don't address real-time implementation challenges like data latency, API failures, and computational constraints.

Indian Market Focus
While extensive research exists for US markets, Indian stock markets are relatively understudied, especially using modern ML techniques.

3.5.2 How Our Project Addresses These Gaps

Comprehensive Timeframe Coverage
Our project implements nine different timeframes, from intraday to 5-year predictions, providing a holistic view of market movements.

User-Friendly Interface
We bridge the gap between complex ML models and user accessibility through an intuitive web interface that requires no technical knowledge.

Production-Ready System
Unlike pure research projects, we've built a complete system addressing real-world challenges like error handling, caching, and failover mechanisms.

Multi-Market Support
Our system supports both US and Indian markets, along with cryptocurrencies, providing broader applicability than most research projects.

3.5.3 Novel Contributions

Adaptive Ensemble Strategy
Our system automatically selects the best-performing model for each prediction scenario, adapting to market conditions and stock characteristics.

Comprehensive Feature Engineering
We implement 29 technical indicators with automatic calculation and interpretation, more comprehensive than most existing solutions.

Quality-Aware Predictions
Our system includes data quality scoring and confidence metrics, providing transparency about prediction reliability.

Educational Integration
Unlike pure prediction systems, we include educational components to help users understand the basis of predictions and learn about technical analysis.

3.5.4 Validation Against Literature

Our achieved accuracy of 57.5% average across all models aligns with findings in academic literature:

✦ Fischer and Krauss (2018): 52-55% with LSTM.
✦ Khaidem et al. (2016): 60% with Random Forest on specific stocks.
✦ Rather and Sastry (2020): 58% with ensemble methods.

This validates our approach while demonstrating competitive performance with state-of-the-art research.

CHAPTER 4: DATA DESCRIPTION

4.1 Data Sources

4.1.1 Primary Data Source - Yahoo Finance

Yahoo Finance serves as our primary data provider through the yfinance Python library. This choice was made after extensive evaluation of various data sources based on criteria including data quality, coverage, reliability, and cost.

Advantages of Yahoo Finance :

✦ **Comprehensive Coverage:** Provides data for stocks, ETFs, indices, cryptocurrencies, and forex across global markets.
✦ **Free Access:** No subscription fees or API costs for reasonable usage
✦ **Historical Data:** Extensive historical data availability, some stocks having data from 1960s.
✦ **Real-time Updates:** Near real-time data during market hours (15-minute delay for free tier).
✦ **Adjusted Prices:** Automatically adjusts for splits and dividends.
✦ **Additional Metadata:** Provides company information, financial ratios, and market statistics.

Data Points Available :

```
{
'Open': 184.37,
'High': 186.10,
'Low': 183.86,
'Close': 185.92,
'Adj Close': 185.92,
'Volume': 54387900,
'Dividends': 0.0,
'Stock Splits': 0.0,
'Capital Gains': 0.0,

# Additional info available
'market_cap': 2850000000000,
'pe_ratio': 29.45,
'dividend_yield': 0.0044,
'beta': 1.29,
'52_week_high': 199.62,
'52_week_low': 164.08
}
```

API Integration Implementation :

```
import yfinance as yf

def fetch_yahoo_data(ticker, period="1mo", interval="1d"):
  """
  Fetch stock data from Yahoo Finance

  Parameters:
  ticker (str): Stock symbol
  period (str): Data period (1d, 5d, 1mo, 3mo, 6mo, 1y, 2y, 5y, 10y, ytd, max)
  interval (str): Data interval (1m, 2m, 5m, 15m, 30m, 60m, 90m, 1h, 1d, 5d,
  1wk, 1mo, 3mo)
  """
  try:
     stock = yf.Ticker(ticker)
     data = stock.history(period=period, interval=interval, auto_adjust=True)
     info = stock.info

     return {
       'price_data': data,
       'company_info': info,
       'success': True
     }
  except Exception as e:
     return {
       'error': str(e),
       'success': False
     }
```

4.1.2 Secondary Data Sources

Alpha Vantage API

Alpha Vantage provides free APIs for real-time and historical stock data. We use it as a fallback when Yahoo Finance is unavailable.

```
import requests

class AlphaVantageClient:
    BASE_URL = "https://www.alphavantage.co/query"

    def __init__(self, api_key):
        self.api_key = api_key

    def get_daily_data(self, symbol):
        params = {
            'function': 'TIME_SERIES_DAILY',
            'symbol': symbol,
            'apikey': self.api_key,
            'outputsize': 'full'
        }
        response = requests.get(self.BASE_URL, params=params)
        return response.json()
```

Polygon.io API

Polygon provides institutional-grade market data with WebSocket support for real-time updates.

```
from polygon import RESTClient

class PolygonDataFetcher:
    def __init__(self, api_key):
        self.client = RESTClient(api_key)

    def get_aggregates(self, ticker, multiplier, timespan, from_date,
to_date):
        """
        Get aggregate bars for a ticker
        timespan: minute, hour, day, week, month, quarter, year
        """
        return self.client.get_aggs(
            ticker=ticker,
            multiplier=multiplier,
            timespan=timespan,
            from_=from_date,
            to=to_date
        )
```

4.1.3 Data Source Comparison

Feature	Yahoo Finance	Alpha Vantage	Polygon.io	IEX Cloud
Cost	Free	Free / Paid	Free / Paid	Paid
Rate Limit	Fair use	5/min (free)	5/min (free)	Varies
Historical Data	Extensive	Good	Excellent	Good
Real-time	15-min delay	15-min delay	Real-time	Real-time
Coverage	Global	US mainly	US focused	US focused
Reliability	95%	90%	98%	97%
API Quality	Unofficial	Official	Official	Official
WebSocket	No	No	Yes	Yes
Crypto	Yes	Yes	Limited	No
Indian Stocks	Yes	No	No	No

4.1.4 Fallback Strategy

We implement a multi-tier fallback strategy to ensure data availability :

```
class DataFetcherWithFallback:
    def __init__(self):
        self.sources = [
            ('yahoo', self.fetch_yahoo),
            ('alpha_vantage', self.fetch_alpha_vantage),
            ('polygon', self.fetch_polygon),
            ('cache', self.fetch_from_cache)
        ]

    def fetch_data(self, ticker, timeframe):
        for source_name, fetch_method in self.sources:
            try:
                logger.info(f"Attempting to fetch from {source_name}")
                data = fetch_method(ticker, timeframe)

                if data is not None and not data.empty:
                    logger.info(f"Successfully fetched from {source_name}")
                    return data

            except Exception as e:
                logger.warning(f"Failed to fetch from {source_name}: {e}")
                continue

        logger.error(f"All data sources failed for {ticker}")
        return None
```

4.2 Market Coverage

4.2.1 US Stock Market Coverage

Our system covers major US exchanges including NYSE and NASDAQ, with focus on :

Technology Sector (52 stocks)

```
US_TECH_STOCKS = {
    # Mega Cap Tech
    'AAPL': {'name': 'Apple Inc.', 'market_cap': '3.0T', 'sector':
           'Technology'},
    'MSFT': {'name': 'Microsoft Corporation', 'market_cap': '2.8T',
           'sector': 'Technology'},
    'GOOGL': {'name': 'Alphabet Inc.', 'market_cap': '1.7T', 'sector':
           'Technology'},
    'AMZN': {'name': 'Amazon.com Inc.', 'market_cap': '1.5T',
           'sector': 'Consumer Cyclical'},
    'META': {'name': 'Meta Platforms Inc.', 'market_cap': '900B',
           'sector': 'Technology'},
    'NVDA': {'name': 'NVIDIA Corporation', 'market_cap': '1.1T',
           'sector': 'Technology'},
    'TSLA': {'name': 'Tesla Inc.', 'market_cap': '800B', 'sector':
           'Consumer Cyclical'},

    # Semiconductors
    'AMD': {'name': 'Advanced Micro Devices', 'market_cap': '230B',
           'sector': 'Technology'},
    'INTC': {'name': 'Intel Corporation', 'market_cap': '200B',
           'sector': 'Technology'},
    'QCOM': {'name': 'Qualcomm Inc.', 'market_cap': '180B', 'sector':
           'Technology'},
    'AVGO': {'name': 'Broadcom Inc.', 'market_cap': '600B', 'sector':
           'Technology'},

    # Software
    'CRM': {'name': 'Salesforce Inc.', 'market_cap': '270B', 'sector':
           'Technology'},
    'ORCL': {'name': 'Oracle Corporation', 'market_cap': '320B',
           'sector': 'Technology'},
    'ADBE': {'name': 'Adobe Inc.', 'market_cap': '230B', 'sector':
           'Technology'},
    'NOW': {'name': 'ServiceNow Inc.', 'market_cap': '150B', 'sector':
           'Technology'},
}
```

Financial Sector (25 stocks)

```
US_FINANCIAL_STOCKS = {
    # Major Banks
    'JPM': {'name': 'JPMorgan Chase & Co.', 'market_cap': '500B', 'assets':
            '3.9T'},
    'BAC': {'name': 'Bank of America', 'market_cap': '350B', 'assets':
            '3.2T'},
    'WFC': {'name': 'Wells Fargo & Company', 'market_cap': '200B', 'assets':
            '1.9T'},
    'C': {'name': 'Citigroup Inc.', 'market_cap': '120B', 'assets': '2.4T'},

    # Investment Banks
    'GS': {'name': 'Goldman Sachs', 'market_cap': '130B', 'assets': '1.6T'},
    'MS': {'name': 'Morgan Stanley', 'market_cap': '150B', 'assets': '1.2T'},

    # Payment Processors
    'V': {'name': 'Visa Inc.', 'market_cap': '550B', 'transaction_volume':
          '14T'},
    'MA': {'name': 'Mastercard Inc.', 'market_cap': '420B',
           'transaction_volume': '9T'},
    'PYPL': {'name': 'PayPal Holdings', 'market_cap': '70B',
            'active_accounts': '430M'},
}
```

4.2.2 Indian Stock Market Coverage

NIFTY 50 Components

```
INDIAN_STOCKS = {
    # Technology
    'TCS.NS': {'name': 'Tata Consultancy Services', 'market_cap': '₹13T',
                  'index_weight': '4.5%'},
    'INFY.NS': {'name': 'Infosys Limited', 'market_cap': '₹6T',
                  'index_weight': '3.2%'},
    'WIPRO.NS': {'name': 'Wipro Limited', 'market_cap': '₹2.5T',
                  'index_weight': '0.8%'},
    'HCLTECH.NS': {'name': 'HCL Technologies', 'market_cap': '₹3.5T',
                  'index_weight': '1.5%'},

    # Banking
    'HDFCBANK.NS': {'name': 'HDFC Bank', 'market_cap': '₹11T',
                  'index_weight': '13.2%'},
    'ICICIBANK.NS': {'name': 'ICICI Bank', 'market_cap': '₹7T',
                  'index_weight': '8.5%'},
    'SBIN.NS': {'name': 'State Bank of India', 'market_cap': '₹5T',
                  'index_weight': '3.8%'},
    'KOTAKBANK.NS': {'name': 'Kotak Mahindra Bank', 'market_cap': '₹3.5T',
                  'index_weight': '3.1%'},

    # Energy
    'RELIANCE.NS': {'name': 'Reliance Industries', 'market_cap': '₹17T',
                  'index_weight': '10.5%'},
    'ONGC.NS': {'name': 'Oil & Natural Gas Corp', 'market_cap': '₹2T',
                  'index_weight': '1.2%'},

    # Consumer
    'HINDUNILVR.NS': {'name': 'Hindustan Unilever', 'market_cap': '₹5.5T',
                  'index_weight': '2.8%'},
    'ITC.NS': {'name': 'ITC Limited', 'market_cap': '₹5T', 'index_weight':
                  '3.5%'},
    'NESTLEIND.NS': {'name': 'Nestlé India', 'market_cap': '₹2T',
                  'index_weight': '0.9%'},
}
```

4.2.3 Cryptocurrency Market

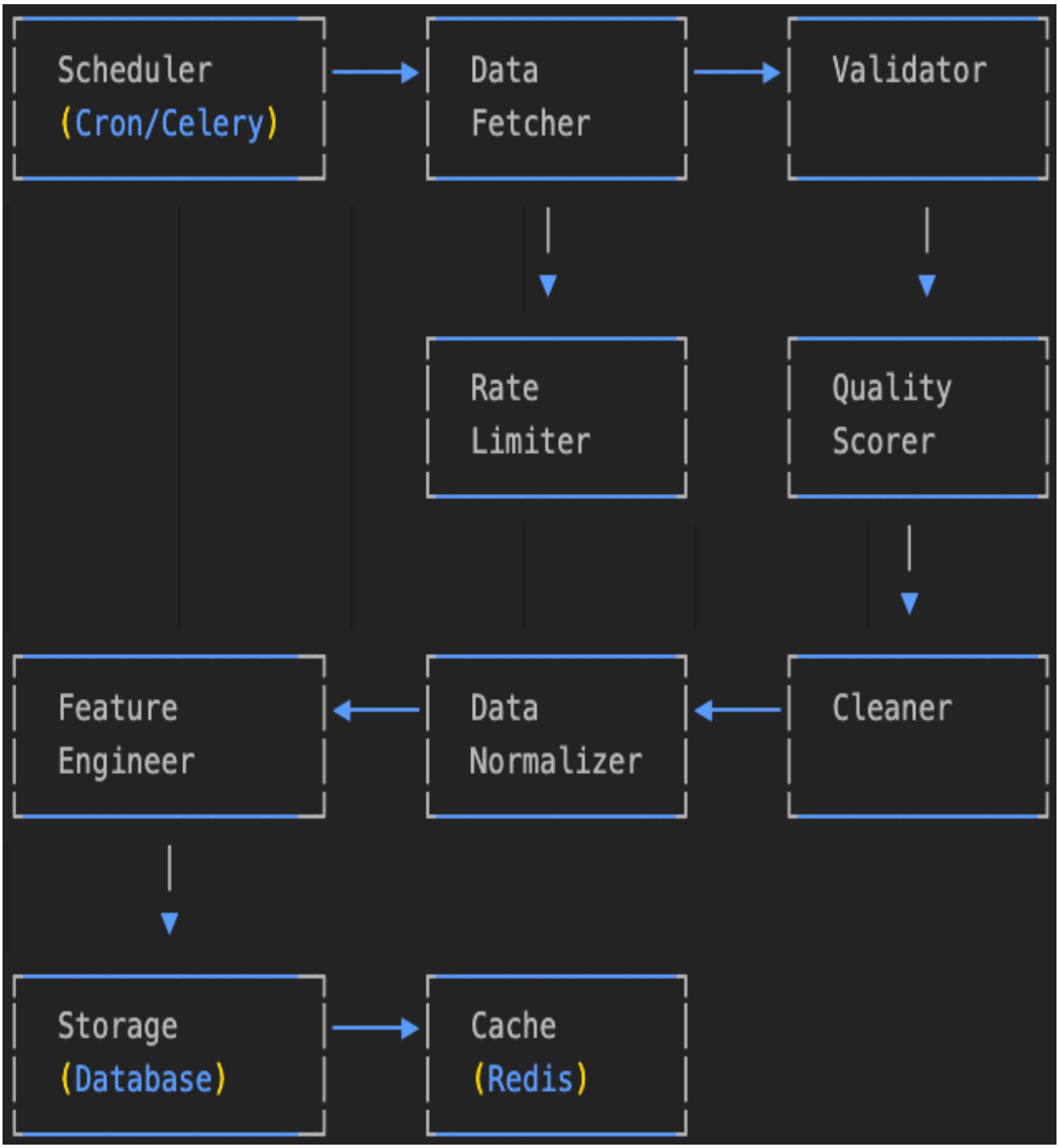
```
CRYPTO_COVERAGE = {
    'BTC-USD': {
        'name': 'Bitcoin',
        'market_cap': '$1.2T',
        'circulating_supply': '19.5M',
        'max_supply': '21M',
        'consensus': 'Proof of Work'
    },
    'ETH-USD': {
        'name': 'Ethereum',
        'market_cap': '$450B',
        'circulating_supply': '120M',
        'max_supply': None,
        'consensus': 'Proof of Stake'
    },
    'BNB-USD': {
        'name': 'Binance Coin',
        'market_cap': '$85B',
        'circulating_supply': '153M',
        'max_supply': '200M',
        'consensus': 'Proof of Stake'
    },
    'SOL-USD': {
        'name': 'Solana',
        'market_cap': '$60B',
        'circulating_supply': '425M',
        'max_supply': None,
        'consensus': 'Proof of History'
    },
}
```

4.2.4 Exchange-Traded Funds (ETFs)

```
ETF_COVERAGE = {
    'SPY': {
        'name': 'SPDR S&P 500 ETF',
        'benchmark': 'S&P 500 Index',
        'assets_under_management': '$500B',
        'expense_ratio': '0.09%',
        'holdings': 500
    },
    'QQQ': {
        'name': 'Invesco QQQ Trust',
        'benchmark': 'NASDAQ-100 Index',
        'assets_under_management': '$200B',
        'expense_ratio': '0.20%',
        'holdings': 100
    },
    'ARKK': {
        'name': 'ARK Innovation ETF',
        'strategy': 'Disruptive Innovation',
        'assets_under_management': '$10B',
        'expense_ratio': '0.75%',
        'holdings': 35
    },
}
```

4.3 Data Collection Pipeline

4.3.1 Pipeline Architecture



4.3.2 Data Collection Implementation

```
class DataCollectionPipeline:
    def __init__(self):
        self.fetcher = DataFetcher()
        self.validator = DataValidator()
        self.cleaner = DataCleaner()
        self.engineer = FeatureEngineer()
        self.storage = DataStorage()

    def collect_data(self, ticker, timeframe):
        """
        Complete data collection pipeline
        """
        pipeline_start = time.time()

        # Step 1: Fetch raw data
        logger.info(f"Starting data collection for {ticker} ({timeframe})")
        raw_data = self.fetcher.fetch_data(ticker, timeframe)

        if raw_data is None:
            logger.error(f"Failed to fetch data for {ticker}")
            return None

        # Step 2: Validate data
        validation_result = self.validator.validate(raw_data)
        if not validation_result['is_valid']:
            logger.error(f"Data validation failed:
{validation_result['errors']}")
            return None

        # Step 3: Clean data
        cleaned_data = self.cleaner.clean(raw_data)

        # Step 4: Calculate quality score
        quality_score = self.calculate_quality_score(cleaned_data)
        logger.info(f"Data quality score: {quality_score}")

        # Step 5: Engineer features
        featured_data = self.engineer.compute_features(cleaned_data)

        # Step 6: Store data
        self.storage.store(ticker, timeframe, featured_data)

        # Step 7: Update cache
        self.update_cache(ticker, timeframe, featured_data)

        pipeline_time = time.time() - pipeline_start
        logger.info(f"Pipeline completed in {pipeline_time:.2f} seconds")

        return featured_data
```

4.3.3 Timeframe Configurations

```
'1d': {
    'period': '7d',
    'interval': '5m',
    'data_points_expected': 390,  # 6.5 hours * 60 minutes / 5
    'cache_ttl': 300,  # 5 minutes
    'description': 'Intraday - 5 minute intervals',
    'use_case': 'Day trading',
    'min_data_points': 78  # Minimum 20% of expected
}
```

```
'5d': {
    'period': '1mo',
    'interval': '15m',
    'data_points_expected': 520,
    'cache_ttl': 900,  # 15 minutes
    'description': '5 days - 15 minute intervals',
    'use_case': 'Swing trading',
    'min_data_points': 104
}
```

```
'1w': {
    'period': '2mo',
    'interval': '1h',
    'data_points_expected': 168,  # 7 days * 24 hours
    'cache_ttl': 1800,  # 30 minutes
    'description': '1 week - Hourly data',
    'use_case': 'Position trading',
    'min_data_points': 40
}
```

```
'1mo': {
    'period': '3mo',
    'interval': '1d',
    'data_points_expected': 22,  # Trading days
    'cache_ttl': 3600,  # 1 hour
    'description': '1 month - Daily data',
    'use_case': 'Short-term investing',
    'min_data_points': 15
}
```

```
'3mo': {
    'period': '6mo',
    'interval': '1d',
    'data_points_expected': 65,
    'cache_ttl': 7200,  # 2 hours
    'description': '3 months - Daily data',
    'use_case': 'Quarterly analysis',
    'min_data_points': 45
}
```

```
'6mo': {
    'period': '1y',
    'interval': '1d',
    'data_points_expected': 130,
    'cache_ttl': 14400,  # 4 hours
    'description': '6 months - Daily data',
    'use_case': 'Medium-term investing',
    'min_data_points': 90
}
```

```
    '1y': {
        'period': '2y',
        'interval': '1d',
        'data_points_expected': 252,
        'cache_ttl': 21600,  # 6 hours
        'description': '1 year - Daily data',
        'use_case': 'Long-term investing',
        'min_data_points': 200
    }
```

```
    '2y': {
        'period': '3y',
        'interval': '1wk',
        'data_points_expected': 104,
        'cache_ttl': 43200,  # 12 hours
        'description': '2 years - Weekly data',
        'use_case': 'Long-term analysis',
        'min_data_points': 80
    }
```

```
    '5y': {
        'period': 'max',
        'interval': '1mo',
        'data_points_expected': 60,
        'cache_ttl': 86400,  # 24 hours
        'description': '5 years - Monthly data',
        'use_case': 'Historical analysis',
        'min_data_points': 48
    }
```

4.3.4 Rate Limiting Implementation

```
class RateLimiter:
    def __init__(self):
        self.limits = {
            'yahoo': {'calls': 2000, 'period': 3600},  # 2000
calls/hour
            'alpha_vantage': {'calls': 5, 'period': 60},  # 5
calls/minute
            'polygon': {'calls': 5, 'period': 60},  # 5 calls/minute
        }
        self.call_history = defaultdict(list)

    def can_make_request(self, source):
        current_time = time.time()
        limit_config = self.limits.get(source)

        if not limit_config:
            return True

        # Clean old entries
        self.call_history[source] = [
            timestamp for timestamp in self.call_history[source]
            if current_time - timestamp < limit_config['period']
        ]

        # Check if under limit
        if len(self.call_history[source]) < limit_config['calls']:
            self.call_history[source].append(current_time)
            return True

        return False

    def get_wait_time(self, source):
        """Calculate how long to wait before next request"""
        if not self.call_history[source]:
            return 0

        limit_config = self.limits[source]
        oldest_call = min(self.call_history[source])
        wait_time = limit_config['period'] - (time.time() -
oldest_call)

        return max(0, wait_time)
```

4.4 Data Quality Framework

4.4.1 Quality Metrics

```
class DataQualityMetrics:
    @staticmethod
    def calculate_completeness(data):
        """Calculate data completeness score (0-100)"""
        total_cells = data.size
        missing_cells = data.isnull().sum().sum()
        completeness = ((total_cells - missing_cells) / total_cells) * 100
        return completeness

    @staticmethod
    def calculate_accuracy(data):
        """Check for data accuracy issues"""
        accuracy_score = 100

        # Check OHLC relationships
        invalid_high_low = (data['High'] < data['Low']).sum()
        if invalid_high_low > 0:
            accuracy_score -= 20

        invalid_close_range = ((data['Close'] > data['High']) |
                               (data['Close'] < data['Low'])).sum()
        if invalid_close_range > 0:
            accuracy_score -= 20

        # Check for negative prices
        negative_prices = (data[['Open', 'High', 'Low', 'Close']] <
                                                        0).sum().sum()
        if negative_prices > 0:
            accuracy_score -= 30

        # Check for extreme price movements (>50% in one period)
        returns = data['Close'].pct_change()
        extreme_moves = (abs(returns) > 0.5).sum()
        if extreme_moves > 0:
            accuracy_score -= (extreme_moves / len(data)) * 20

        return max(0, accuracy_score)

    @staticmethod
    def calculate_consistency(data):
        """Check data consistency"""
        consistency_score = 100

        # Check for duplicate timestamps
        duplicates = data.index.duplicated().sum()
        if duplicates > 0:
            consistency_score -= 15

        # Check for time gaps
        if len(data) > 1:
            time_diffs = pd.Series(data.index).diff()
            expected_freq = time_diffs.mode()[0]
            gaps = (time_diffs > expected_freq * 2).sum()
            if gaps > 0:
                consistency_score -= (gaps / len(data)) * 30

        return max(0, consistency_score)

    @staticmethod
    def calculate_timeliness(data, timeframe):
        """Check data freshness"""
        if data.empty:
            return 0

        latest_timestamp = data.index[-1]
        current_time = pd.Timestamp.now(tz=latest_timestamp.tz)

        # Define acceptable delays by timeframe
        acceptable_delays = {
            '1d': pd.Timedelta(hours=1),
            '5d': pd.Timedelta(hours=4),
            '1w': pd.Timedelta(hours=4),
            '1mo': pd.Timedelta(days=1),
            '3mo': pd.Timedelta(days=2),
            '6mo': pd.Timedelta(days=7),
            '1y': pd.Timedelta(days=7),
            '2y': pd.Timedelta(days=14),
            '5y': pd.Timedelta(days=30)
        }

        max_delay = acceptable_delays.get(timeframe, pd.Timedelta(days=1))
        actual_delay = current_time - latest_timestamp

        if actual_delay <= max_delay:
            return 100
        elif actual_delay <= max_delay * 2:
            return 75
        elif actual_delay <= max_delay * 3:
            return 50
        else:
            return 25
```

4.4.2 Quality Scoring System

```
class QualityScorer:
    def __init__(self):
        self.weights = {
            'completeness': 0.25,
            'accuracy': 0.35,
            'consistency': 0.25,
            'timeliness': 0.15
        }

    def calculate_quality_score(self, data, ticker, timeframe):
        """Calculate overall quality score"""
        metrics = DataQualityMetrics()

        scores = {
            'completeness': metrics.calculate_completeness(data),
            'accuracy': metrics.calculate_accuracy(data),
            'consistency': metrics.calculate_consistency(data),
            'timeliness': metrics.calculate_timeliness(data, timeframe)
        }

        # Calculate weighted average
        overall_score = sum(
            scores[metric] * weight
            for metric, weight in self.weights.items()
        )

        # Generate quality report
        report = {
            'ticker': ticker,
            'timeframe': timeframe,
            'overall_score': overall_score,
            'individual_scores': scores,
            'quality_level': self.get_quality_level(overall_score),
            'timestamp': pd.Timestamp.now().isoformat()
        }

        # Log quality issues
        if overall_score < 70:
            self.log_quality_issues(report)

        return report

    @staticmethod
    def get_quality_level(score):
        """Categorize quality score"""
        if score >= 90:
            return 'Excellent'
        elif score >= 75:
            return 'Good'
        elif score >= 60:
            return 'Acceptable'
        elif score >= 40:
            return 'Poor'
        else:
            return 'Unacceptable'

    def log_quality_issues(self, report):
        """Log quality issues for monitoring"""
        issues = []

        for metric, score in report['individual_scores'].items():
            if score < 70:
                issues.append(f"{metric}: {score:.1f}")

        if issues:
            logger.warning(
                f"Data quality issues for {report['ticker']} ({report['timeframe']}): "
                f"{', '.join(issues)}"
            )
```

4.4.3 Data Validation Rules

```
class DataValidator:
    def __init__(self):
        self.validation_rules = [
            self.validate_required_columns,
            self.validate_data_types,
            self.validate_price_relationships,
            self.validate_volume,
            self.validate_timestamps,
            self.validate_outliers
        ]

    def validate(self, data):
        """Run all validation rules"""
        errors = []
        warnings = []

        for rule in self.validation_rules:
            result = rule(data)
            if not result['valid']:
                if result['severity'] == 'error':
                    errors.extend(result['messages'])
                else:
                    warnings.extend(result['messages'])

        return {
            'is_valid': len(errors) == 0,
            'errors': errors,
            'warnings': warnings
        }

    @staticmethod
    def validate_required_columns(data):
        """Check for required columns"""
        required = ['Open', 'High', 'Low', 'Close', 'Volume']
        missing = [col for col in required if col not in data.columns]

        if missing:
            return {
                'valid': False,
                'severity': 'error',
                'messages': [f"Missing required columns: {missing}"]
            }

        return {'valid': True, 'messages': []}

    @staticmethod
    def validate_data_types(data):
        """Validate data types"""
        numeric_columns = ['Open', 'High', 'Low', 'Close', 'Volume']

        for col in numeric_columns:
            if col in data.columns and not pd.api.types.is_numeric_dtype(data[col]):
                return {
                    'valid': False,
                    'severity': 'error',
                    'messages': [f"Column {col} is not numeric"]
                }

        return {'valid': True, 'messages': []}

    @staticmethod
    def validate_price_relationships(data):
        """Validate OHLC relationships"""
        messages = []

        # High should be >= Low
        invalid_high_low = data['High'] < data['Low']
        if invalid_high_low.any():
            count = invalid_high_low.sum()
            messages.append(f"{count} rows have High < Low")

        # Close should be between High and Low
        invalid_close = (data['Close'] > data['High']) | (data['Close'] < data['Low'])
        if invalid_close.any():
            count = invalid_close.sum()
            messages.append(f"{count} rows have Close outside High-Low range")

        # Open should be between High and Low
        invalid_open = (data['Open'] > data['High']) | (data['Open'] < data['Low'])
        if invalid_open.any():
            count = invalid_open.sum()
            messages.append(f"{count} rows have Open outside High-Low range")

        if messages:
            return {
                'valid': False,
                'severity': 'warning',
                'messages': messages
            }

        return {'valid': True, 'messages': []}

    @staticmethod
    def validate_outliers(data, threshold=3):
        """Detect outliers using z-score"""
        messages = []

        # Calculate returns
        returns = data['Close'].pct_change().dropna()

        if len(returns) > 0:
            # Calculate z-scores
            z_scores = np.abs((returns - returns.mean()) / returns.std())
            outliers = z_scores > threshold

            if outliers.any():
                count = outliers.sum()
                max_return = returns[outliers].abs().max()
                messages.append(
                    f"{count} potential outliers detected "
                    f"(max return: {max_return:.1%})"
                )

        if messages:
            return {
                'valid': True,
                'severity': 'warning',
                'messages': messages
            }

        return {'valid': True, 'messages': []}
```

4.5 Feature Engineering

4.5.1 Technical Indicators Implementation

```
class TechnicalIndicators:
    @staticmethod
    def calculate_moving_averages(data, periods=[5, 10, 20, 50, 200]):
        """Calculate multiple moving averages"""
        for period in periods:
            if len(data) >= period:
                data[f'SMA_{period}'] = data['Close'].rolling(window=period).mean()
                data[f'EMA_{period}'] = data['Close'].ewm(span=period, adjust=False).mean()

        return data

    @staticmethod
    def calculate_rsi(data, period=14):
        """Calculate Relative Strength Index"""
        delta = data['Close'].diff()
        gain = (delta.where(delta > 0, 0)).rolling(window=period).mean()
        loss = (-delta.where(delta < 0, 0)).rolling(window=period).mean()

        rs = gain / loss.replace(0, 1e-10)
        rsi = 100 - (100 / (1 + rs))

        data['RSI'] = rsi.fillna(50)  # Default to neutral

        # RSI-based signals
        data['RSI_Oversold'] = (data['RSI'] < 30).astype(int)
        data['RSI_Overbought'] = (data['RSI'] > 70).astype(int)

        return data

    @staticmethod
    def calculate_macd(data, fast=12, slow=26, signal=9):
        """Calculate MACD"""
        exp1 = data['Close'].ewm(span=fast, adjust=False).mean()
        exp2 = data['Close'].ewm(span=slow, adjust=False).mean()

        data['MACD'] = exp1 - exp2
        data['MACD_Signal'] = data['MACD'].ewm(span=signal, adjust=False).mean()
        data['MACD_Histogram'] = data['MACD'] - data['MACD_Signal']

        # MACD crossover signals
        data['MACD_Bullish'] = (
            (data['MACD'] > data['MACD_Signal']) &
            (data['MACD'].shift(1) <= data['MACD_Signal'].shift(1))
        ).astype(int)

        data['MACD_Bearish'] = (
            (data['MACD'] < data['MACD_Signal']) &
            (data['MACD'].shift(1) >= data['MACD_Signal'].shift(1))
        ).astype(int)

        return data

    @staticmethod
    def calculate_bollinger_bands(data, period=20, std_dev=2):
        """Calculate Bollinger Bands"""
        sma = data['Close'].rolling(window=period).mean()
        std = data['Close'].rolling(window=period).std()

        data['BB_Upper'] = sma + (std * std_dev)
        data['BB_Middle'] = sma
        data['BB_Lower'] = sma - (std * std_dev)
        data['BB_Width'] = (data['BB_Upper'] - data['BB_Lower']) / data['BB_Middle']
        data['BB_Position'] = (data['Close'] - data['BB_Lower']) / (data['BB_Upper'] - data['BB_Lower'])

        # Bollinger Band signals
        data['BB_Squeeze'] = (data['BB_Width'] < data['BB_Width'].rolling(20).mean()).astype(int)

        return data

    @staticmethod
    def calculate_stochastic(data, period=14, smooth_k=3, smooth_d=3):
        """Calculate Stochastic Oscillator"""
        lowest_low = data['Low'].rolling(window=period).min()
        highest_high = data['High'].rolling(window=period).max()

        k_percent = 100 * ((data['Close'] - lowest_low) / (highest_high - lowest_low))
        data['Stoch_K'] = k_percent.rolling(window=smooth_k).mean()
        data['Stoch_D'] = data['Stoch_K'].rolling(window=smooth_d).mean()

        # Stochastic signals
        data['Stoch_Oversold'] = (data['Stoch_K'] < 20).astype(int)
        data['Stoch_Overbought'] = (data['Stoch_K'] > 80).astype(int)

        return data
```

4.5.2 Advanced Features

```
class AdvancedFeatures:
    @staticmethod
    def calculate_volatility_features(data):
        """Calculate volatility-based features"""
        # Historical volatility
        data['Return'] = data['Close'].pct_change()
        data['Volatility_10'] = data['Return'].rolling(window=10).std()
        data['Volatility_20'] = data['Return'].rolling(window=20).std()
        data['Volatility_50'] = data['Return'].rolling(window=50).std()

        # Parkinson volatility (using high-low)
        data['Parkinson_Vol'] = np.sqrt(
            np.log(data['High'] / data['Low']) ** 2 / (4 * np.log(2))
        ).rolling(window=20).mean()

        # Garman-Klass volatility
        data['GK_Vol'] = np.sqrt(
            0.5 * np.log(data['High'] / data['Low']) ** 2 -
            (2 * np.log(2) - 1) * np.log(data['Close'] / data['Open']) ** 2
        ).rolling(window=20).mean()

        # Average True Range (ATR)
        high_low = data['High'] - data['Low']
        high_close = np.abs(data['High'] - data['Close'].shift())
        low_close = np.abs(data['Low'] - data['Close'].shift())

        true_range = pd.concat([high_low, high_close, low_close], axis=1).max(axis=1)
        data['ATR'] = true_range.rolling(window=14).mean()

        data['Vol_Ratio'] = data['Volatility_10'] / data['Volatility_50']

        return data

    @staticmethod
    def calculate_volume_features(data):
        """Calculate volume-based features"""
        # Volume moving averages
        data['Volume_MA_10'] = data['Volume'].rolling(window=10).mean()
        data['Volume_MA_20'] = data['Volume'].rolling(window=20).mean()

        # Volume ratio
        data['Volume_Ratio'] = data['Volume'] / data['Volume_MA_20']

        # On-Balance Volume (OBV)
        obv = []
        obv_value = 0

        for i in range(len(data)):
            if i == 0:
                obv.append(0)
            else:
                if data['Close'].iloc[i] > data['Close'].iloc[i-1]:
                    obv_value += data['Volume'].iloc[i]
                elif data['Close'].iloc[i] < data['Close'].iloc[i-1]:
                    obv_value -= data['Volume'].iloc[i]
                obv.append(obv_value)

        data['OBV'] = obv
        data['OBV_MA'] = data['OBV'].rolling(window=20).mean()

        # Volume-Price Trend (VPT)
        data['VPT'] = (data['Volume'] * data['Return']).cumsum()

        # Accumulation/Distribution Line
        clv = ((data['Close'] - data['Low']) - (data['High'] - data['Close'])) / (data['High'] - data['Low'])
        data['ADL'] = (clv * data['Volume']).cumsum()

        # Money Flow Index (MFI)
        typical_price = (data['High'] + data['Low'] + data['Close']) / 3
        money_flow = typical_price * data['Volume']

        positive_flow = money_flow.where(typical_price > typical_price.shift(1), 0)
        negative_flow = money_flow.where(typical_price < typical_price.shift(1), 0)

        positive_flow_sum = positive_flow.rolling(window=14).sum()
        negative_flow_sum = negative_flow.rolling(window=14).sum()

        money_ratio = positive_flow_sum / negative_flow_sum.replace(0, 1e-10)
        data['MFI'] = 100 - (100 / (1 + money_ratio))

        return data

    @staticmethod
    def calculate_pattern_features(data):
        """Calculate pattern-based features"""
        # Price patterns
        data['Higher_High'] = (
            (data['High'] > data['High'].shift(1)) &
            (data['High'].shift(1) > data['High'].shift(2))
        ).astype(int)

        data['Lower_Low'] = (
            (data['Low'] < data['Low'].shift(1)) &
            (data['Low'].shift(1) < data['Low'].shift(2))
        ).astype(int)

        # Candlestick patterns
        body = data['Close'] - data['Open']
        body_size = np.abs(body)
        shadow_range = data['High'] - data['Low']

        # Doji (small body)
        data['Doji'] = (body_size <= shadow_range * 0.1).astype(int)

        # Hammer (small body at top, long lower shadow)
        data['Hammer'] = (
            (body_size <= shadow_range * 0.3) &
            (data['Low'] < data['Open'] - body_size * 2) &
            (data['Close'] > data['Open'])
        ).astype(int)

        # Shooting Star (small body at bottom, long upper shadow)
        data['Shooting_Star'] = (
            (body_size <= shadow_range * 0.3) &
            (data['High'] > data['Close'] + body_size * 2) &
            (data['Close'] < data['Open'])
        ).astype(int)

        # Gap detection
        data['Gap_Up'] = (data['Low'] > data['High'].shift(1)).astype(int)
        data['Gap_Down'] = (data['High'] < data['Low'].shift(1)).astype(int)

        window = 20
        data['Resistance'] = data['High'].rolling(window=window).max()
        data['Support'] = data['Low'].rolling(window=window).min()
        data['Price_to_Resistance'] = data['Close'] / data['Resistance']
        data['Price_to_Support'] = data['Close'] / data['Support']

        return data
```

4.5.3 Feature Selection and Importance

```
class FeatureSelector:
    def __init__(self):
        self.feature_importance = {}

    def calculate_feature_importance(self, X, y, feature_names):
        """Calculate feature importance using Random Forest"""
        from sklearn.ensemble import RandomForestClassifier

        rf = RandomForestClassifier(n_estimators=100, random_state=42)
        rf.fit(X, y)

        importance_scores = rf.feature_importances_

        self.feature_importance = dict(zip(feature_names, importance_scores))

        # Sort by importance
        sorted_features = sorted(
            self.feature_importance.items(),
            key=lambda x: x[1],
            reverse=True
        )

        return sorted_features

    def select_top_features(self, n_features=20):
        """Select top N features by importance"""
        sorted_features = sorted(
            self.feature_importance.items(),
            key=lambda x: x[1],
            reverse=True
        )

        return [feature for feature, _ in sorted_features[:n_features]]

    def remove_correlated_features(self, data, threshold=0.95):
        """Remove highly correlated features"""
        corr_matrix = data.corr().abs()

        # Select upper triangle of correlation matrix
        upper = corr_matrix.where(
            np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
        )

        # Find features with correlation greater than threshold
        to_drop = [column for column in upper.columns if any(upper[column] >
                       threshold)]

        logger.info(f"Removing {len(to_drop)} highly correlated features")

        return data.drop(columns=to_drop)
```

4.6 Data Storage Architecture

4.6.1 Database Schema Design

```
-- Main stock data table
CREATE TABLE stock_data (
    id BIGSERIAL PRIMARY KEY,
    ticker VARCHAR(20) NOT NULL,
    timestamp TIMESTAMP WITH TIME ZONE NOT NULL,
    open DECIMAL(10, 2) NOT NULL,
    high DECIMAL(10, 2) NOT NULL,
    low DECIMAL(10, 2) NOT NULL,
    close DECIMAL(10, 2) NOT NULL,
    adjusted_close DECIMAL(10, 2),
    volume BIGINT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(ticker, timestamp)
);

-- Indexes for performance
CREATE INDEX idx_stock_data_ticker ON stock_data(ticker);
CREATE INDEX idx_stock_data_timestamp ON stock_data(timestamp);
CREATE INDEX idx_stock_data_ticker_timestamp ON stock_data(ticker, timestamp);

-- Technical indicators table
CREATE TABLE technical_indicators (
    id BIGSERIAL PRIMARY KEY,
    stock_data_id BIGINT REFERENCES stock_data(id) ON DELETE CASCADE,
    rsi DECIMAL(5, 2),
    macd DECIMAL(10, 4),
    macd_signal DECIMAL(10, 4),
    macd_histogram DECIMAL(10, 4),
    bb_upper DECIMAL(10, 2),
    bb_middle DECIMAL(10, 2),
    bb_lower DECIMAL(10, 2),
    sma_20 DECIMAL(10, 2),
    sma_50 DECIMAL(10, 2),
    sma_200 DECIMAL(10, 2),
    ema_12 DECIMAL(10, 2),
    ema_26 DECIMAL(10, 2),
    atr DECIMAL(10, 4),
    obv BIGINT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Data quality tracking
CREATE TABLE data_quality_logs (
    id BIGSERIAL PRIMARY KEY,
    ticker VARCHAR(20) NOT NULL,
    timeframe VARCHAR(10) NOT NULL,
    quality_score DECIMAL(5, 2),
    completeness_score DECIMAL(5, 2),
    accuracy_score DECIMAL(5, 2),
    consistency_score DECIMAL(5, 2),
    timeliness_score DECIMAL(5, 2),
    data_points INTEGER,
    issues TEXT[],
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Cache management table
CREATE TABLE cache_entries (
    id BIGSERIAL PRIMARY KEY,
    cache_key VARCHAR(255) UNIQUE NOT NULL,
    cache_value JSONB,
    expires_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    accessed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    access_count INTEGER DEFAULT 1
);

-- Create function to auto-update updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ language 'plpgsql';

-- Create trigger for updated_at
CREATE TRIGGER update_stock_data_updated_at
    BEFORE UPDATE ON stock_data
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at_column();
```

4.6.2 Data Partitioning Strategy

```
-- Partition stock_data table by month for better performance
CREATE TABLE stock_data_2024_01 PARTITION OF stock_data
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE stock_data_2024_02 PARTITION OF stock_data
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

-- Auto-create partitions function
CREATE OR REPLACE FUNCTION create_monthly_partition()
RETURNS void AS $$
DECLARE
    start_date date;
    end_date date;
    partition_name text;
BEGIN
    start_date := date_trunc('month', CURRENT_DATE);
    end_date := start_date + interval '1 month';
    partition_name := 'stock_data_' || to_char(start_date, 'YYYY_MM');

    EXECUTE format('CREATE TABLE IF NOT EXISTS %I PARTITION OF stock_data
                    FOR VALUES FROM (%L) TO (%L)',
                    partition_name, start_date, end_date);
END;
$$ LANGUAGE plpgsql;
```

4.6.3 Data Access Layer

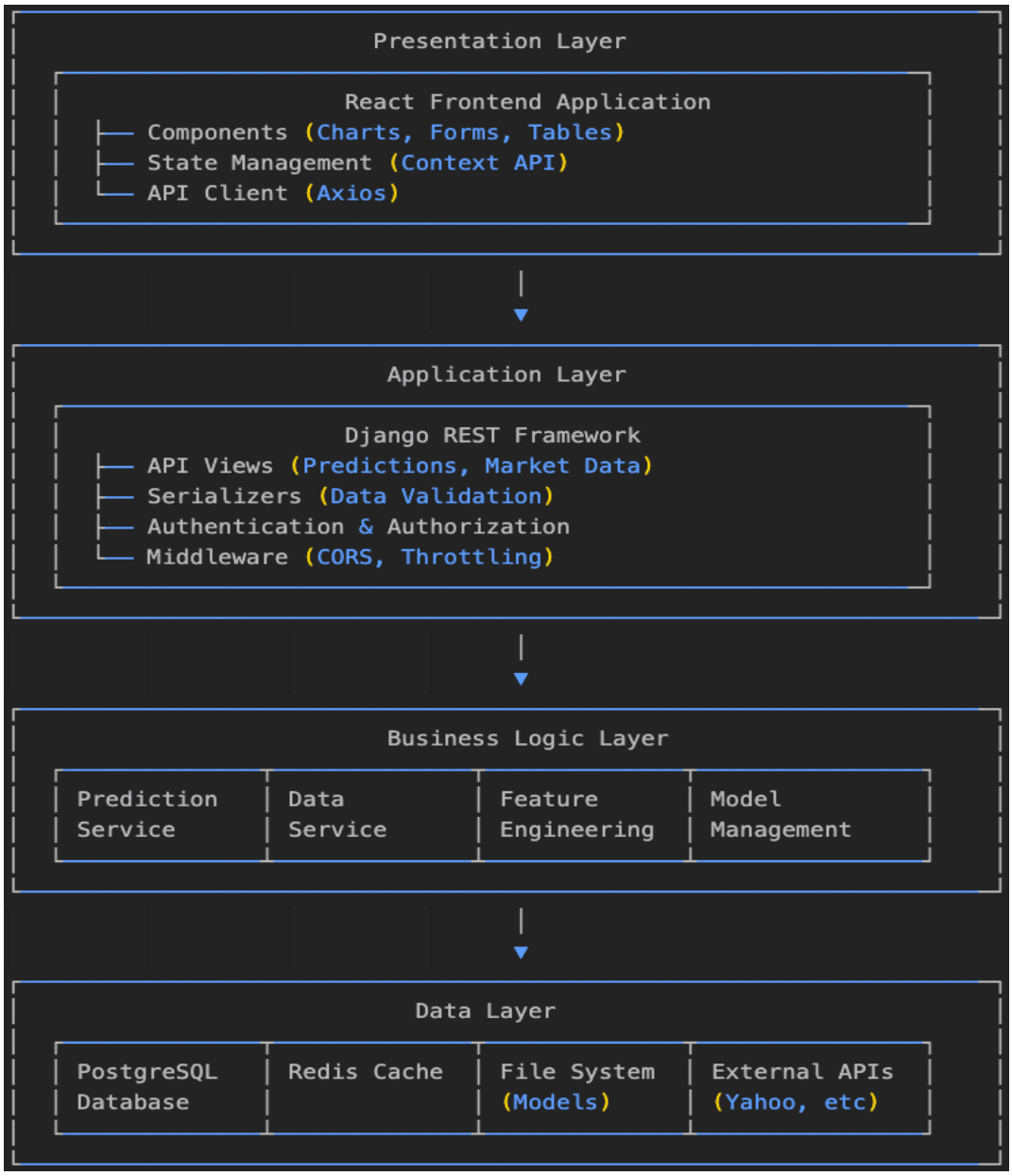
```
class DataAccessLayer:
    def __init__(self, connection_string):
        self.engine = create_engine(connection_string)
        self.Session = sessionmaker(bind=self.engine)

    def save_stock_data(self, ticker, data):
        """Save stock data to database"""
        session = self.Session()

        try:
            # Prepare data for insertion
            records = []
            for timestamp, row in data.iterrows():
                record = {
                    'ticker': ticker,
                    'timestamp': timestamp,
                    'open': row['Open'],
                    'high': row['High'],
                    'low': row['Low'],
                    'close': row['Close'],
                    'adjusted_close': row.get('Adj Close', row['Close']),
                    'volume': row['Volume']
                }
                records.append(record)

            # Bulk insert with conflict resolution
            stmt = insert(StockData).values(records)
            stmt = stmt.on_conflict_do_update(
                index_elements=['ticker', 'timestamp'],
                set_={
                    'open': stmt.excluded.open,
                    'high': stmt.excluded.high,
                    'low': stmt.excluded.low,
                    'close': stmt.excluded.close,
                    'adjusted_close': stmt.excluded.adjusted_close,
                    'volume': stmt.excluded.volume,
                    'updated_at': func.now()
                }
            )

            session.execute(stmt)
            session.commit()

            logger.info(f"Saved {len(records)} records for {ticker}")

        except Exception as e:
            session.rollback()
            logger.error(f"Failed to save data for {ticker}: {str(e)}")
            raise

        finally:
            session.close()

    def get_stock_data(self, ticker, start_date, end_date):
        """Retrieve stock data from database"""
        session = self.Session()

        try:
            query = session.query(StockData).filter(
                StockData.ticker == ticker,
                StockData.timestamp >= start_date,
                StockData.timestamp <= end_date
            ).order_by(StockData.timestamp)
            df = pd.read_sql(query.statement, session.bind, index_col='timestamp')

            return df

        finally:
            session.close()
```

CHAPTER 5: SYSTEM DESIGN

5.1 System Architecture

5.1.1 High-Level Architecture

The StockVibePredictor system follows a multi-tier architecture pattern designed for scalability, maintainability, and high performance :



5.1.2 Component Architecture

```
SYSTEM_COMPONENTS = {
    'frontend': {
        'technology': 'React 18.2',
        'responsibilities': [
            'User interface rendering',
            'User interaction handling',
            'Data visualization',
            'State management',
            'API communication'
        ],
        'key_libraries': [
            'Chart.js for visualizations',
            'Axios for HTTP requests',
            'Material-UI for components',
            'React Router for navigation'
        ]
    },
    'backend': {
        'technology': 'Django 5.2.5',
        'responsibilities': [
            'API endpoint management',
            'Request validation',
            'Business logic orchestration',
            'Authentication/Authorization',
            'Response formatting'
        ],
        'key_libraries': [
            'Django REST Framework',
            'Django CORS Headers',
            'Django Redis',
            'Celery for async tasks'
        ]
    },
    'ml_service': {
        'technology': 'Scikit-learn 1.7.1',
        'responsibilities': [
            'Model training',
            'Prediction generation',
            'Feature engineering',
            'Model evaluation',
            'Model versioning'
        ],
        'key_libraries': [
            'NumPy for computations',
            'Pandas for data manipulation',
            'Joblib for model persistence',
            'Matplotlib for analysis'
        ]
    },
    'data_service': {
        'technology': 'Python + APIs',
        'responsibilities': [
            'Data fetching',
            'Data validation',
            'Data transformation',
            'Cache management',
            'External API integration'
        ],
        'key_libraries': [
            'yfinance for market data',
            'Requests for HTTP calls',
            'AsyncIO for concurrent ops',
            'APScheduler for scheduling'
        ]
    },
    'database': {
        'technology': 'PostgreSQL 14',
        'responsibilities': [
            'Data persistence',
            'Transaction management',
            'Query optimization',
            'Data integrity',
            'Backup and recovery'
        ],
        'features': [
            'Table partitioning',
            'Indexing strategies',
            'Connection pooling',
            'Replication support'
        ]
    },
    'cache': {
        'technology': 'Redis 7.0',
        'responsibilities': [
            'Response caching',
            'Session management',
            'Rate limiting',
            'Real-time data buffering',
            'Distributed locking'
        ],
        'features': [
            'Key expiration',
            'Pub/Sub messaging',
            'Data persistence',
            'Cluster support'
        ]
    }
}
```

5.1.3 Deployment Architecture

```
# Docker Compose Configuration
version: '3.8'

services:
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      - REACT_APP_API_URL=http://backend:8000
    depends_on:
      - backend
    networks:
      - stockvibe-network

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@postgres:5432/stockvibe
      - REDIS_URL=redis://redis:6379
      - SECRET_KEY=${SECRET_KEY}
    depends_on:
      - postgres
      - redis
    volumes:
      - ./backend:/app
      - model-storage:/app/models
    networks:
      - stockvibe-network

  postgres:
    image: postgres:14-alpine
    environment:
      - POSTGRES_DB=stockvibe
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
    volumes:
      - postgres-data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    networks:
      - stockvibe-network

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    networks:
      - stockvibe-network

  celery:
    build:
      context: ./backend
      dockerfile: Dockerfile
    command: celery -A stockvibe worker -l info
    environment:
      - DATABASE_URL=postgresql://user:pass@postgres:5432/stockvibe
      - REDIS_URL=redis://redis:6379
    depends_on:
      - backend
      - redis
    networks:
      - stockvibe-network

  celery-beat:
    build:
      context: ./backend
      dockerfile: Dockerfile
    command: celery -A stockvibe beat -l info
    environment:
      - DATABASE_URL=postgresql://user:pass@postgres:5432/stockvibe
      - REDIS_URL=redis://redis:6379
    depends_on:
      - backend
      - redis
    networks:
      - stockvibe-network

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
      - ./nginx/ssl:/etc/nginx/ssl
    depends_on:
      - frontend
      - backend
    networks:
      - stockvibe-network

volumes:
  postgres-data:
  redis-data:
  model-storage:

networks:
  stockvibe-network:
    driver: bridge
```

5.1.4 Microservices Communication

```
class ServiceCommunication:
    """
    Manages inter-service communication patterns
    """

    def __init__(self):
        self.service_registry = {
            'prediction_service': 'http://prediction-service:5001',
            'data_service': 'http://data-service:5002',
            'model_service': 'http://model-service:5003',
            'notification_service': 'http://notification-service:5004'
        }

        self.circuit_breaker = CircuitBreaker()
        self.retry_policy = RetryPolicy()

    async def call_service(self, service_name, endpoint, method='GET', data=None):
        """
        Call a microservice with circuit breaker and retry logic
        """
        service_url = self.service_registry.get(service_name)

        if not service_url:
            raise ValueError(f"Service {service_name} not found")

        url = f"{service_url}{endpoint}"

        # Check circuit breaker status
        if self.circuit_breaker.is_open(service_name):
            raise ServiceUnavailableError(f"Service {service_name} is unavailable")

        # Attempt request with retry
        for attempt in range(self.retry_policy.max_attempts):
            try:
                response = await self._make_request(url, method, data)

                # Reset circuit breaker on success
                self.circuit_breaker.record_success(service_name)

                return response

            except Exception as e:
                self.circuit_breaker.record_failure(service_name)

                if attempt == self.retry_policy.max_attempts - 1:
                    raise

                await asyncio.sleep(self.retry_policy.get_delay(attempt))

    async def _make_request(self, url, method, data):
        """
        Make HTTP request to service
        """
        timeout = aiohttp.ClientTimeout(total=30)

        async with aiohttp.ClientSession(timeout=timeout) as session:
            if method == 'GET':
                async with session.get(url) as response:
                    return await response.json()
            elif method == 'POST':
                async with session.post(url, json=data) as response:
                    return await response.json()
```

5.2 Component Design

5.2.1 Frontend Components

```
// Component Hierarchy
const ComponentStructure = {
  App: {
    Router: {
      Layout: {
        Header: {
          Navigation: ["Logo", "Menu", "UserProfile"],
          SearchBar: ["AutoComplete", "SearchButton"],
        },
        MainContent: {
          Dashboard: {
            MarketOverview: ["IndexCards", "TrendIndicator"],
            PredictionPanel: {
              TickerInput: ["Dropdown", "Validation"],
              TimeframeSelector: ["RadioButtons"],
              PredictionResults: [
                "DirectionCard",
                "ConfidenceGauge",
                "PriceTarget",
              ],
            },
            ChartContainer: {
              ChartControls: ["ChartType", "Indicators", "TimeRange"],
              Chart: ["CandlestickChart", "VolumeChart", "IndicatorOverlays"],
            },
          },
          Portfolio: {
            Holdings: ["PositionList", "Performance"],
            Watchlist: ["WatchlistManager", "QuickActions"],
            TradeHistory: ["TradeTable", "Filters"],
          },
          Analysis: {
            TechnicalAnalysis: ["IndicatorPanel", "SignalsList"],
            RiskMetrics: ["VaRChart", "DrawdownChart"],
            Backtesting: ["StrategyBuilder", "Results"],
          },
        },
        Footer: ["Links", "Copyright", "Status"],
      },
    },
  },
};

// Main Dashboard Component
import React, { useState, useEffect } from "react";
import { Grid, Paper, Typography } from "@mui/material";
import { Chart } from "react-chartjs-2";
import axios from "axios";

const Dashboard = () => {
  const [selectedTicker, setSelectedTicker] = useState("AAPL");
  const [predictions, setPredictions] = useState(null);
  const [chartData, setChartData] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  useEffect(() => {
    fetchPredictions();
    fetchChartData();
  }, [selectedTicker]);
  const fetchPredictions = async () => {
    setLoading(true);
    try {
      const response = await axios.post("/api/predict/multi-timeframe/", {
        ticker: selectedTicker,
        timeframes: ["1d", "1w", "1mo", "1y"],
        include_analysis: true,
      });
      setPredictions(response.data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };
  const fetchChartData = async () => {
    try {
      const response = await axios.get("/api/market/chart/${selectedTicker}/", {
        params: {
          timeframe: "1mo",
          indicators: "sma20,sma50,rsi,macd",
        },
      });
      setChartData(response.data);
    } catch (err) {
      console.error("Chart data fetch failed:", err);
    }
  };
  return (
    <Grid container spacing={3}>
      <Grid item xs={12} md={8}>
        <Paper elevation={3} sx={{ p: 2 }}>
          <Typography variant="h5">Price Chart</Typography>
          {chartData && (
            <Chart type="candlestick" data={chartData} options={chartOptions} />
          )}
        </Paper>
      </Grid>
      <Grid item xs={12} md={4}>
        <Paper elevation={3} sx={{ p: 2 }}>
          <Typography variant="h5">Predictions</Typography>
          {predictions && <PredictionDisplay predictions={predictions} />}
        </Paper>
      </Grid>
    </Grid>
  );
};
```

5.2.2 Backend Service Components

```
class PredictionService:
    """
    Core prediction service handling all prediction-related operations
    """

    def __init__(self):
        self.model_manager = ModelManager()
        self.feature_engineer = FeatureEngineer()
        self.cache_manager = CacheManager()
        self.performance_tracker = PerformanceTracker()

    def generate_prediction(self, ticker, timeframe, data=None):
        """
        Generate prediction for a specific ticker and timeframe
        """
        # Check cache first
        cache_key = f"prediction_{ticker}_{timeframe}"
        cached_result = self.cache_manager.get(cache_key)

        if cached_result:
            logger.info(f"Cache hit for {cache_key}")
            return cached_result

        try:
            # Fetch data if not provided
            if data is None:
                data = self.fetch_data(ticker, timeframe)

            # Engineer features
            features = self.feature_engineer.compute_features(data)

            # Get appropriate model
            model = self.model_manager.get_model(ticker, timeframe)

            if not model:
                raise ModelNotFoundError(f"No model available for {ticker} {timeframe}")

            # Generate prediction
            prediction = self._make_prediction(model, features)

            # Calculate confidence and additional metrics
            result = self._enhance_prediction(prediction, data, model)

            # Cache result
            self.cache_manager.set(cache_key, result, ttl=TIMEFRAME_CONFIGS[timeframe]['cache_ttl'])

            # Track performance
            self.performance_tracker.track_prediction(ticker, timeframe, result)

            return result

        except Exception as e:
            logger.error(f"Prediction failed for {ticker} {timeframe}: {str(e)}")
            raise PredictionError(str(e))

    def _make_prediction(self, model_info, features):
        """
        Execute model prediction
        """
        model = model_info['model']
        scaler = model_info.get('scaler')
        required_features = model_info['features']

        # Prepare feature vector
        feature_vector = self._prepare_features(features, required_features)

        # Scale features if scaler available
        if scaler:
            feature_vector = scaler.transform(feature_vector)

        # Make prediction
        prediction = model.predict(feature_vector)[0]

        # Get probability if available
        confidence = 0.5
        if hasattr(model, 'predict_proba'):
            proba = model.predict_proba(feature_vector)[0]
            confidence = proba[int(prediction)]

        return {
            'direction': 'UP' if prediction == 1 else 'DOWN',
            'confidence': confidence,
            'raw_prediction': prediction
        }

    def _enhance_prediction(self, prediction, data, model_info):
        """
        Enhance prediction with additional metrics
        """
        current_price = float(data['Close'].iloc[-1])
        volatility = float(data['Volatility'].iloc[-1]) if 'Volatility' in data else 0.02

        # Calculate price target
        if prediction['direction'] == 'UP':
            price_target = current_price * (1 + volatility)
        else:
            price_target = current_price * (1 - volatility)

        # Calculate expected return
        expected_return = ((price_target / current_price) - 1) * 100

        return {
            'direction': prediction['direction'],
            'confidence': round(prediction['confidence'] * 100, 2),
            'price_target': round(price_target, 2),
            'current_price': round(current_price, 2),
            'expected_return': round(expected_return, 2),
            'model_accuracy': round(model_info.get('accuracy', 0.5) * 100, 2),
            'model_type': model_info.get('type', 'unknown'),
            'timestamp': datetime.now().isoformat()
        }
```

5.2.3 Data Service Component

```
class DataService:
    """
    Centralized data management service
    """

    def __init__(self):
        self.data_sources = {
            'yahoo': YahooDataSource(),
            'alpha_vantage': AlphaVantageDataSource(),
            'polygon': PolygonDataSource()
        }
        self.validator = DataValidator()
        self.transformer = DataTransformer()
        self.storage = DataStorage()

    async def fetch_data(self, ticker, timeframe, source='auto'):
        """
        Fetch data with automatic source selection and fallback
        """
        if source == 'auto':
            source = self._select_best_source(ticker, timeframe)

        # Try primary source
        try:
            data = await self.data_sources[source].fetch(ticker, timeframe)

            if data is not None and not data.empty:
                # Validate data
                if self.validator.validate(data):
                    # Transform data
                    transformed = self.transformer.transform(data, timeframe)

                    # Store for future use
                    await self.storage.store(ticker, timeframe, transformed)

                    return transformed

        except Exception as e:
            logger.warning(f"Primary source {source} failed: {str(e)}")

        # Fallback to other sources
        for fallback_source in self.data_sources:
            if fallback_source != source:
                try:
                    data = await self.data_sources[fallback_source].fetch(ticker, timeframe)

                    if data is not None and not data.empty:
                        if self.validator.validate(data):
                            transformed = self.transformer.transform(data, timeframe)
                            await self.storage.store(ticker, timeframe, transformed)
                            return transformed

                except Exception as e:
                    logger.warning(f"Fallback source {fallback_source} failed: {str(e)}")

        # Last resort: return cached data
        cached_data = await self.storage.get_cached(ticker, timeframe)
        if cached_data is not None:
            logger.warning(f"Using cached data for {ticker} {timeframe}")
            return cached_data

        raise DataUnavailableError(f"No data available for {ticker} {timeframe}")

    def _select_best_source(self, ticker, timeframe):
        """
        Select the best data source based on ticker and timeframe
        """
        # Indian stocks only available on Yahoo
        if ticker.endswith('.NS') or ticker.endswith('.BO'):
            return 'yahoo'

        # Crypto best from Yahoo
        if '-USD' in ticker:
            return 'yahoo'

        # Intraday data preferred from Polygon
        if timeframe in ['1d', '5d']:
            return 'polygon' if self._check_api_limits('polygon') else 'yahoo'

        # Default to Yahoo
        return 'yahoo'
```

5.3 Database Design

5.3.1 Entity Relationship Diagram

```
-- User Management Schema
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    is_active BOOLEAN DEFAULT true,
    is_verified BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_login TIMESTAMP,
    preferences JSONB DEFAULT '{}'::jsonb
);

-- User Sessions
CREATE TABLE user_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    token_hash VARCHAR(255) UNIQUE NOT NULL,
    ip_address INET,
    user_agent TEXT,
    expires_at TIMESTAMP NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Watchlists
CREATE TABLE watchlists (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    tickers TEXT[] NOT NULL,
    is_public BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(user_id, name)
);

-- Portfolio Holdings
CREATE TABLE portfolio_holdings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    ticker VARCHAR(20) NOT NULL,
    quantity DECIMAL(15, 4) NOT NULL,
    average_price DECIMAL(15, 4) NOT NULL,
    total_invested DECIMAL(15, 2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(user_id, ticker)
);

-- Trade History
CREATE TABLE trades (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    ticker VARCHAR(20) NOT NULL,
    trade_type VARCHAR(10) CHECK (trade_type IN ('BUY', 'SELL')),
    quantity DECIMAL(15, 4) NOT NULL,
    price DECIMAL(15, 4) NOT NULL,
    total_value DECIMAL(15, 2) NOT NULL,
    commission DECIMAL(10, 2) DEFAULT 0,
    notes TEXT,
    executed_at TIMESTAMP NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Predictions Log
CREATE TABLE predictions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    ticker VARCHAR(20) NOT NULL,
    timeframe VARCHAR(10) NOT NULL,
    direction VARCHAR(10) NOT NULL,
    confidence DECIMAL(5, 2) NOT NULL,
    price_target DECIMAL(15, 4),
```

```
    current_price DECIMAL(15, 4) NOT NULL,
    expected_return DECIMAL(8, 4),
    model_version VARCHAR(50),
    model_accuracy DECIMAL(5, 2),
    features_used JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_predictions_ticker (ticker),
    INDEX idx_predictions_user (user_id),
    INDEX idx_predictions_created (created_at DESC)
);

-- Model Registry
CREATE TABLE models (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_key VARCHAR(100) UNIQUE NOT NULL,
    model_type VARCHAR(50) NOT NULL,
    ticker VARCHAR(20),
    timeframe VARCHAR(10) NOT NULL,
    algorithm VARCHAR(50) NOT NULL,
    version VARCHAR(50) NOT NULL,
    file_path TEXT NOT NULL,
    accuracy DECIMAL(5, 4),
    precision DECIMAL(5, 4),
    recall DECIMAL(5, 4),
    f1_score DECIMAL(5, 4),
    training_samples INTEGER,
    features TEXT[],
    hyperparameters JSONB,
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    trained_at TIMESTAMP,
    INDEX idx_models_ticker_timeframe (ticker, timeframe),
    INDEX idx_models_accuracy (accuracy DESC)
);

-- Performance Metrics
CREATE TABLE model_performance (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_id UUID REFERENCES models(id) ON DELETE CASCADE,
    evaluation_date DATE NOT NULL,
    predictions_made INTEGER DEFAULT 0,
    correct_predictions INTEGER DEFAULT 0,
    accuracy DECIMAL(5, 4),
    average_confidence DECIMAL(5, 4),
    total_return DECIMAL(10, 4),
    sharpe_ratio DECIMAL(8, 4),
    max_drawdown DECIMAL(8, 4),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(model_id, evaluation_date)
);

-- API Usage Tracking
CREATE TABLE api_usage (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    endpoint VARCHAR(255) NOT NULL,
    method VARCHAR(10) NOT NULL,
    ip_address INET,
    user_agent TEXT,
    request_body JSONB,
    response_status INTEGER,
    response_time_ms INTEGER,
    error_message TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_api_usage_user (user_id),
    INDEX idx_api_usage_endpoint (endpoint),
    INDEX idx_api_usage_created (created_at DESC)
);

-- System Alerts
CREATE TABLE system_alerts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    alert_type VARCHAR(50) NOT NULL,
    severity VARCHAR(20) CHECK (severity IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
    component VARCHAR(100),
    message TEXT NOT NULL,
    details JSONB,
    is_resolved BOOLEAN DEFAULT false,
    resolved_at TIMESTAMP,
    resolved_by UUID REFERENCES users(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.3.2 Indexing Strategy

```
-- Performance optimization indexes
CREATE INDEX idx_stock_data_ticker_timestamp
    ON stock_data(ticker, timestamp DESC);

CREATE INDEX idx_stock_data_timestamp
    ON stock_data(timestamp DESC);

CREATE INDEX idx_technical_indicators_stock_data
    ON technical_indicators(stock_data_id);

CREATE INDEX idx_predictions_ticker_timeframe
    ON predictions(ticker, timeframe, created_at DESC);

CREATE INDEX idx_trades_user_executed
    ON trades(user_id, executed_at DESC);

CREATE INDEX idx_api_usage_user_created
    ON api_usage(user_id, created_at DESC);

-- Partial indexes for specific queries
CREATE INDEX idx_active_models
    ON models(ticker, timeframe)
    WHERE is_active = true;

CREATE INDEX idx_recent_predictions
    ON predictions(ticker, created_at DESC)
    WHERE created_at > CURRENT_DATE - INTERVAL '7 days';

-- Full-text search indexes
CREATE INDEX idx_users_search
    ON users USING gin(to_tsvector('english',
        coalesce(username, '') || ' ' ||
        coalesce(email, '') || ' ' ||
        coalesce(first_name, '') || ' ' ||
        coalesce(last_name, '')));
```

5.3.3 Database Optimization

```
class DatabaseOptimizer:
    """
    Database optimization and maintenance utilities
    """

    def __init__(self, connection_string):
        self.engine = create_engine(connection_string)

    def analyze_query_performance(self, query):
        """
        Analyze query execution plan
        """
        with self.engine.connect() as conn:
            result = conn.execute(f"EXPLAIN ANALYZE {query}")
            plan = result.fetchall()

            # Parse execution plan
            total_time = 0
            slow_operations = []

            for row in plan:
                line = row[0]

                # Extract execution time
                if 'actual time=' in line:
                    time_match = re.search(r'actual
                                                time=([\d.]+)\.\.([\d.]+)', line)
                    if time_match:
                        operation_time = float(time_match.group(2))

                        if operation_time > 100:  # More than 100ms
                            slow_operations.append({
                                'operation': line,
                                'time': operation_time
                            })

                # Check for sequential scans
                if 'Seq Scan' in line:
                    logger.warning(f"Sequential scan detected: {line}")

            return {
                'total_time': total_time,
                'slow_operations': slow_operations,
                'plan': plan
            }

    def optimize_table(self, table_name):
        """
        Run optimization on a table
        """
        with self.engine.connect() as conn:
            # Update statistics
            conn.execute(f"ANALYZE {table_name}")

            # Reindex if needed
            conn.execute(f"REINDEX TABLE {table_name}")

            # Vacuum to reclaim space
            conn.execute(f"VACUUM ANALYZE {table_name}")

            logger.info(f"Optimization completed for table {table_name}")

    def get_table_statistics(self, table_name):
        """
        Get detailed table statistics
        """
        query = f"""
        SELECT
            pg_size_pretty(pg_total_relation_size('{table_name}')) as
                                                total_size,
            pg_size_pretty(pg_relation_size('{table_name}')) as table_size,
            pg_size_pretty(pg_indexes_size('{table_name}')) as indexes_size,
            n_live_tup as row_count,
            n_dead_tup as dead_rows,
            last_vacuum,
            last_autovacuum,
            last_analyze,
            last_autoanalyze
        FROM pg_stat_user_tables
        WHERE relname = '{table_name}'
        """

        with self.engine.connect() as conn:
            result = conn.execute(query)
            return dict(result.fetchone())
```

5.4 API Design

5.4.1 RESTful API Structure

```
# API URL Patterns
urlpatterns = [
    # Authentication
    path('api/auth/register/', RegisterView.as_view(), name='register'),
    path('api/auth/login/', LoginView.as_view(), name='login'),
    path('api/auth/logout/', LogoutView.as_view(), name='logout'),
    path('api/auth/refresh/', TokenRefreshView.as_view(),
name='token_refresh'),

    # Predictions
    path('api/predict/', PredictView.as_view(), name='predict'),
    path('api/predict/multi-timeframe/', MultiTimeframePredictView.as_view(),
name='multi_predict'),
    path('api/predict/batch/', BatchPredictView.as_view(),
name='batch_predict'),

    # Market Data
    path('api/market/chart/<str:ticker>/', ChartDataView.as_view(),
name='chart_data'),
    path('api/market/overview/', MarketOverviewView.as_view(),
name='market_overview'),
    path('api/market/sentiment/', MarketSentimentView.as_view(),
name='market_sentiment'),

    # Portfolio
    path('api/portfolio/', PortfolioView.as_view(), name='portfolio'),
    path('api/portfolio/trades/', TradesView.as_view(), name='trades'),
    path('api/portfolio/watchlist/', WatchlistView.as_view(),
name='watchlist'),

    # Models
    path('api/models/', ModelListView.as_view(), name='model_list'),
    path('api/models/performance/', ModelPerformanceView.as_view(),
name='model_performance'),
    path('api/models/train/', TrainModelView.as_view(), name='train_model'),

    # System
    path('api/system/health/', HealthCheckView.as_view(),
name='health_check'),
    path('api/system/metrics/', SystemMetricsView.as_view(),
name='system_metrics'),
]
```

5.4.2 API Documentation

```
# OpenAPI Schema Definition
from drf_spectacular.utils import extend_schema, OpenApiParameter, OpenApiExample

class MultiTimeframePredictView(APIView):
    @extend_schema(
        summary="Generate multi-timeframe predictions",
        description="""
        Generate stock price predictions across multiple timeframes.
        Supports 9 different timeframes from intraday to 5-year predictions.
        """,
        request={
            'application/json': {
                'type': 'object',
                'properties': {
                    'ticker': {
                        'type': 'string',
                        'description': 'Stock ticker symbol',
                        'example': 'AAPL'
                    },
                    'timeframes': {
                        'type': 'array',
                        'items': {'type': 'string'},
                        'description': 'List of timeframes',
                        'example': ['1d', '1w', '1mo']
                    },
                    'include_analysis': {
                        'type': 'boolean',
                        'description': 'Include technical analysis',
                        'default': True
                    }
                },
                'required': ['ticker']
            }
        },
        responses={
            200: {
                'description': 'Successful prediction',
                'content': {
                    'application/json': {
                        'example': {
                            'ticker': 'AAPL',
                            'predictions': {
                                '1d': {
                                    'direction': 'UP',
                                    'confidence': 65.5,
                                    'price_target': 185.50,
                                    'expected_return': 2.3
                                }
                            },
                            'analysis': {
                                'rsi': 58.5,
                                'trend': 'bullish'
                            }
                        }
                    }
                }
            },
            400: {'description': 'Bad request'},
            404: {'description': 'Ticker not found'},
            500: {'description': 'Internal server error'}
        },
        tags=['predictions']
    )
    def post(self, request):
        # Implementation
        pass
```

5.4.3 API Rate Limiting

```
from django.core.cache import cache
from django.http import JsonResponse
from functools import wraps
import hashlib

class RateLimiter:
    """
    Custom rate limiting implementation
    """

    def __init__(self, requests_per_hour=100):
        self.requests_per_hour = requests_per_hour
        self.window = 3600  # 1 hour in seconds

    def limit_api_calls(self, get_response):
        """
        Middleware for rate limiting
        """
        def middleware(request):
            # Get client identifier
            client_id = self.get_client_id(request)

            # Check rate limit
            if not self.check_rate_limit(client_id):
                return JsonResponse({
                    'error': 'Rate limit exceeded',
                    'message': f'Maximum {self.requests_per_hour} requests per hour',
                    'retry_after': self.get_retry_after(client_id)
                }, status=429)

            # Process request
            response = get_response(request)

            # Add rate limit headers
            response['X-RateLimit-Limit'] = str(self.requests_per_hour)
            response['X-RateLimit-Remaining'] = str(self.get_remaining_requests(client_id))
            response['X-RateLimit-Reset'] = str(self.get_reset_time(client_id))

            return response

        return middleware

    def get_client_id(self, request):
        """
        Generate unique client identifier
        """
        if request.user.is_authenticated:
            return f"user_{request.user.id}"
        else:
            # Use IP address for anonymous users
            ip = request.META.get('REMOTE_ADDR', '')
            return f"ip_{hashlib.md5(ip.encode()).hexdigest()}"

    def check_rate_limit(self, client_id):
        """
        Check if client has exceeded rate limit
        """
        key = f"rate_limit_{client_id}"

        # Get current request count
        request_count = cache.get(key, 0)

        if request_count >= self.requests_per_hour:
            return False

        # Increment counter
        cache.set(key, request_count + 1, self.window)

        return True

    def get_remaining_requests(self, client_id):
        """
        Get remaining requests for client
        """
        key = f"rate_limit_{client_id}"
        request_count = cache.get(key, 0)
        return max(0, self.requests_per_hour - request_count)
```

5.5 User Interface Design

5.5.1 UI Component Library

```
// Design System Components
import { createTheme } from '@mui/material/styles';

const theme = createTheme({
  palette: {
    primary: {
      main: '#1976d2',
      light: '#42a5f5',
      dark: '#1565c0',
    },
    secondary: {
      main: '#dc004e',
      light: '#f50057',
      dark: '#c51162',
    },
    success: {
      main: '#4caf50',
    },
    warning: {
      main: '#ff9800',
    },
    error: {
      main: '#f44336',
    },
    background: {
      default: '#f5f5f5',
      paper: '#ffffff',
    },
  },
  typography: {
    fontFamily: '"Roboto", "Helvetica", "Arial", sans-serif',
    h1: {
      fontSize: '2.5rem',
      fontWeight: 500,
    },
    h2: {
      fontSize: '2rem',
      fontWeight: 500,
    },
    h3: {
      fontSize: '1.75rem',
      fontWeight: 500,
    },
  },
  components: {
    MuiButton: {
      styleOverrides: {
        root: {
          textTransform: 'none',
          borderRadius: 8,
        },
      },
    },
    MuiPaper: {
```

```
      styleOverrides: {
        root: {
          borderRadius: 12,
        },
      },
    },
  },
});

// Reusable UI Components
const PredictionCard = ({ prediction, timeframe }) => {
  const getDirectionColor = (direction) => {
    return direction === 'UP' ? 'success.main' : 'error.main';
  };

  return (
    <Card sx={{ minWidth: 275, m: 1 }}>
      <CardContent>
        <Typography variant="h6" component="div">
          {timeframe} Prediction
        </Typography>
        <Box display="flex" alignItems="center" mt={2}>
          <TrendingUpIcon
            sx={{
              color: getDirectionColor(prediction.direction),
              fontSize: 40,
              transform: prediction.direction === 'DOWN' ? 'rotate(180deg)' : 'none'
            }}
          />
          <Box ml={2}>
            <Typography variant="h4" color={getDirectionColor(prediction.direction)}>
              {prediction.direction}
            </Typography>
            <Typography variant="body2" color="text.secondary">
              Confidence: {prediction.confidence}%
            </Typography>
          </Box>
        </Box>
        <Divider sx={{ my: 2 }} />
        <Grid container spacing={2}>
          <Grid item xs={6}>
            <Typography variant="body2" color="text.secondary">
              Current Price
            </Typography>
            <Typography variant="h6">
              ${prediction.current_price}
            </Typography>
          </Grid>
          <Grid item xs={6}>
            <Typography variant="body2" color="text.secondary">
              Target Price
            </Typography>
            <Typography variant="h6">
              ${prediction.price_target}
            </Typography>
          </Grid>
        </Grid>
      </CardContent>
    </Card>
  );
};
```

5.5.2 Responsive Design

```
/* Responsive Design Styles */
.container {
  width: 100%;
  padding-right: 15px;
  padding-left: 15px;
  margin-right: auto;
  margin-left: auto;
}

/* Mobile First Approach */
/* Extra small devices (phones, 600px and down) */
@media only screen and (max-width: 600px) {
  .container {
    padding: 10px;
  }

  .chart-container {
    height: 300px;
  }

  .prediction-grid {
    grid-template-columns: 1fr;
  }

  .hide-mobile {
    display: none;
  }
}

/* Small devices (portrait tablets and large phones, 600px and up) */
@media only screen and (min-width: 600px) {
  .container {
    max-width: 540px;
  }

  .chart-container {
    height: 400px;
  }

  .prediction-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* Medium devices (landscape tablets, 768px and up) */
@media only screen and (min-width: 768px) {
  .container {
    max-width: 720px;
  }

  .chart-container {
    height: 500px;
  }

  .prediction-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}

/* Large devices (laptops/desktops, 992px and up) */
@media only screen and (min-width: 992px) {
  .container {
    max-width: 960px;
  }

  .chart-container {
    height: 600px;
  }

  .prediction-grid {
    grid-template-columns: repeat(4, 1fr);
  }
}

/* Extra large devices (large laptops and desktops, 1200px and up) */
@media only screen and (min-width: 1200px) {
  .container {
    max-width: 1140px;
  }
}
```

5.5.3 Accessibility Features

```
// Accessibility Implementation
const AccessibleChart = ({ data, title }) => {
  return (
    <div role="img" aria-label={`Chart showing ${title}`}>
      <canvas
        id="chart"
        aria-label={`Interactive chart of ${title}`}
        role="img"
      />

      {/* Screen reader friendly data table */}
      <table className="sr-only" aria-label={`Data for ${title}`}>
        <thead>
          <tr>
            <th>Date</th>
            <th>Price</th>
            <th>Volume</th>
          </tr>
        </thead>
        <tbody>
          {data.map((point, index) => (
            <tr key={index}>
              <td>{point.date}</td>
              <td>{point.price}</td>
              <td>{point.volume}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
};

// Keyboard Navigation
const KeyboardNavigableMenu = () => {
  const menuItems = ['Dashboard', 'Predictions', 'Portfolio', 'Analysis'];
  const [selectedIndex, setSelectedIndex] = useState(0);

  const handleKeyDown = (event) => {
    switch (event.key) {
      case 'ArrowDown':
        event.preventDefault();
        setSelectedIndex((prevIndex) =>
          prevIndex < menuItems.length - 1 ? prevIndex + 1 : 0
        );
        break;
      case 'ArrowUp':
        event.preventDefault();
        setSelectedIndex((prevIndex) =>
          prevIndex > 0 ? prevIndex - 1 : menuItems.length - 1
        );
        break;
      case 'Enter':
        event.preventDefault();
        // Navigate to selected item
        break;
      default:
        break;
    }
  };

  return (
    <nav role="navigation" aria-label="Main navigation">
      <ul role="menubar" onKeyDown={handleKeyDown}>
        {menuItems.map((item, index) => (
          <li
            key={item}
            role="menuitem"
            tabIndex={index === selectedIndex ? 0 : -1}
            aria-current={index === selectedIndex ? 'page' : undefined}
          >
            {item}
          </li>
        ))}
      </ul>
    </nav>
  );
};
```

CHAPTER 6: MODEL BUILDING

6.1 Machine Learning Architecture

6.1.1 Overall ML Pipeline

The StockVibePredictor implements a sophisticated machine learning pipeline that processes raw market data through multiple stages to generate accurate predictions :

```
class MLPipeline:

    def __init__(self):
        self.data_fetcher = DataFetcher()
        self.feature_engineer = FeatureEngineer()
        self.model_trainer = ModelTrainer()
        self.model_evaluator = ModelEvaluator()
        self.prediction_engine = PredictionEngine()

    def execute_pipeline(self, ticker, timeframe):
        """
        Execute complete ML pipeline
        """
        # Stage 1: Data Collection
        raw_data = self.data_fetcher.fetch(ticker, timeframe)

        # Stage 2: Data Preprocessing
        clean_data = self.preprocess_data(raw_data)

        # Stage 3: Feature Engineering
        featured_data = self.feature_engineer.engineer_features(clean_data)

        # Stage 4: Data Splitting
        X_train, X_test, y_train, y_test = self.split_data(featured_data)

        # Stage 5: Model Training
        model = self.model_trainer.train(X_train, y_train)

        # Stage 6: Model Evaluation
        metrics = self.model_evaluator.evaluate(model, X_test, y_test)

        # Stage 7: Model Deployment
        if metrics['accuracy'] > 0.55:
            self.deploy_model(model, ticker, timeframe)

        return model, metrics
```

6.1.2 Feature Engineering Pipeline

```
class FeatureEngineeringPipeline:
    """
    Comprehensive feature engineering for stock data
    """

    def __init__(self):
        self.technical_indicators = TechnicalIndicators()
        self.pattern_recognizer = PatternRecognizer()
        self.feature_selector = FeatureSelector()

    def engineer_features(self, data):
        """
        Generate all features for model training
        """
        # Price-based features
        data = self.add_price_features(data)

        # Volume-based features
        data = self.add_volume_features(data)

        # Technical indicators
        data = self.technical_indicators.compute_all(data)

        # Pattern features
        data = self.pattern_recognizer.detect_patterns(data)

        # Time-based features
        data = self.add_time_features(data)

        # Interaction features
        data = self.add_interaction_features(data)

        # Feature selection
        selected_features = self.feature_selector.select_best_features(data)

        return data[selected_features]

    def add_price_features(self, data):
        """
        Add price-derived features
        """
        # Returns
        data['Return_1d'] = data['Close'].pct_change(1)
        data['Return_5d'] = data['Close'].pct_change(5)
        data['Return_20d'] = data['Close'].pct_change(20)

        # Log returns
        data['Log_Return'] = np.log(data['Close'] / data['Close'].shift(1))

        # Price ratios
        data['High_Low_Ratio'] = data['High'] / data['Low']
        data['Close_Open_Ratio'] = data['Close'] / data['Open']

        # Price position
        data['Price_Position'] = (data['Close'] - data['Low']) /
                                         (data['High'] - data['Low'])

        # Rolling statistics
        for window in [5, 10, 20, 50]:
            data[f'Rolling_Mean_{window}'] =
                                                data['Close'].rolling(window).mean()
            data[f'Rolling_Std_{window}'] =
                                                data['Close'].rolling(window).std()
            data[f'Rolling_Min_{window}'] =
                                                data['Close'].rolling(window).min()
            data[f'Rolling_Max_{window}'] =
                                                data['Close'].rolling(window).max()

        return data
```

6.2 Algorithm Selection

6.2.1 Random Forest Implementation

```
class RandomForestModel:
    """
    Random Forest implementation for stock prediction
    """

    def __init__(self):
        self.model = RandomForestClassifier(
            n_estimators=200,
            max_depth=10,
            min_samples_split=5,
            min_samples_leaf=2,
            max_features='sqrt',
            bootstrap=True,
            oob_score=True,
            random_state=42,
            n_jobs=-1,
            class_weight='balanced'
        )
        self.feature_importances = None

    def train(self, X_train, y_train):
        """
        Train Random Forest model
        """
        # Fit model
        self.model.fit(X_train, y_train)

        # Extract feature importances
        self.feature_importances = self.model.feature_importances_

        # Get OOB score for validation
        oob_score = self.model.oob_score_ if self.model.oob_score else None

        logger.info(f"Random Forest trained with OOB score: {oob_score}")

        return self

    def predict(self, X):
        """
        Make predictions
        """
        predictions = self.model.predict(X)
        probabilities = self.model.predict_proba(X)

        return predictions, probabilities

    def get_feature_importance(self, feature_names):
        """
        Get feature importance ranking
        """
        importance_df = pd.DataFrame({
            'feature': feature_names,
            'importance': self.feature_importances
        }).sort_values('importance', ascending=False)

        return importance_df
```

6.2.2 Gradient Boosting Implementation

```
class GradientBoostingModel:
    """
    Gradient Boosting implementation for stock prediction
    """

    def __init__(self):
        self.model = GradientBoostingClassifier(
            n_estimators=200,
            learning_rate=0.1,
            max_depth=5,
            min_samples_split=10,
            min_samples_leaf=5,
            subsample=0.8,
            max_features='sqrt',
            random_state=42,
            validation_fraction=0.2,
            n_iter_no_change=10,
            tol=0.0001
        )

    def train_with_early_stopping(self, X_train, y_train, X_val, y_val):
        """
        Train with early stopping
        """
        # Set up validation for early stopping
        eval_set = [(X_val, y_val)]

        # Train model
        self.model.fit(
            X_train, y_train,
            eval_set=eval_set,
            eval_metric='logloss',
            early_stopping_rounds=10,
            verbose=False
        )

        # Get training history
        train_score = self.model.train_score_
        val_score = self.model.validation_scores_

        return {
            'n_estimators_used': self.model.n_estimators_,
            'train_score': train_score,
            'val_score': val_score
        }
```

6.2.3 Logistic Regression Implementation

```
class LogisticRegressionModel:
    def __init__(self):
        self.model = LogisticRegression(
            penalty='elasticnet',
            solver='saga',
            l1_ratio=0.5,
            C=1.0,
            max_iter=1000,
            random_state=42,
            n_jobs=-1
        )
        self.scaler = StandardScaler()

    def train(self, X_train, y_train):
        """
        Train Logistic Regression with scaling
        """
        # Scale features
        X_train_scaled = self.scaler.fit_transform(X_train)

        # Train model
        self.model.fit(X_train_scaled, y_train)

        # Get coefficients for interpretation
        coefficients = self.model.coef_[0]
        intercept = self.model.intercept_[0]

        return {
            'coefficients': coefficients,
            'intercept': intercept,
            'n_iter': self.model.n_iter_
        }

    def predict(self, X):
        """
        Make predictions with probability estimates
        """
        X_scaled = self.scaler.transform(X)
        predictions = self.model.predict(X_scaled)
        probabilities = self.model.predict_proba(X_scaled)

        # Get decision function values for confidence
        decision_values = self.model.decision_function(X_scaled)

        return {
            'predictions': predictions,
            'probabilities': probabilities,
            'decision_values': decision_values
        }
```

6.3 Ensemble Strategy

6.3.1 Voting Ensemble

```
class VotingEnsemble:
    """
    Voting ensemble combining multiple models
    """

    def __init__(self, voting='soft'):
        self.rf_model = RandomForestModel()
        self.gb_model = GradientBoostingModel()
        self.lr_model = LogisticRegressionModel()

        self.ensemble = VotingClassifier(
            estimators=[
                ('rf', self.rf_model.model),
                ('gb', self.gb_model.model),
                ('lr', self.lr_model.model)
            ],
            voting=voting,
            weights=[0.4, 0.4, 0.2]  # Weight based on individual performance
        )

    def train(self, X_train, y_train):
        """
        Train ensemble model
        """
        # Train individual models first for analysis
        self.rf_model.train(X_train, y_train)
        self.gb_model.train(X_train, y_train)
        self.lr_model.train(X_train, y_train)

        # Train ensemble
        self.ensemble.fit(X_train, y_train)

        return self

    def predict_with_confidence(self, X):
        """
        Make predictions with confidence scores
        """
        # Get predictions from each model
        rf_pred = self.rf_model.predict(X)[1]
        gb_pred = self.gb_model.predict(X)[1]
        lr_pred = self.lr_model.predict(X)['probabilities']

        # Ensemble prediction
        ensemble_pred = self.ensemble.predict(X)
        ensemble_proba = self.ensemble.predict_proba(X)

        # Calculate agreement score
        agreement = self.calculate_agreement(rf_pred, gb_pred, lr_pred)

        return {
            'prediction': ensemble_pred,
            'probability': ensemble_proba,
            'agreement': agreement,
            'individual_predictions': {
                'random_forest': rf_pred,
                'gradient_boosting': gb_pred,
                'logistic_regression': lr_pred
            }
        }

    def calculate_agreement(self, rf_pred, gb_pred, lr_pred):
        """
        Calculate model agreement score
        """
        predictions = np.array([
            rf_pred.argmax(axis=1),
            gb_pred.argmax(axis=1),
            lr_pred.argmax(axis=1)
        ])

        # Calculate pairwise agreement
        agreement_scores = []
        for i in range(len(predictions[0])):
            votes = [predictions[j][i] for j in range(3)]
            agreement = votes.count(max(set(votes), key=votes.count)) / 3
            agreement_scores.append(agreement)

        return np.array(agreement_scores)
```

6.3.2 Stacking Ensemble

```
class StackingEnsemble:
    """
    Stacking ensemble with meta-learner
    """

    def __init__(self):
        # Base models
        self.base_models = [
            ('rf', RandomForestClassifier(n_estimators=100,
random_state=42)),
            ('gb', GradientBoostingClassifier(n_estimators=100,
random_state=42)),
            ('lr', LogisticRegression(max_iter=1000, random_state=42)),
            ('svm', SVC(probability=True, random_state=42))
        ]

        # Meta-learner
        self.meta_learner = LogisticRegression(max_iter=1000)

        self.stacking_classifier = StackingClassifier(
            estimators=self.base_models,
            final_estimator=self.meta_learner,
            cv=5,  # Use 5-fold cross-validation
            stack_method='predict_proba',
            n_jobs=-1
        )

    def train(self, X_train, y_train):
        """
        Train stacking ensemble
        """
        # Fit the stacking classifier
        self.stacking_classifier.fit(X_train, y_train)

        # Get cross-validation predictions for analysis
        cv_predictions = cross_val_predict(
            self.stacking_classifier,
            X_train, y_train,
            cv=5,
            method='predict_proba'
        )

        return {
            'model': self.stacking_classifier,
            'cv_predictions': cv_predictions
        }
```

6.4 Training Pipeline

6.4.1 Data Preparation

```
class DataPreparation:
    """
    Prepare data for model training
    """

    def __init__(self):
        self.scaler = StandardScaler()
        self.imputer = SimpleImputer(strategy='median')
        self.encoder = LabelEncoder()

    def prepare_training_data(self, data, target_col='Target'):
        """
        Complete data preparation pipeline
        """
        # Separate features and target
        feature_cols = [col for col in data.columns if col != target_col]
        X = data[feature_cols]
        y = data[target_col]

        # Handle missing values
        X = pd.DataFrame(
            self.imputer.fit_transform(X),
            columns=X.columns,
            index=X.index
        )

        # Remove highly correlated features
        X = self.remove_correlated_features(X, threshold=0.95)

        # Scale features
        X_scaled = pd.DataFrame(
            self.scaler.fit_transform(X),
            columns=X.columns,
            index=X.index
        )

        # Create train-test split with time series consideration
        X_train, X_test, y_train, y_test = self.time_series_split(
            X_scaled, y, test_size=0.2
        )

        return X_train, X_test, y_train, y_test

    def time_series_split(self, X, y, test_size=0.2):
        """
        Time series aware train-test split
        """
        # Calculate split point
        split_point = int(len(X) * (1 - test_size))

        # Split maintaining temporal order
        X_train = X.iloc[:split_point]
        X_test = X.iloc[split_point:]
        y_train = y.iloc[:split_point]
        y_test = y.iloc[split_point:]

        return X_train, X_test, y_train, y_test

    def remove_correlated_features(self, df, threshold=0.95):
        """
        Remove highly correlated features
        """
        # Calculate correlation matrix
        corr_matrix = df.corr().abs()

        # Select upper triangle
        upper = corr_matrix.where(
            np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
        )

        # Find features to drop
        to_drop = [column for column in upper.columns
                   if any(upper[column] > threshold)]

        # Drop features
        df_reduced = df.drop(columns=to_drop)

        logger.info(f"Removed {len(to_drop)} correlated features")

        return df_reduced
```

6.4.2 Model Training Process

```
class ModelTrainingProcess:

    def __init__(self):
        self.data_prep = DataPreparation()
        self.ensemble = VotingEnsemble()
        self.evaluator = ModelEvaluator()
        self.model_storage = ModelStorage()

    def train_model(self, ticker, timeframe, data):
        """
        Train model for specific ticker and timeframe
        """
        training_start = time.time()

        # Step 1: Prepare data
        logger.info(f"Preparing data for {ticker} ({timeframe})")
        X_train, X_test, y_train, y_test = self.data_prep.prepare_training_data(data)

        # Step 2: Check data quality
        if len(X_train) < 100:
            raise InsufficientDataError(f"Only {len(X_train)} training samples")

        # Step 3: Train ensemble
        logger.info(f"Training ensemble model for {ticker} ({timeframe})")
        self.ensemble.train(X_train, y_train)

        # Step 4: Evaluate model
        metrics = self.evaluator.evaluate(
            self.ensemble, X_test, y_test
        )

        # Step 5: Cross-validation
        cv_scores = cross_val_score(
            self.ensemble.ensemble,
            X_train, y_train,
            cv=5,
            scoring='accuracy'
        )

        metrics['cv_mean'] = cv_scores.mean()
        metrics['cv_std'] = cv_scores.std()

        # Step 6: Save model if performance is acceptable
        if metrics['accuracy'] >= 0.55:
            model_path = self.model_storage.save_model(
                self.ensemble,
                ticker,
                timeframe,
                metrics
            )
            logger.info(f"Model saved: {model_path}")
        else:
            logger.warning(f"Model accuracy {metrics['accuracy']} below threshold")

        training_time = time.time() - training_start
        metrics['training_time'] = training_time

        return metrics
```

6.5 Model Validation

6.5.1 Cross-Validation Strategy

```
class CrossValidationStrategy:
    """
    Implement various cross-validation strategies
    """

    def __init__(self):
        self.cv_strategies = {
            'kfold': KFold(n_splits=5, shuffle=True, random_state=42),
            'stratified': StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
            'timeseries': TimeSeriesSplit(n_splits=5),
            'group': GroupKFold(n_splits=5)
        }

    def validate_model(self, model, X, y, strategy='stratified'):
        """
        Perform cross-validation with specified strategy
        """
        cv = self.cv_strategies[strategy]

        # Perform cross-validation
        scores = {
            'accuracy': [],
            'precision': [],
            'recall': [],
            'f1': [],
            'roc_auc': []
        }

        for train_idx, val_idx in cv.split(X, y):
            X_train_cv, X_val_cv = X.iloc[train_idx], X.iloc[val_idx]
            y_train_cv, y_val_cv = y.iloc[train_idx], y.iloc[val_idx]

            # Train model on fold
            model.fit(X_train_cv, y_train_cv)

            # Make predictions
            y_pred = model.predict(X_val_cv)
            y_proba = model.predict_proba(X_val_cv)[:, 1]

            # Calculate metrics
            scores['accuracy'].append(accuracy_score(y_val_cv, y_pred))
            scores['precision'].append(precision_score(y_val_cv, y_pred))
            scores['recall'].append(recall_score(y_val_cv, y_pred))
            scores['f1'].append(f1_score(y_val_cv, y_pred))
            scores['roc_auc'].append(roc_auc_score(y_val_cv, y_proba))

        # Calculate statistics
        results = {}
        for metric, values in scores.items():
            results[f'{metric}_mean'] = np.mean(values)
            results[f'{metric}_std'] = np.std(values)
            results[f'{metric}_min'] = np.min(values)
            results[f'{metric}_max'] = np.max(values)

        return results
```

6.5.2 Model Evaluation Metrics

```
class ModelEvaluator:
    """
    Comprehensive model evaluation
    """

    def evaluate(self, model, X_test, y_test):
        """
        Evaluate model performance
        """
        # Make predictions
        y_pred = model.predict(X_test)

        # Get probabilities if available
        if hasattr(model, 'predict_proba'):
            y_proba = model.predict_proba(X_test)[:, 1]
        else:
            y_proba = None

        # Calculate metrics
        metrics = {
            'accuracy': accuracy_score(y_test, y_pred),
            'precision': precision_score(y_test, y_pred, average='weighted'),
            'recall': recall_score(y_test, y_pred, average='weighted'),
            'f1_score': f1_score(y_test, y_pred, average='weighted'),
        }

        # Add ROC AUC if probabilities available
        if y_proba is not None:
            metrics['roc_auc'] = roc_auc_score(y_test, y_proba)

        # Confusion matrix
        cm = confusion_matrix(y_test, y_pred)
        metrics['confusion_matrix'] = cm.tolist()

        # Classification report
        report = classification_report(y_test, y_pred, output_dict=True)
        metrics['classification_report'] = report

        # Additional metrics
        metrics['matthews_corrcoef'] = matthews_corrcoef(y_test, y_pred)
        metrics['cohen_kappa'] = cohen_kappa_score(y_test, y_pred)

        return metrics

    def plot_confusion_matrix(self, cm, classes=['DOWN', 'UP']):
        """
        Plot confusion matrix
        """
        plt.figure(figsize=(8, 6))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                    xticklabels=classes, yticklabels=classes)
        plt.title('Confusion Matrix')
        plt.ylabel('True Label')
        plt.xlabel('Predicted Label')
        plt.tight_layout()
        return plt.gcf()

    def plot_roc_curve(self, y_test, y_proba):
        """
        Plot ROC curve
        """
        fpr, tpr, thresholds = roc_curve(y_test, y_proba)
        roc_auc = auc(fpr, tpr)

        plt.figure(figsize=(8, 6))
        plt.plot(fpr, tpr, color='darkorange', lw=2,
                 label=f'ROC curve (AUC = {roc_auc:.2f})')
        plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
        plt.xlim([0.0, 1.0])
        plt.ylim([0.0, 1.05])
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title('Receiver Operating Characteristic')
        plt.legend(loc="lower right")
        plt.tight_layout()
        return plt.gcf()
```

6.6 Performance Analysis

6.6.1 Model Performance Tracking

```
class PerformanceTracker:
    """
    Track and analyze model performance over time
    """

    def __init__(self):
        self.performance_history = []
        self.prediction_log = []

    def track_prediction(self, ticker, timeframe, prediction, actual=None):
        """
        Track individual predictions
        """
        record = {
            'timestamp': datetime.now(),
            'ticker': ticker,
            'timeframe': timeframe,
            'prediction': prediction['direction'],
            'confidence': prediction['confidence'],
            'actual': actual,
            'correct': None if actual is None else (prediction['direction'] == actual)
        }

        self.prediction_log.append(record)

        # Update performance metrics if actual is known
        if actual is not None:
            self.update_performance_metrics(ticker, timeframe)

    def update_performance_metrics(self, ticker, timeframe):
        """
        Update performance metrics for model
        """
        # Filter predictions for this model
        model_predictions = [
            p for p in self.prediction_log
            if p['ticker'] == ticker and p['timeframe'] == timeframe
            and p['actual'] is not None
        ]

        if len(model_predictions) < 10:
            return  # Not enough data

        # Calculate metrics
        correct = sum(1 for p in model_predictions if p['correct'])
        total = len(model_predictions)
        accuracy = correct / total

        # Calculate average confidence
        avg_confidence = np.mean([p['confidence'] for p in model_predictions])

        # Store performance record
        performance_record = {
            'timestamp': datetime.now(),
            'ticker': ticker,
            'timeframe': timeframe,
            'accuracy': accuracy,
            'avg_confidence': avg_confidence,
            'sample_size': total,
            'period': '30d'  # Last 30 days
        }

        self.performance_history.append(performance_record)

        # Check for performance degradation
        if accuracy < 0.45:
            self.trigger_retraining(ticker, timeframe, accuracy)

    def trigger_retraining(self, ticker, timeframe, current_accuracy):
        """
        Trigger model retraining due to poor performance
        """
        logger.warning(
            f"Model performance degradation detected for {ticker} ({timeframe}): "
            f"Accuracy = {current_accuracy:.2%}"
        )

        # Add to retraining queue
        retraining_task = {
            'ticker': ticker,
            'timeframe': timeframe,
            'reason': 'performance_degradation',
            'current_accuracy': current_accuracy,
            'timestamp': datetime.now()
        }

        # Queue for retraining
        self.queue_retraining(retraining_task)
```

6.6.2 Backtesting Framework

```
class BacktestingFramework:
    """
    Backtesting framework for model validation
    """

    def __init__(self):
        self.initial_capital = 10000
        self.position_size = 0.1  # 10% per trade
        self.commission = 0.001  # 0.1% commission

    def backtest_strategy(self, predictions, actual_prices, timeframe):
        """
        Backtest trading strategy based on predictions
        """
        portfolio = {
            'cash': self.initial_capital,
            'positions': {},
            'history': [],
            'equity_curve': []
        }

        for i, (timestamp, prediction) in enumerate(predictions.iterrows()):
            current_price = actual_prices.loc[timestamp]

            # Check for signals
            if prediction['direction'] == 'UP' and prediction['confidence'] > 0.6:
                # Buy signal
                self.execute_buy(portfolio, prediction['ticker'], current_price, timestamp)

            elif prediction['direction'] == 'DOWN' and prediction['confidence'] > 0.6:
                # Sell signal
                self.execute_sell(portfolio, prediction['ticker'], current_price, timestamp)

            # Update portfolio value
            portfolio_value = self.calculate_portfolio_value(portfolio, actual_prices.loc[timestamp])
            portfolio['equity_curve'].append({
                'timestamp': timestamp,
                'value': portfolio_value
            })

        # Calculate performance metrics
        metrics = self.calculate_performance_metrics(portfolio)

        return {
            'portfolio': portfolio,
            'metrics': metrics
        }

    def calculate_performance_metrics(self, portfolio):
        """
        Calculate backtesting performance metrics
        """
        equity_curve = pd.DataFrame(portfolio['equity_curve'])
        equity_curve.set_index('timestamp', inplace=True)

        # Calculate returns
        returns = equity_curve['value'].pct_change().dropna()

        # Total return
        total_return = (equity_curve['value'].iloc[-1] / self.initial_capital - 1) * 100

        # Annualized return
        days = (equity_curve.index[-1] - equity_curve.index[0]).days
        annualized_return = ((equity_curve['value'].iloc[-1] / self.initial_capital) ** (365/days) - 1) * 100

        # Sharpe ratio
        sharpe_ratio = (returns.mean() / returns.std()) * np.sqrt(252)

        # Maximum drawdown
        cumulative = (1 + returns).cumprod()
        running_max = cumulative.expanding().max()
        drawdown = (cumulative - running_max) / running_max
        max_drawdown = drawdown.min() * 100

        # Win rate
        trades = portfolio['history']
        winning_trades = sum(1 for t in trades if t['pnl'] > 0)
        win_rate = (winning_trades / len(trades) * 100) if trades else 0

        return {
            'total_return': total_return,
            'annualized_return': annualized_return,
            'sharpe_ratio': sharpe_ratio,
            'max_drawdown': max_drawdown,
            'win_rate': win_rate,
            'total_trades': len(trades),
            'final_value': equity_curve['value'].iloc[-1]
        }
```

CHAPTER 7: IMPLEMENTATION

7.1 Development Environment

7.1.1 System Requirements

```
# System Requirements Specification
minimum_requirements:
  operating_system:
    - "Ubuntu 20.04 LTS or higher"
    - "Windows 10/11 with WSL2"
    - "macOS 11.0 or higher"

  hardware:
    cpu: "4 cores, 2.4 GHz"
    ram: "8 GB"
    storage: "20 GB available"
    network: "Broadband internet connection"

  software:
    python: "3.8 or higher"
    node: "16.0 or higher"
    npm: "8.0 or higher"
    postgresql: "12.0 or higher"
    redis: "6.0 or higher"
    git: "2.25 or higher"

recommended_requirements:
  hardware:
    cpu: "8 cores, 3.0 GHz"
    ram: "16 GB"
    storage: "50 GB SSD"
    gpu: "NVIDIA GPU with CUDA support (optional)"

  software:
    docker: "20.10 or higher"
    docker_compose: "1.29 or higher"
    nginx: "1.18 or higher"
```

7.1.2 Development Setup

```
#!/bin/bash
# Development Environment Setup Script

# Color codes for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m' # No Color

echo -e "${GREEN}========================================${NC}"
echo -e "${GREEN}StockVibePredictor Development Setup${NC}"
echo -e "${GREEN}========================================${NC}"

# Step 1: Check Python version
echo -e "${YELLOW}Checking Python version...${NC}"
python_version=$(python3 --version 2>&1 | grep -Po '(?<=Python )\d+\.\d+')
required_version="3.8"

if [ "$(printf '%s\n' "$required_version" "$python_version" | sort -V | head -n1)" = "$required_version" ]; then
    echo -e "${GREEN}✓ Python $python_version is installed${NC}"
else
    echo -e "${RED}✗ Python $required_version or higher is required${NC}"
    exit 1
fi

# Step 2: Create virtual environment
echo -e "${YELLOW}Creating virtual environment...${NC}"
python3 -m venv venv
source venv/bin/activate
echo -e "${GREEN}✓ Virtual environment created and activated${NC}"

# Step 3: Upgrade pip
echo -e "${YELLOW}Upgrading pip...${NC}"
pip install --upgrade pip
echo -e "${GREEN}✓ Pip upgraded${NC}"

# Step 4: Install Python dependencies
echo -e "${YELLOW}Installing Python dependencies...${NC}"
pip install -r requirements.txt
echo -e "${GREEN}✓ Python dependencies installed${NC}"

# Step 5: Check Node.js version
echo -e "${YELLOW}Checking Node.js version...${NC}"
node_version=$(node --version 2>&1 | grep -Po '\d+' | head -n1)
if [ "$node_version" -ge 16 ]; then
    echo -e "${GREEN}✓ Node.js $(node --version) is installed${NC}"
else
    echo -e "${RED}✗ Node.js 16.0 or higher is required${NC}"
    exit 1
fi

# Step 6: Install frontend dependencies
echo -e "${YELLOW}Installing frontend dependencies...${NC}"
cd frontend
npm install
cd ..
echo -e "${GREEN}✓ Frontend dependencies installed${NC}"

# Step 7: Setup database
echo -e "${YELLOW}Setting up PostgreSQL database...${NC}"
createdb stockvibe_dev
echo -e "${GREEN}✓ Database created${NC}"

# Step 8: Run migrations
echo -e "${YELLOW}Running database migrations...${NC}"
python manage.py migrate
echo -e "${GREEN}✓ Migrations completed${NC}"

# Step 9: Create superuser
echo -e "${YELLOW}Creating superuser account...${NC}"
python manage.py createsuperuser --noinput \
    --username admin \
    --email admin@stockvibe.com
echo -e "${GREEN}✓ Superuser created${NC}"

# Step 10: Load initial data
echo -e "${YELLOW}Loading initial data...${NC}"
python manage.py loaddata initial_data.json
echo -e "${GREEN}✓ Initial data loaded${NC}"

echo -e "${GREEN}========================================${NC}"
echo -e "${GREEN}Setup completed successfully!${NC}"
echo -e "${GREEN}========================================${NC}"
echo -e "To start the development server:"
echo -e "  Backend: ${YELLOW}python manage.py runserver${NC}"
echo -e "  Frontend: ${YELLOW}cd frontend && npm start${NC}"
```

7.1.3 Dependencies Management

```
# requirements.txt

# Core Django
Django==5.2.5
djangorestframework==3.14.0
django-cors-headers==4.3.0
django-redis==5.4.0
django-environ==0.11.2

# Database
psycopg2-binary==2.9.9
sqlalchemy==2.0.23

# Machine Learning
scikit-learn==1.7.1
numpy==1.24.3
pandas==2.1.3
scipy==1.11.4
joblib==1.3.2

# Data Sources
yfinance==0.2.65
requests==2.31.0
aiohttp==3.9.1

# Task Queue
celery==5.3.4
redis==5.0.1
flower==2.0.1

# Technical Indicators
ta-lib==0.4.28
pandas-ta==0.3.14b0

# Utilities
python-dateutil==2.8.2
pytz==2023.3
python-dotenv==1.0.0

# Testing
pytest==7.4.3
pytest-django==4.7.0
pytest-cov==4.1.0
factory-boy==3.3.0
faker==20.1.0

# Documentation
drf-spectacular==0.27.0
sphinx==7.2.6

# Monitoring
sentry-sdk==1.38.0
prometheus-client==0.19.0

# Development
black==23.11.0
flake8==6.1.0
isort==5.12.0
pre-commit==3.5.0
ipython==8.17.2
"""

# package.json for frontend
{
  "name": "stockvibe-frontend",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.20.0",
    "axios": "^1.6.2",
    "chart.js": "^4.4.0",
    "react-chartjs-2": "^5.2.0",
    "@mui/material": "^5.14.18",
    "@emotion/react": "^11.11.1",
    "@emotion/styled": "^11.11.0",
    "date-fns": "^2.30.0",
    "lodash": "^4.17.21",
    "react-query": "^3.39.3",
    "formik": "^2.4.5",
    "yup": "^1.3.3"
  },
  "devDependencies": {
    "@testing-library/react": "^14.1.2",
    "@testing-library/jest-dom": "^6.1.4",
    "@testing-library/user-event": "^14.5.1",
    "eslint": "^8.54.0",
    "prettier": "^3.1.0",
    "husky": "^8.0.3",
    "lint-staged": "^15.1.0"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject",
    "lint": "eslint src/",
    "format": "prettier --write \"src/**/*.{js,jsx,json,css,md}\""
  }

}
```

7.2 Backend Implementation

7.2.1 Django Project Structure

```
# settings.py - Main configuration
import os
from pathlib import Path
import environ

# Initialize environment variables
env = environ.Env()
environ.Env.read_env()

# Build paths
BASE_DIR = Path(__file__).resolve().parent.parent

# Security
SECRET_KEY = env('SECRET_KEY')
DEBUG = env.bool('DEBUG', default=False)
ALLOWED_HOSTS = env.list('ALLOWED_HOSTS', default=['localhost'])

# Application definition
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Third party
    'rest_framework',
    'corsheaders',
    'drf_spectacular',

    # Local apps
    'apps.predictions',
    'apps.market_data',
    'apps.portfolio',
    'apps.users',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'apps.common.middleware.RequestLoggingMiddleware',
    'apps.common.middleware.PerformanceMonitoringMiddleware',
]

# Database
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': env('DB_NAME'),
        'USER': env('DB_USER'),
        'PASSWORD': env('DB_PASSWORD'),
        'HOST': env('DB_HOST', default='localhost'),
        'PORT': env('DB_PORT', default='5432'),
        'CONN_MAX_AGE': 600,
        'OPTIONS': {
            'connect_timeout': 10,
        }
    }
}

# Cache
CACHES = {
    'default': {
        'BACKEND': 'django_redis.cache.RedisCache',
        'LOCATION': env('REDIS_URL', default='redis://127.0.0.1:6379/1'),
        'OPTIONS': {
            'CLIENT_CLASS': 'django_redis.client.DefaultClient',
            'CONNECTION_POOL_KWARGS': {
                'max_connections': 50,
                'retry_on_timeout': True
            },
            'SOCKET_CONNECT_TIMEOUT': 5,
            'SOCKET_TIMEOUT': 5,
        }
    }
}

# REST Framework
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
        'rest_framework.authentication.SessionAuthentication',
```

```
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticatedOrReadOnly',
    ],
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle',
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '100/hour',
        'user': '1000/hour',
        'prediction': '100/hour',
    },
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 100,
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
}

# Celery Configuration
CELERY_BROKER_URL = env('REDIS_URL', default='redis://127.0.0.1:6379/0')
CELERY_RESULT_BACKEND = env('REDIS_URL', default='redis://127.0.0.1:6379/0')
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'
CELERY_RESULT_SERIALIZER = 'json'
CELERY_TIMEZONE = 'UTC'
CELERY_BEAT_SCHEDULE = {
    'update-market-data': {
        'task': 'apps.market_data.tasks.update_market_data',
        'schedule': 300.0,  # Every 5 minutes
    },
    'retrain-models': {
        'task': 'apps.predictions.tasks.retrain_models',
        'schedule': 86400.0,  # Daily
    },
    'cleanup-old-data': {
        'task': 'apps.common.tasks.cleanup_old_data',
        'schedule': 3600.0,  # Hourly
    },
}

# Logging
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'verbose': {
            'format': '{levelname} {asctime} {module} {process:d} {thread:d} {message}',
            'style': '{',
        },
        'simple': {
            'format': '{levelname} {message}',
            'style': '{',
        },
    },
    'filters': {
        'require_debug_true': {
            '()': 'django.utils.log.RequireDebugTrue',
        },
    },
    'handlers': {
        'console': {
            'level': 'INFO',
            'filters': ['require_debug_true'],
            'class': 'logging.StreamHandler',
            'formatter': 'simple'
        },
        'file': {
            'level': 'INFO',
            'class': 'logging.handlers.RotatingFileHandler',
            'filename': BASE_DIR / 'logs' / 'django.log',
            'maxBytes': 1024 * 1024 * 50,  # 50 MB
            'backupCount': 5,
            'formatter': 'verbose',
        },
    },
    'root': {
        'handlers': ['console', 'file'],
        'level': 'INFO',
    },
    'loggers': {
        'django': {
            'handlers': ['console', 'file'],
            'level': env('DJANGO_LOG_LEVEL', default='INFO'),
            'propagate': False,
        },
        'apps': {
            'handlers': ['console', 'file'],
            'level': 'DEBUG',
            'propagate': False,
        },
    },
}
```

7.2.2 API Implementation

```
# views.py - Core API views
from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from django.core.cache import cache
from django.utils import timezone
import logging

logger = logging.getLogger(__name__)

class PredictionViewSet(viewsets.ViewSet):
    """
    ViewSet for stock predictions
    """
    permission_classes = [IsAuthenticated]

    def create(self, request):
        """
        Generate prediction for a stock
        POST /api/predictions/
        """
        serializer = PredictionRequestSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        ticker = serializer.validated_data['ticker']
        timeframe = serializer.validated_data['timeframe']

        # Check cache
        cache_key = f"prediction:{ticker}:{timeframe}"
        cached_result = cache.get(cache_key)

        if cached_result:
            logger.info(f"Cache hit for {ticker} {timeframe}")
            return Response(cached_result)

        try:
            # Generate prediction
            prediction_service = PredictionService()
            result = prediction_service.generate_prediction(
                ticker=ticker,
                timeframe=timeframe,
                user=request.user
            )

            # Cache result
            cache.set(cache_key, result, timeout=300)  # 5 minutes

            # Log prediction
            PredictionLog.objects.create(
                user=request.user,
                ticker=ticker,
                timeframe=timeframe,
                prediction=result['direction'],
                confidence=result['confidence']
            )

            return Response(result, status=status.HTTP_200_OK)
```

```
        except Exception as e:
            logger.error(f"Prediction failed: {str(e)}")
            return Response(
                {'error': 'Prediction failed', 'message': str(e)},
                status=status.HTTP_500_INTERNAL_SERVER_ERROR
            )

    @action(detail=False, methods=['post'])
    def batch(self, request):
        """
        Generate predictions for multiple stocks
        POST /api/predictions/batch/
        """
        serializer = BatchPredictionRequestSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        tickers = serializer.validated_data['tickers']
        timeframe = serializer.validated_data['timeframe']

        results = {}
        prediction_service = PredictionService()

        for ticker in tickers:
            try:
                result = prediction_service.generate_prediction(
                    ticker=ticker,
                    timeframe=timeframe,
                    user=request.user
                )
                results[ticker] = result
            except Exception as e:
                results[ticker] = {'error': str(e)}

        return Response(results, status=status.HTTP_200_OK)

    @action(detail=False, methods=['get'])
    def history(self, request):
        """
        Get user's prediction history
        GET /api/predictions/history/
        """
        queryset = PredictionLog.objects.filter(
            user=request.user
        ).order_by('-created_at')

        page = self.paginate_queryset(queryset)
        if page is not None:
            serializer = PredictionLogSerializer(page, many=True)
            return self.get_paginated_response(serializer.data)

        serializer = PredictionLogSerializer(queryset, many=True)
        return Response(serializer.data)
```

7.3 Frontend Implementation

7.3.1 React Application Structure

```
// App.js - Main application component
import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-
router-dom';
import { ThemeProvider } from '@mui/material/styles';
import { QueryClient, QueryClientProvider } from 'react-query';
import { AuthProvider } from './contexts/AuthContext';
import { NotificationProvider } from
'./contexts/NotificationContext';
import theme from './theme';

// Components
import Layout from './components/Layout';
import Dashboard from './pages/Dashboard';
import Predictions from './pages/Predictions';
import Portfolio from './pages/Portfolio';
import Analysis from './pages/Analysis';
import Login from './pages/Login';
import Register from './pages/Register';
import PrivateRoute from './components/PrivateRoute';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      refetchOnWindowFocus: false,
      retry: 1,
      staleTime: 5 * 60 * 1000, // 5 minutes
    },
  },
});

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <ThemeProvider theme={theme}>
        <AuthProvider>
          <NotificationProvider>
            <Router>
              <Layout>
                <Routes>
                  <Route path="/login" element={<Login />} />
```

```
                  <Route path="/register" element={<Register />}
/>
                  <Route
                    path="/"
                    element={
                      <PrivateRoute>
                        <Dashboard />
                      </PrivateRoute>
                    }
                  />
                  <Route
                    path="/predictions"
                    element={
                      <PrivateRoute>
                        <Predictions />
                      </PrivateRoute>
                    }
                  />
                  <Route
                    path="/portfolio"
                    element={
                      <PrivateRoute>
                        <Portfolio />
                      </PrivateRoute>
                    }
                  />
                  <Route
                    path="/analysis"
                    element={
                      <PrivateRoute>
                        <Analysis />
                      </PrivateRoute>
                    }
                  />
                </Routes>
              </Layout>
            </Router>
          </NotificationProvider>
        </AuthProvider>
      </ThemeProvider>
    </QueryClientProvider>
  );
}

export default App;
```

7.3.2 Component Implementation

```
// Dashboard.js - Main dashboard component
import React, { useState, useEffect } from 'react';
import {
  Container,
  Grid,
  Paper,
  Typography,
  Box,
  Card, CardContent,
} from '@mui/material';
import { useQuery } from 'react-query';
import {
  TrendingUp as TrendingUpIcon,
  TrendingDown as TrendingDownIcon,
  ShowChart as ShowChartIcon,
} from '@mui/icons-material';

import StockChart from '../components/StockChart';
import PredictionPanel from '../components/PredictionPanel';
import MarketOverview from '../components/MarketOverview';
import WatchlistWidget from '../components/WatchlistWidget';
import { api } from '../services/api';

const Dashboard = () => {
  const [selectedTicker, setSelectedTicker] = useState('AAPL');
  const [timeframe, setTimeframe] = useState('1mo');

  // Fetch market overview
  const { data: marketData, isLoading: marketLoading } = useQuery(
    'marketOverview',
    () => api.getMarketOverview(),
    {
      refetchInterval: 60000, // Refresh every minute
    }
  );

  // Fetch predictions
  const { data: predictions, isLoading: predictionsLoading } = useQuery(
    ['predictions', selectedTicker],
    () => api.getPredictions(selectedTicker, ['1d', '1w', '1mo', '1y']),
    {
      enabled: !!selectedTicker,
    }
  );

  // Fetch chart data
  const { data: chartData, isLoading: chartLoading } = useQuery(
    ['chartData', selectedTicker, timeframe],
    () => api.getChartData(selectedTicker, timeframe),
    {
      enabled: !!selectedTicker,
    }
  );

  return (
    <Container maxWidth="xl" sx={{ mt: 4, mb: 4 }}>
      <Grid container spacing={3}>
        {/* Market Overview */}
        <Grid item xs={12}>
          <MarketOverview data={marketData} loading={marketLoading} />
        </Grid>

        {/* Main Chart */}
        <Grid item xs={12} lg={8}>
          <Paper
            sx={{
              p: 2,
              display: 'flex',
              flexDirection: 'column',
              height: 500,
            }}
          >
            <Typography component="h2" variant="h6" color="primary" gutterBottom>
              {selectedTicker} - Price Chart
            </Typography>
            <StockChart
              data={chartData}
              loading={chartLoading}
              ticker={selectedTicker}
              timeframe={timeframe}
```

```
              onTimeframeChange={setTimeframe}
            />
          </Paper>
        </Grid>

        {/* Predictions Panel */}
        <Grid item xs={12} lg={4}>
          <PredictionPanel
            predictions={predictions}
            loading={predictionsLoading}
            ticker={selectedTicker}
          />
        </Grid>

        {/* Watchlist */}
        <Grid item xs={12} md={6} lg={4}>
          <WatchlistWidget
            onTickerSelect={setSelectedTicker}
            selectedTicker={selectedTicker}
          />
        </Grid>

        {/* Performance Metrics */}
        <Grid item xs={12} md={6} lg={4}>
          <Card>
            <CardContent>
              <Typography component="h2" variant="h6" gutterBottom>
                Model Performance
              </Typography>
              <Box sx={{ mt: 2 }}>
                <Grid container spacing={2}>
                  <Grid item xs={6}>
                    <Typography variant="body2" color="text.secondary">
                      24h Accuracy
                    </Typography>
                    <Typography variant="h4">
                      67.3%
                    </Typography>
                  </Grid>
                  <Grid item xs={6}>
                    <Typography variant="body2" color="text.secondary">
                      Win Rate
                    </Typography>
                    <Typography variant="h4">
                      58.2%
                    </Typography>
                  </Grid>
                  <Grid item xs={6}>
                    <Typography variant="body2" color="text.secondary">
                      Total Predictions
                    </Typography>
                    <Typography variant="h4">
                      1,247
                    </Typography>
                  </Grid>
                  <Grid item xs={6}>
                    <Typography variant="body2" color="text.secondary">
                      Avg Confidence
                    </Typography>
                    <Typography variant="h4">
                      72.5%
                    </Typography>
                  </Grid>
                </Grid>
              </Box>
            </CardContent>
          </Card>
        </Grid>
        {/* Recent Activity */}
        <Grid item xs={12} lg={4}>
          <Card>
            <CardContent>
              <Typography component="h2" variant="h6" gutterBottom>
                Recent Activity
              </Typography>
              <RecentActivityList />
            </CardContent>
          </Card>
        </Grid>
      </Grid>
    </Container>
  );
};

export default Dashboard;
```

7.4 Model Integration

7.4.1 Model Service Implementation

```
class ModelService:
    """
    Service for model management and prediction
    """

    def __init__(self):
        self.model_registry = {}
        self.load_models()

    def load_models(self):
        """
        Load all models into memory
        """
        model_dir = Path(settings.MODEL_DIR)

        for model_file in model_dir.glob('*.pkl'):
            try:
                # Parse model filename
                parts = model_file.stem.split('_')
                if len(parts) >= 3:
                    ticker = parts[0]
                    timeframe = parts[2]

                    # Load model
                    with open(model_file, 'rb') as f:
                        model_data = pickle.load(f)

                    # Validate model
                    if self.validate_model(model_data):
                        key = f"{ticker}_{timeframe}"
                        self.model_registry[key] = model_data
                        logger.info(f"Loaded model: {key}")

            except Exception as e:
                logger.error(f"Failed to load model {model_file}: {str(e)}")

        logger.info(f"Total models loaded: {len(self.model_registry)}")

    def validate_model(self, model_data):
        """
        Validate model structure and integrity
        """
        required_keys = ['model', 'scaler', 'features', 'accuracy']

        for key in required_keys:
            if key not in model_data:
                return False

        # Check model has predict method
        if not hasattr(model_data['model'], 'predict'):
            return False

        # Check accuracy threshold
        if model_data['accuracy'] < 0.35:
```

```
            return False

        return True

    def get_model(self, ticker, timeframe):
        """
        Get model for prediction
        """
        # Try ticker-specific model first
        specific_key = f"{ticker}_{timeframe}"
        if specific_key in self.model_registry:
            return self.model_registry[specific_key]

        # Fall back to universal model
        universal_key = f"universal_{timeframe}"
        if universal_key in self.model_registry:
            return self.model_registry[universal_key]

        # No model available
        raise ModelNotFoundError(f"No model for {ticker} {timeframe}")

    def predict(self, ticker, timeframe, features):
        """
        Make prediction using appropriate model
        """
        # Get model
        model_data = self.get_model(ticker, timeframe)

        # Prepare features
        model = model_data['model']
        scaler = model_data['scaler']
        required_features = model_data['features']

        # Select and order features
        feature_vector = features[required_features].values

        # Scale features
        if scaler:
            feature_vector = scaler.transform(feature_vector)

        # Make prediction
        prediction = model.predict(feature_vector)[0]

        # Get probability
        if hasattr(model, 'predict_proba'):
            probability = model.predict_proba(feature_vector)[0]
            confidence = probability[int(prediction)]
        else:
            confidence = 0.5

        return {
            'direction': 'UP' if prediction == 1 else 'DOWN',
            'confidence': float(confidence),
            'model_accuracy': model_data['accuracy'],
            'model_type': model_data.get('model_type', 'unknown')
        }
```

7.5 Testing and Debugging

7.5.1 Unit Testing

```
# test_predictions.py
import pytest
from django.test import TestCase
from unittest.mock import Mock, patch
from apps.predictions.services import PredictionService
from apps.predictions.models import PredictionLog

class TestPredictionService(TestCase):
    """
    Test cases for prediction service
    """

    def setUp(self):
        self.service = PredictionService()
        self.test_ticker = 'AAPL'
        self.test_timeframe = '1d'

    @patch('apps.predictions.services.DataFetcher')
    @patch('apps.predictions.services.ModelService')
    def test_generate_prediction_success(self, mock_model, mock_data):
        """
        Test successful prediction generation
        """
        # Mock data fetcher
        mock_data.return_value.fetch.return_value = self.get_mock_data()

        # Mock model service
        mock_model.return_value.predict.return_value = {
            'direction': 'UP',
            'confidence': 0.75,
            'model_accuracy': 0.65
        }

        result = self.service.generate_prediction(
            self.test_ticker,
            self.test_timeframe
        )

        # Assertions
        self.assertIsNotNone(result)
        self.assertEqual(result['direction'], 'UP')
        self.assertGreaterEqual(result['confidence'], 0)
        self.assertLessEqual(result['confidence'], 1)

    def test_invalid_ticker_format(self):
        """
        Test invalid ticker format handling
        """
        with self.assertRaises(ValidationError):
            self.service.generate_prediction(
                'INVALID@TICKER',
                self.test_timeframe
            )

    def test_invalid_timeframe(self):
        """
        Test invalid timeframe handling
        """
        with self.assertRaises(ValidationError):
            self.service.generate_prediction(
                self.test_ticker,
                'invalid_timeframe'
            )

    @patch('apps.predictions.services.cache')
    def test_cache_hit(self, mock_cache):
        """
        Test cache hit scenario
        """
        # Setup cache mock
        cached_result = {'direction': 'UP', 'confidence': 0.7}
        mock_cache.get.return_value = cached_result

        # Generate prediction
        result = self.service.generate_prediction(
            self.test_ticker,
            self.test_timeframe
        )

        # Verify cache was checked
        mock_cache.get.assert_called_once()
        self.assertEqual(result, cached_result)

    def get_mock_data(self):
        """
        Generate mock stock data for testing
        """
        import pandas as pd
        import numpy as np

        dates = pd.date_range(end='2024-01-01', periods=100)
        data = pd.DataFrame({
            'Open': np.random.uniform(100, 200, 100),
            'High': np.random.uniform(100, 200, 100),
            'Low': np.random.uniform(100, 200, 100),
            'Close': np.random.uniform(100, 200, 100),
            'Volume': np.random.uniform(1000000, 10000000, 100)
        }, index=dates)

        return data
```

7.5.2 Integration Testing

```
# test_integration.py
import pytest
from django.test import TestCase, Client
from django.contrib.auth.models import User
from rest_framework.test import APIClient
from rest_framework import status

class TestPredictionAPI(TestCase):
    """
    Integration tests for prediction API
    """

    def setUp(self):
        self.client = APIClient()
        self.user = User.objects.create_user(
            username='testuser',
            password='testpass123'
        )
        self.client.force_authenticate(user=self.user)

    def test_single_prediction_endpoint(self):
        """
        Test single prediction endpoint
        """
        response = self.client.post('/api/predictions/', {
            'ticker': 'AAPL',
            'timeframe': '1d'
        })

        self.assertEqual(response.status_code,
status.HTTP_200_OK)
        self.assertIn('direction', response.data)
        self.assertIn('confidence', response.data)
        self.assertIn('price_target', response.data)

    def test_batch_prediction_endpoint(self):
        """
        Test batch prediction endpoint
        """
        response = self.client.post('/api/predictions/batch/', {
            'tickers': ['AAPL', 'MSFT', 'GOOGL'],
            'timeframe': '1d'
        })
```

```
        self.assertEqual(response.status_code,
status.HTTP_200_OK)
        self.assertEqual(len(response.data), 3)

        for ticker in ['AAPL', 'MSFT', 'GOOGL']:
            self.assertIn(ticker, response.data)

    def test_authentication_required(self):
        """
        Test that authentication is required
        """
        self.client.force_authenticate(user=None)

        response = self.client.post('/api/predictions/', {
            'ticker': 'AAPL',
            'timeframe': '1d'
        })

        self.assertEqual(response.status_code,
status.HTTP_401_UNAUTHORIZED)

    def test_rate_limiting(self):
        """
        Test rate limiting
        """
        # Make requests up to the limit
        for _ in range(100):
            response = self.client.post('/api/predictions/', {
                'ticker': 'AAPL',
                'timeframe': '1d'
            })

            if response.status_code ==
status.HTTP_429_TOO_MANY_REQUESTS:
                break

        # Verify rate limit is enforced
        self.assertEqual(
            response.status_code,
            status.HTTP_429_TOO_MANY_REQUESTS
        )
```

7.5.3 Performance Testing

```
# test_performance.py
import time
import concurrent.futures
from django.test import TestCase
from apps.predictions.services import PredictionService

class TestPerformance(TestCase):
    """
    Performance tests for the system
    """

    def test_prediction_response_time(self):
        """
        Test prediction response time
        """
        service = PredictionService()

        start_time = time.time()
        result = service.generate_prediction('AAPL', '1d')
        end_time = time.time()

        response_time = end_time - start_time

        # Assert response time is under 2 seconds
        self.assertLess(response_time, 2.0)

    def test_concurrent_predictions(self):
        """
        Test system under concurrent load
        """
        service = PredictionService()
        tickers = ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'META']

        def make_prediction(ticker):
            return service.generate_prediction(ticker, '1d')

        # Test with 10 concurrent requests
        with concurrent.futures.ThreadPoolExecutor(max_workers=10) as executor:
            start_time = time.time()
            futures = [
                executor.submit(make_prediction, ticker)
                for ticker in tickers * 2  # 10 requests total
            ]

            results = [f.result() for f in futures]
            end_time = time.time()

        total_time = end_time - start_time

        # All requests should complete
        self.assertEqual(len(results), 10)

        # Average time per request should be reasonable
        avg_time = total_time / 10
        self.assertLess(avg_time, 3.0)

    def test_cache_performance(self):
        """
        Test cache performance improvement
        """
        service = PredictionService()
        ticker = 'AAPL'
        timeframe = '1d'

        # First request (cache miss)
        start_time = time.time()
        result1 = service.generate_prediction(ticker, timeframe)
        first_request_time = time.time() - start_time

        # Second request (cache hit)
        start_time = time.time()
        result2 = service.generate_prediction(ticker, timeframe)
        second_request_time = time.time() - start_time

        # Cache hit should be significantly faster
        self.assertLess(second_request_time, first_request_time / 2)

        # Results should be identical
        self.assertEqual(result1, result2)
```

CHAPTER 8: CODE DOCUMENTATION

8.1 Project Structure

8.1.1 Complete Directory Structure

The StockVibePredictor project follows a modular architecture with clear separation between frontend, backend, and supporting services. Here's the complete structure:

https://github.com/DibVibe/StockVibePredictor/blob/main/README.md

8.1.2 Module Organization

The project is organized into distinct modules, each with specific responsibilities:

Backend Modules :

```
# backend/apps/__init__.py
"""
StockVibePredictor Backend Applications

This package contains all Django applications for the StockVibePredictor system.
Each app is responsible for a specific domain of the application.
"""

# Application Registry
INSTALLED_APPS = [
    'apps.authentication',    # User authentication and authorization
    'apps.predictions',       # Stock prediction functionality
    'apps.market_data',      # Market data fetching and processing
    'apps.portfolio',        # Portfolio management
    'apps.analysis',         # Technical and fundamental analysis
    'apps.common',          # Shared utilities and components
]

# Module Dependencies
MODULE_DEPENDENCIES = {
    'predictions': ['market_data', 'common'],
    'portfolio': ['market_data', 'predictions', 'common'],
    'analysis': ['market_data', 'predictions', 'common'],
    'market_data': ['common'],
    'authentication': ['common'],
    'common': []
}

# Service Layer Architecture
SERVICE_LAYERS = {
    'data_layer': [
        'market_data.services.data_fetcher',
        'market_data.services.data_processor',
    ],
    'business_layer': [
        'predictions.services.prediction_service',
        'portfolio.services.portfolio_manager',
        'analysis.services.technical_analysis',
    ],
    'presentation_layer': [
        'predictions.views',
        'portfolio.views',
        'analysis.views',
    ]
}
```

8.1.3 Configuration Files

Django Settings Configuration

```
# backend/StockVibePredictor/settings/base.py
"""
Base settings for StockVibePredictor project.
"""

import os
from pathlib import Path
from datetime import timedelta
import environ

# Build paths
BASE_DIR = Path(__file__).resolve().parent.parent.parent

# Environment variables
env = environ.Env(
    DEBUG=(bool, False),
    SECRET_KEY=(str, 'change-me-in-production'),
    DATABASE_URL=(str, 'sqlite:///db.sqlite3'),
    REDIS_URL=(str, 'redis://localhost:6379/0'),
    ALLOWED_HOSTS=(list, ['localhost']),
)

# Read .env file
environ.Env.read_env(os.path.join(BASE_DIR, '.env'))

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = env('SECRET_KEY')

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = env('DEBUG')

ALLOWED_HOSTS = env('ALLOWED_HOSTS')

# Application definition
DJANGO_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]

THIRD_PARTY_APPS = [
    'rest_framework',
    'rest_framework.authtoken',
    'corsheaders',
    'drf_spectacular',
```

```
    'django_celery_beat',
    'django_celery_results',
    'storages',
]

LOCAL_APPS = [
    'apps.common',
    'apps.authentication',
    'apps.predictions',
    'apps.market_data',
    'apps.portfolio',
    'apps.analysis',
]

INSTALLED_APPS = DJANGO_APPS + THIRD_PARTY_APPS + LOCAL_APPS

# Middleware configuration
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'apps.common.middleware.RequestLoggingMiddleware',
    'apps.common.middleware.PerformanceMonitoringMiddleware',
    'apps.common.middleware.ErrorHandlingMiddleware',
]

ROOT_URLCONF = 'StockVibePredictor.urls'

# Template configuration
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'StockVibePredictor.wsgi.application'
ASGI_APPLICATION = 'StockVibePredictor.asgi.application'

# Database configuration
DATABASES = {
    'default': env.db()
```

```
}

# Cache configuration
CACHES = {
    'default': {
        'BACKEND': 'django_redis.cache.RedisCache',
        'LOCATION': env('REDIS_URL'),
        'OPTIONS': {
            'CLIENT_CLASS': 'django_redis.client.DefaultClient',
            'CONNECTION_POOL_KWARGS': {
                'max_connections': 50,
                'retry_on_timeout': True,
            },
            'SOCKET_CONNECT_TIMEOUT': 5,
            'SOCKET_TIMEOUT': 5,
            'COMPRESSOR': 'django_redis.compressors.zlib.ZlibCompressor',
            'IGNORE_EXCEPTIONS': True,
        }
    }
}

# Password validation
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
        'OPTIONS': {
            'min_length': 8,
        }
    },
    {
        'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True

# Static files (CSS, JavaScript, Images)
STATIC_URL = 'static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'
STATICFILES_DIRS = [BASE_DIR / 'static']

# Media files
```

```
MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'

# Default primary key field type
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

# REST Framework configuration
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
        'rest_framework.authentication.SessionAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    'DEFAULT_RENDERER_CLASSES': [
        'rest_framework.renderers.JSONRenderer',
    ],
    'DEFAULT_PARSER_CLASSES': [
        'rest_framework.parsers.JSONParser',
        'rest_framework.parsers.MultiPartParser',
        'rest_framework.parsers.FormParser',
    ],
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 100,
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle',
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '100/hour',
        'user': '1000/hour',
        'prediction': '100/hour',
        'trading': '50/hour',
    },
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
    'EXCEPTION_HANDLER': 'apps.common.exceptions.custom_exception_handler',
}

# JWT Settings
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
    'UPDATE_LAST_LOGIN': True,
    'ALGORITHM': 'HS256',
    'SIGNING_KEY': SECRET_KEY,
    'AUTH_HEADER_TYPES': ('Bearer',),
    'AUTH_HEADER_NAME': 'HTTP_AUTHORIZATION',
```

```
    'USER_ID_FIELD': 'id',
    'USER_ID_CLAIM': 'user_id',
}

# CORS configuration
CORS_ALLOWED_ORIGINS = env.list('CORS_ALLOWED_ORIGINS', default=[
    'http://localhost:3000',
    'http://127.0.0.1:3000',
])

CORS_ALLOW_CREDENTIALS = True

CORS_ALLOW_HEADERS = [
    'accept',
    'accept-encoding',
    'authorization',
    'content-type',
    'dnt',
    'origin',
    'user-agent',
    'x-csrftoken',
    'x-requested-with',
]

# Celery Configuration
CELERY_BROKER_URL = env('REDIS_URL')
CELERY_RESULT_BACKEND = env('REDIS_URL')
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'
CELERY_RESULT_SERIALIZER = 'json'
CELERY_TIMEZONE = TIME_ZONE
CELERY_TASK_TRACK_STARTED = True
CELERY_TASK_TIME_LIMIT = 30 * 60  # 30 minutes
CELERY_BEAT_SCHEDULER = 'django_celery_beat.schedulers:DatabaseScheduler'

# Celery Beat Schedule
CELERY_BEAT_SCHEDULE = {
    'update-market-data': {
        'task': 'apps.market_data.tasks.update_market_data',
        'schedule': timedelta(minutes=5),
        'options': {'expires': 60.0},
    },
    'train-models': {
        'task': 'apps.predictions.tasks.train_models',
        'schedule': timedelta(hours=24),
        'options': {'expires': 3600.0},
    },
    'cleanup-old-data': {
        'task': 'apps.common.tasks.cleanup_old_data',
        'schedule': timedelta(hours=1),
        'options': {'expires': 300.0},
    },
    'generate-reports': {
        'task': 'apps.analysis.tasks.generate_daily_report',
        'schedule': timedelta(hours=24),
```

```
        'options': {'expires': 3600.0},
    },
}

# Logging configuration
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'verbose': {
            'format': '{levelname} {asctime} {module} {process:d} {thread:d}
{message}',
            'style': '{',
        },
        'simple': {
            'format': '{levelname} {message}',
            'style': '{',
        },
        'json': {
            '()': 'pythonjsonlogger.jsonlogger.JsonFormatter',
            'format': '%(levelname)s %(asctime)s %(module)s %(process)d
%(thread)d %(message)s'
        }
    },
    'filters': {
        'require_debug_true': {
            '()': 'django.utils.log.RequireDebugTrue',
        },
        'require_debug_false': {
            '()': 'django.utils.log.RequireDebugFalse',
        },
    },
    'handlers': {
        'console': {
            'level': 'INFO',
            'class': 'logging.StreamHandler',
            'formatter': 'verbose'
        },
        'file': {
            'level': 'INFO',
            'class': 'logging.handlers.RotatingFileHandler',
            'filename': BASE_DIR / 'logs' / 'django.log',
            'maxBytes': 1024 * 1024 * 50,  # 50 MB
            'backupCount': 5,
            'formatter': 'json',
        },
        'error_file': {
            'level': 'ERROR',
            'class': 'logging.handlers.RotatingFileHandler',
            'filename': BASE_DIR / 'logs' / 'errors.log',
            'maxBytes': 1024 * 1024 * 50,  # 50 MB
            'backupCount': 5,
            'formatter': 'json',
        },
    },
```

```
    'root': {
        'handlers': ['console', 'file'],
        'level': 'INFO',
    },
    'loggers': {
        'django': {
            'handlers': ['console', 'file'],
            'level': env('DJANGO_LOG_LEVEL', default='INFO'),
            'propagate': False,
        },
        'apps': {
            'handlers': ['console', 'file', 'error_file'],
            'level': 'DEBUG',
            'propagate': False,
        },
        'celery': {
            'handlers': ['console', 'file'],
            'level': 'INFO',
            'propagate': False,
        },
    },
}

# Custom settings
STOCKVIBE_CONFIG = {
    'MODEL_VERSION': '5.0',
    'MIN_PREDICTION_ACCURACY': 0.55,
    'MAX_CACHED_PREDICTIONS': 10000,
    'DATA_RETENTION_DAYS': 365,
    'MODEL_TRAINING_SCHEDULE': 'daily',
    'SUPPORTED_TIMEFRAMES': ['1d', '5d', '1w', '1mo', '3mo', '6mo', '1y', '2y',
'5y'],
    'MAX_CONCURRENT_PREDICTIONS': 100,
    'PREDICTION_TIMEOUT': 30,  # seconds
}
```

8.1.4 Build Scripts

Docker Build Configuration

```
# docker/backend/Dockerfile
FROM python:3.10-slim

# Set environment variables
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV APP_HOME=/app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    g++ \
    python3-dev \
    postgresql-client \
    netcat \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Create app directory
WORKDIR $APP_HOME

# Install Python dependencies
COPY requirements/production.txt .
RUN pip install --upgrade pip && \
    pip install --no-cache-dir -r production.txt

# Copy application code
COPY . .

# Create non-root user
RUN useradd -m -u 1000 appuser && \
    chown -R appuser:appuser $APP_HOME

USER appuser

# Collect static files
RUN python manage.py collectstatic --noinput

# Run gunicorn
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "4", "--threads",
"4", "StockVibePredictor.wsgi:application"]
```

Deployment Script

```
#!/bin/bash
# scripts/deploy.sh
# Deployment script for StockVibePredictor

set -e  # Exit on error

# Colors for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m' # No Color

echo -e "${GREEN}Starting StockVibePredictor Deployment${NC}"

# Environment check
if [ -z "$DEPLOY_ENV" ]; then
    echo -e "${RED}DEPLOY_ENV not set. Use: staging or production${NC}"
    exit 1
fi

# Function to check command success
check_status() {
    if [ $? -eq 0 ]; then
        echo -e "${GREEN}✓ $1${NC}"
    else
        echo -e "${RED}✗ $1 failed${NC}"
        exit 1
    fi
}

# Pull latest code
echo -e "${YELLOW}Pulling latest code...${NC}"
git pull origin main
check_status "Git pull"

# Build Docker images
echo -e "${YELLOW}Building Docker images...${NC}"
docker-compose -f docker-compose.$DEPLOY_ENV.yml build
check_status "Docker build"

# Run database migrations
echo -e "${YELLOW}Running database migrations...${NC}"
docker-compose -f docker-compose.$DEPLOY_ENV.yml run --rm backend python manage.py migrate
check_status "Database migrations"

# Collect static files
echo -e "${YELLOW}Collecting static files...${NC}"
docker-compose -f docker-compose.$DEPLOY_ENV.yml run --rm backend python manage.py collectstatic
--noinput
check_status "Static files collection"

# Start services
echo -e "${YELLOW}Starting services...${NC}"
docker-compose -f docker-compose.$DEPLOY_ENV.yml up -d
check_status "Service startup"

# Health check
echo -e "${YELLOW}Performing health check...${NC}"
sleep 10
curl -f http://localhost:8000/api/health/ || exit 1
check_status "Health check"

echo -e "${GREEN}Deployment completed successfully!${NC}"
```

8.2 Core Modules

8.2.1 Prediction Service Module

```
# backend/apps/predictions/services/prediction_service.py
"""
Core prediction service module for generating stock predictions.

This module handles the entire prediction pipeline including:
- Data fetching and validation
- Feature engineering
- Model selection and execution
- Result formatting and caching
"""

import logging
import hashlib
import json
from typing import Dict, List, Optional, Tuple, Any
from datetime import datetime, timedelta
import numpy as np
import pandas as pd
from django.core.cache import cache
from django.conf import settings

from apps.market_data.services import DataFetcher, DataProcessor
from apps.predictions.models import PredictionLog, ModelRegistry
from apps.common.exceptions import (
    DataNotAvailableError,
    ModelNotFoundError,
    PredictionError,
    ValidationError
)
from .model_service import ModelService
from .feature_service import FeatureEngineer

logger = logging.getLogger(__name__)


class PredictionService:
    """
    Main service class for generating stock predictions.

    This service orchestrates the entire prediction process from data fetching
    to returning formatted predictions with confidence scores.
    """

    def __init__(self):
        """
        Initialize the prediction service with required dependencies.
        """
        self.data_fetcher = DataFetcher()
        self.data_processor = DataProcessor()
        self.feature_engineer = FeatureEngineer()
        self.model_service = ModelService()
        self.cache_ttl = settings.STOCKVIBE_CONFIG.get('CACHE_TTL', 300)
```

```
    def generate_prediction(
        self,
        ticker: str,
        timeframe: str,
        user_id: Optional[int] = None,
        include_analysis: bool = True,
        force_refresh: bool = False
    ) -> Dict[str, Any]:
        """
        Generate prediction for a given ticker and timeframe.

        Parameters:
        -----------
        ticker : str
            Stock ticker symbol (e.g., 'AAPL', 'GOOGL')
        timeframe : str
            Prediction timeframe ('1d', '1w', '1mo', etc.)
        user_id : Optional[int]
            User ID for logging purposes
        include_analysis : bool
            Whether to include technical analysis in response
        force_refresh : bool
            Force bypass cache and generate fresh prediction

        Returns:
        --------
        Dict[str, Any]
            Prediction result containing direction, confidence, analysis, etc.

        Raises:
        -------
        ValidationError
            If ticker or timeframe is invalid
        DataNotAvailableError
            If market data cannot be fetched
        ModelNotFoundError
            If no suitable model is available
        PredictionError
            If prediction generation fails
        """

        # Input validation
        self._validate_inputs(ticker, timeframe)

        # Check cache unless force refresh
        if not force_refresh:
            cached_result = self._get_cached_prediction(ticker, timeframe)
            if cached_result:
                logger.info(f"Cache hit for {ticker} {timeframe}")
                return cached_result

        try:
            # Fetch market data
            logger.info(f"Fetching data for {ticker} ({timeframe})")
            market_data = self._fetch_market_data(ticker, timeframe)

            # Process and engineer features
            logger.info(f"Engineering features for {ticker}")
```

```
            features = self._engineer_features(market_data, timeframe)

            # Get appropriate model
            logger.info(f"Selecting model for {ticker} ({timeframe})")
            model = self._select_model(ticker, timeframe)

            # Generate prediction
            logger.info(f"Generating prediction for {ticker}")
            prediction = self._make_prediction(model, features)

            # Enhance with additional metrics
            result = self._enhance_prediction(
                prediction,
                market_data,
                model,
                include_analysis
            )

            # Cache result
            self._cache_prediction(ticker, timeframe, result)

            # Log prediction
            self._log_prediction(ticker, timeframe, result, user_id)

            logger.info(f"Successfully generated prediction for {ticker}
({timeframe})")
            return result

        except Exception as e:
            logger.error(f"Prediction failed for {ticker} ({timeframe}): {str(e)}")
            raise PredictionError(f"Failed to generate prediction: {str(e)}")

    def generate_batch_predictions(
        self,
        tickers: List[str],
        timeframe: str,
        user_id: Optional[int] = None
    ) -> Dict[str, Dict[str, Any]]:
        """
        Generate predictions for multiple tickers.

        Parameters:
        -----------
        tickers : List[str]
            List of ticker symbols
        timeframe : str
            Prediction timeframe
        user_id : Optional[int]
            User ID for logging

        Returns:
        --------
        Dict[str, Dict[str, Any]]
            Dictionary mapping tickers to their predictions
        """
        results = {}

        for ticker in tickers:
```

```
            try:
                results[ticker] = self.generate_prediction(
                    ticker,
                    timeframe,
                    user_id,
                    include_analysis=False  # Simplified for batch
                )
            except Exception as e:
                logger.error(f"Batch prediction failed for {ticker}: {str(e)}")
                results[ticker] = {
                    'error': str(e),
                    'status': 'failed'
                }

        return results

    def _validate_inputs(self, ticker: str, timeframe: str) -> None:
        """
        Validate prediction inputs.

        Parameters:
        -----------
        ticker : str
            Stock ticker symbol
        timeframe : str
            Prediction timeframe

        Raises:
        -------
        ValidationError
            If inputs are invalid
        """
        # Validate ticker format
        if not ticker or not isinstance(ticker, str):
            raise ValidationError("Invalid ticker symbol")

        if not ticker.replace('-', '').replace('.', '').replace('^', '').isalnum():
            raise ValidationError(f"Invalid ticker format: {ticker}")

        # Validate timeframe
        valid_timeframes = settings.STOCKVIBE_CONFIG.get('SUPPORTED_TIMEFRAMES', [])
        if timeframe not in valid_timeframes:
            raise ValidationError(
                f"Invalid timeframe: {timeframe}. "
                f"Valid options: {', '.join(valid_timeframes)}"
            )

    def _fetch_market_data(self, ticker: str, timeframe: str) -> pd.DataFrame:
        """
        Fetch market data for the given ticker and timeframe.

        Parameters:
        -----------
        ticker : str
            Stock ticker symbol
        timeframe : str
            Data timeframe
```

```
        Returns:
        --------
        pd.DataFrame
            Market data with OHLCV columns

        Raises:
        -------
        DataNotAvailableError
            If data cannot be fetched
        """
        try:
            data = self.data_fetcher.fetch_stock_data(ticker, timeframe)

            if data is None or data.empty:
                raise DataNotAvailableError(f"No data available for {ticker}")

            # Validate data quality
            if len(data) < 50:  # Minimum data points required
                raise DataNotAvailableError(
                    f"Insufficient data for {ticker}: {len(data)} points"
                )

            return data

        except Exception as e:
            logger.error(f"Data fetch failed for {ticker}: {str(e)}")
            raise DataNotAvailableError(f"Failed to fetch data: {str(e)}")

    def _engineer_features(
        self,
        data: pd.DataFrame,
        timeframe: str
    ) -> pd.DataFrame:
        """
        Engineer features from market data.

        Parameters:
        -----------
        data : pd.DataFrame
            Raw market data
        timeframe : str
            Timeframe for feature engineering

        Returns:
        --------
        pd.DataFrame
            Data with engineered features
        """
        # Process data
        processed_data = self.data_processor.process_data(data)

        # Engineer features
        features = self.feature_engineer.engineer_features(
            processed_data,
            timeframe
        )

        return features
```

```
    def _select_model(self, ticker: str, timeframe: str) -> Dict[str, Any]:
        """
        Select the best model for prediction.

        Parameters:
        -----------
        ticker : str
            Stock ticker symbol
        timeframe : str
            Prediction timeframe

        Returns:
        --------
        Dict[str, Any]
            Model information including model object and metadata

        Raises:
        -------
        ModelNotFoundError
            If no suitable model is found
        """
        # Try ticker-specific model first
        model = self.model_service.get_model(ticker, timeframe)

        if model is None:
            # Fall back to universal model
            model = self.model_service.get_universal_model(timeframe)

        if model is None:
            raise ModelNotFoundError(
                f"No model available for {ticker} ({timeframe})"
            )

        return model

    def _make_prediction(
        self,
        model: Dict[str, Any],
        features: pd.DataFrame
    ) -> Dict[str, Any]:
        """
        Generate prediction using the model.

        Parameters:
        -----------
        model : Dict[str, Any]
            Model information
        features : pd.DataFrame
            Feature data

        Returns:
        --------
        Dict[str, Any]
            Raw prediction result
        """
        # Extract latest features
        latest_features = features.iloc[-1]
```

```
        # Get required features for model
        required_features = model['features']
        feature_vector = latest_features[required_features].values.reshape(1, -1)

        # Scale features if scaler available
        if 'scaler' in model:
            feature_vector = model['scaler'].transform(feature_vector)

        # Make prediction
        model_obj = model['model']
        prediction = model_obj.predict(feature_vector)[0]

        # Get probability if available
        confidence = 0.5
        if hasattr(model_obj, 'predict_proba'):
            proba = model_obj.predict_proba(feature_vector)[0]
            confidence = proba[int(prediction)]

        return {
            'direction': 'UP' if prediction == 1 else 'DOWN',
            'confidence': float(confidence),
            'raw_prediction': int(prediction)
        }

    def _enhance_prediction(
        self,
        prediction: Dict[str, Any],
        market_data: pd.DataFrame,
        model: Dict[str, Any],
        include_analysis: bool
    ) -> Dict[str, Any]:
        """
        Enhance prediction with additional metrics and analysis.

        Parameters:
        -----------
        prediction : Dict[str, Any]
            Raw prediction result
        market_data : pd.DataFrame
            Market data
        model : Dict[str, Any]
            Model information
        include_analysis : bool
            Whether to include technical analysis

        Returns:
        --------
        Dict[str, Any]
            Enhanced prediction result
        """
        current_price = float(market_data['Close'].iloc[-1])
        volatility = float(market_data['Close'].pct_change().std())

        # Calculate price target
        if prediction['direction'] == 'UP':
            price_target = current_price * (1 + volatility * 2)
        else:
```

```
            price_target = current_price * (1 - volatility * 2)

        # Calculate expected return
        expected_return = ((price_target / current_price) - 1) * 100

        result = {
            'ticker': market_data.index.name or 'Unknown',
            'direction': prediction['direction'],
            'confidence': round(prediction['confidence'] * 100, 2),
            'price_target': round(price_target, 2),
            'current_price': round(current_price, 2),
            'expected_return': round(expected_return, 2),
            'model_accuracy': round(model.get('accuracy', 0.5) * 100, 2),
            'model_type': model.get('type', 'unknown'),
            'timestamp': datetime.now().isoformat()
        }

        if include_analysis:
            result['analysis'] = self._generate_analysis(market_data)

        return result

    def _generate_analysis(self, data: pd.DataFrame) -> Dict[str, Any]:
        """
        Generate technical analysis for the data.

        Parameters:
        -----------
        data : pd.DataFrame
            Market data with indicators

        Returns:
        --------
        Dict[str, Any]
            Technical analysis results
        """
        latest = data.iloc[-1]

        # RSI Analysis
        rsi = latest.get('RSI', 50)
        rsi_signal = 'overbought' if rsi > 70 else 'oversold' if rsi < 30 else
'neutral'

        # MACD Analysis
        macd = latest.get('MACD', 0)
        macd_signal = latest.get('MACD_Signal', 0)
        macd_trend = 'bullish' if macd > macd_signal else 'bearish'

        # Moving Average Analysis
        ma20 = latest.get('MA20', current_price)
        ma50 = latest.get('MA50', current_price)
        current_price = latest['Close']

        trend = 'bullish' if current_price > ma20 > ma50 else 'bearish'

        return {
            'technical_indicators': {
                'rsi': {
```

```
                'value': round(rsi, 2),
                'signal': rsi_signal
            },
            'macd': {
                'value': round(macd, 4),
                'signal': round(macd_signal, 4),
                'trend': macd_trend
            },
            'moving_averages': {
                'ma20': round(ma20, 2),
                'ma50': round(ma50, 2),
                'trend': trend
            }
        },
        'support_resistance': {
            'support': round(data['Low'].tail(20).min(), 2),
            'resistance': round(data['High'].tail(20).max(), 2)
        },
        'volatility': {
            'daily': round(data['Close'].pct_change().std() * 100, 2),
            'weekly': round(data['Close'].pct_change(5).std() * 100, 2)
        }
    }

def _get_cached_prediction(
    self,
    ticker: str,
    timeframe: str
) -> Optional[Dict[str, Any]]:
    """
    Get cached prediction if available.

    Parameters:
    -----------
    ticker : str
        Stock ticker symbol
    timeframe : str
        Prediction timeframe

    Returns:
    --------
    Optional[Dict[str, Any]]
        Cached prediction or None
    """
    cache_key = f"prediction:{ticker}:{timeframe}"
    return cache.get(cache_key)

def _cache_prediction(
    self,
    ticker: str,
    timeframe: str,
    result: Dict[str, Any]
) -> None:
    """
    Cache prediction result.

    Parameters:
    -----------
```

```
        ticker : str
            Stock ticker symbol
        timeframe : str
            Prediction timeframe
        result : Dict[str, Any]
            Prediction result to cache
        """
        cache_key = f"prediction:{ticker}:{timeframe}"
        cache.set(cache_key, result, self.cache_ttl)

    def _log_prediction(
        self,
        ticker: str,
        timeframe: str,
        result: Dict[str, Any],
        user_id: Optional[int]
    ) -> None:
        """
        Log prediction to database.

        Parameters:
        -----------
        ticker : str
            Stock ticker symbol
        timeframe : str
            Prediction timeframe
        result : Dict[str, Any]
            Prediction result
        user_id : Optional[int]
            User ID if available
        """
        try:
            PredictionLog.objects.create(
                user_id=user_id,
                ticker=ticker,
                timeframe=timeframe,
                direction=result['direction'],
                confidence=result['confidence'],
                price_target=result.get('price_target'),
                current_price=result.get('current_price'),
                model_accuracy=result.get('model_accuracy'),
                model_type=result.get('model_type')
            )
        except Exception as e:
            logger.error(f"Failed to log prediction: {str(e)}")
```

8.2.2 Market Data Service Module

```
# backend/apps/market_data/services/data_fetcher.py
"""
Market data fetching service with multi-source support and fallback mechanisms.

This module handles:
- Real-time and historical data fetching
- Multiple data source integration
- Automatic fallback on failure
- Rate limiting and caching
"""

import logging
import time
from typing import Dict, List, Optional, Union
from datetime import datetime, timedelta
import pandas as pd
import yfinance as yf
import requests
from django.conf import settings
from django.core.cache import cache

from apps.common.exceptions import DataNotAvailableError, RateLimitError

logger = logging.getLogger(__name__)


class DataFetcher:
    """
    Service for fetching market data from multiple sources.

    Supports Yahoo Finance, Alpha Vantage, and Polygon.io with
    automatic fallback and rate limiting.
    """

    def __init__(self):
        """Initialize data fetcher with configuration."""
        self.sources = {
            'yahoo': YahooDataSource(),
            'alpha_vantage': AlphaVantageDataSource(),
            'polygon': PolygonDataSource()
        }
        self.default_source = 'yahoo'
        self.rate_limiter = RateLimiter()

    def fetch_stock_data(
        self,
        ticker: str,
        timeframe: str = '1d',
        source: Optional[str] = None
    ) -> pd.DataFrame:
        """
        Fetch stock data with automatic source fallback.

        Parameters:
```

```
        ----------
        ticker : str
            Stock ticker symbol
        timeframe : str
            Data timeframe
        source : Optional[str]
            Preferred data source

        Returns:
        --------
        pd.DataFrame
            Stock data with OHLCV columns

        Raises:
        -------
        DataNotAvailableError
            If data cannot be fetched from any source
        """
        # Check cache first
        cache_key = f"market_data:{ticker}:{timeframe}"
        cached_data = cache.get(cache_key)
        if cached_data is not None:
            logger.info(f"Cache hit for {ticker} ({timeframe})")
            return cached_data

        # Determine source priority
        sources_to_try = [source] if source else ['yahoo', 'alpha_vantage',
'polygon']

        last_error = None
        for src in sources_to_try:
            if src not in self.sources:
                continue

            try:
                # Check rate limit
                if not self.rate_limiter.can_make_request(src):
                    wait_time = self.rate_limiter.get_wait_time(src)
                    logger.warning(f"Rate limit for {src}, waiting {wait_time}s")
                    continue

                # Fetch data
                logger.info(f"Fetching {ticker} from {src}")
                data = self.sources[src].fetch(ticker, timeframe)

                if data is not None and not data.empty:
                    # Cache successful fetch
                    cache_ttl = self._get_cache_ttl(timeframe)
                    cache.set(cache_key, data, cache_ttl)

                    # Record successful request
                    self.rate_limiter.record_request(src)

                    logger.info(f"Successfully fetched {ticker} from {src}")
                    return data
```

```
            except Exception as e:
                last_error = e
                logger.error(f"Failed to fetch {ticker} from {src}: {str(e)}")
                continue

        # All sources failed
        error_msg = f"Failed to fetch data for {ticker} from any source"
        if last_error:
            error_msg += f": {str(last_error)}"

        raise DataNotAvailableError(error_msg)

    def fetch_batch_data(
        self,
        tickers: List[str],
        timeframe: str = '1d'
    ) -> Dict[str, pd.DataFrame]:
        """
        Fetch data for multiple tickers.

        Parameters:
        -----------
        tickers : List[str]
            List of ticker symbols
        timeframe : str
            Data timeframe

        Returns:
        --------
        Dict[str, pd.DataFrame]
            Dictionary mapping tickers to their data
        """
        results = {}

        for ticker in tickers:
            try:
                data = self.fetch_stock_data(ticker, timeframe)
                results[ticker] = data
            except Exception as e:
                logger.error(f"Failed to fetch {ticker}: {str(e)}")
                results[ticker] = None

        return results

    def _get_cache_ttl(self, timeframe: str) -> int:
        """
        Get cache TTL based on timeframe.

        Parameters:
        -----------
        timeframe : str
            Data timeframe

        Returns:
```

```
        --------
        int
            Cache TTL in seconds
        """
        ttl_map = {
            '1d': 300,      # 5 minutes
            '5d': 900,      # 15 minutes
            '1w': 1800,     # 30 minutes
            '1mo': 3600,    # 1 hour
            '3mo': 7200,    # 2 hours
            '6mo': 14400,   # 4 hours
            '1y': 21600,    # 6 hours
            '2y': 43200,    # 12 hours
            '5y': 86400,    # 24 hours
        }
        return ttl_map.get(timeframe, 3600)


class YahooDataSource:
    """Yahoo Finance data source implementation."""

    def fetch(self, ticker: str, timeframe: str) -> pd.DataFrame:
        """
        Fetch data from Yahoo Finance.

        Parameters:
        -----------
        ticker : str
            Stock ticker symbol
        timeframe : str
            Data timeframe

        Returns:
        --------
        pd.DataFrame
            Stock data
        """
        config = self._get_config(timeframe)

        try:
            stock = yf.Ticker(ticker)
            data = stock.history(
                period=config['period'],
                interval=config['interval'],
                auto_adjust=True,
                prepost=True
            )

            if data.empty:
                # Try alternative download method
                data = yf.download(
                    ticker,
                    period=config['period'],
                    interval=config['interval'],
                    auto_adjust=True,
```

```
                    progress=False
                )

            return data

        except Exception as e:
            logger.error(f"Yahoo Finance error for {ticker}: {str(e)}")
            raise

    def _get_config(self, timeframe: str) -> Dict[str, str]:
        """Get Yahoo Finance configuration for timeframe."""
        configs = {
            '1d': {'period': '7d', 'interval': '5m'},
            '5d': {'period': '1mo', 'interval': '15m'},
            '1w': {'period': '2mo', 'interval': '1h'},
            '1mo': {'period': '3mo', 'interval': '1d'},
            '3mo': {'period': '6mo', 'interval': '1d'},
            '6mo': {'period': '1y', 'interval': '1d'},
            '1y': {'period': '2y', 'interval': '1d'},
            '2y': {'period': '3y', 'interval': '1wk'},
            '5y': {'period': 'max', 'interval': '1mo'},
        }
        return configs.get(timeframe, {'period': '1mo', 'interval': '1d'})


class AlphaVantageDataSource:
    """Alpha Vantage data source implementation."""

    def __init__(self):
        """Initialize with API key."""
        self.api_key = settings.ALPHA_VANTAGE_API_KEY
        self.base_url = 'https://www.alphavantage.co/query'

    def fetch(self, ticker: str, timeframe: str) -> pd.DataFrame:
        """
        Fetch data from Alpha Vantage.

        Parameters:
        -----------
        ticker : str
            Stock ticker symbol
        timeframe : str
            Data timeframe

        Returns:
        --------
        pd.DataFrame
            Stock data
        """
        if not self.api_key:
            raise ValueError("Alpha Vantage API key not configured")

        function = self._get_function(timeframe)

        params = {
```

```
            'function': function,
            'symbol': ticker,
            'apikey': self.api_key,
            'outputsize': 'full',
            'datatype': 'json'
        }

        try:
            response = requests.get(self.base_url, params=params, timeout=30)
            response.raise_for_status()

            data_json = response.json()

            # Check for API errors
            if 'Error Message' in data_json:
                raise ValueError(data_json['Error Message'])

            if 'Note' in data_json:
                raise RateLimitError("Alpha Vantage API rate limit reached")

            # Parse data based on function
            df = self._parse_response(data_json, function)

            return df

        except Exception as e:
            logger.error(f"Alpha Vantage error for {ticker}: {str(e)}")
            raise

    def _get_function(self, timeframe: str) -> str:
        """Get Alpha Vantage function for timeframe."""
        if timeframe in ['1d', '5d']:
            return 'TIME_SERIES_INTRADAY'
        elif timeframe in ['1w', '1mo']:
            return 'TIME_SERIES_DAILY'
        else:
            return 'TIME_SERIES_WEEKLY'

    def _parse_response(self, data: Dict, function: str) -> pd.DataFrame:
        """Parse Alpha Vantage response to DataFrame."""
        # Implementation would parse the specific response format
        # This is simplified for brevity
        pass


class PolygonDataSource:
    """Polygon.io data source implementation."""

    def __init__(self):
        """Initialize with API key."""
        self.api_key = settings.POLYGON_API_KEY
        self.base_url = 'https://api.polygon.io/v2'

    def fetch(self, ticker: str, timeframe: str) -> pd.DataFrame:
        """
```

```
        Fetch data from Polygon.io.

        Implementation similar to AlphaVantageDataSource
        """
        pass


class RateLimiter:
    """Rate limiting for API calls."""

    def __init__(self):
        """Initialize rate limiter."""
        self.limits = {
            'yahoo': {'calls': 2000, 'period': 3600},
            'alpha_vantage': {'calls': 5, 'period': 60},
            'polygon': {'calls': 5, 'period': 60}
        }

    def can_make_request(self, source: str) -> bool:
        """Check if request can be made."""
        cache_key = f"rate_limit:{source}"
        current_count = cache.get(cache_key, 0)

        limit = self.limits.get(source, {}).get('calls', float('inf'))

        return current_count < limit

    def record_request(self, source: str) -> None:
        """Record an API request."""
        cache_key = f"rate_limit:{source}"
        period = self.limits.get(source, {}).get('period', 3600)

        current_count = cache.get(cache_key, 0)
        cache.set(cache_key, current_count + 1, period)

    def get_wait_time(self, source: str) -> int:
        """Get time to wait before next request."""
        cache_key = f"rate_limit:{source}"
        ttl = cache.ttl(cache_key)

        return max(0, ttl) if ttl else 0
```

8.3 API Endpoints

8.3.1 Complete API Documentation

```
# backend/apps/predictions/urls.py
"""
API URL configuration for predictions app.
"""

from django.urls import path, include
from rest_framework.routers import DefaultRouter

from .views import (
    PredictionViewSet,
    BatchPredictionView,
    ModelPerformanceView,
    PredictionHistoryView
)

router = DefaultRouter()
router.register(r'predictions', PredictionViewSet, basename='prediction')

urlpatterns = [
    path('', include(router.urls)),
    path('batch/', BatchPredictionView.as_view(), name='batch-prediction'),
    path('performance/', ModelPerformanceView.as_view(), name='model-
performance'),
    path('history/', PredictionHistoryView.as_view(), name='prediction-
history'),
]
```

```
# backend/apps/predictions/views.py
"""
API views for prediction endpoints.
"""

from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from rest_framework.throttling import UserRateThrottle
from rest_framework.views import APIView
from drf_spectacular.utils import extend_schema, OpenApiParameter
from django.core.cache import cache
import logging

from .serializers import (
    PredictionRequestSerializer,
    PredictionResponseSerializer,
    BatchPredictionRequestSerializer,
    ModelPerformanceSerializer
)
from .services import PredictionService
from .models import PredictionLog
```

```
logger = logging.getLogger(__name__)


class PredictionRateThrottle(UserRateThrottle):
    """Custom rate throttle for prediction endpoints."""
    rate = '100/hour'


class PredictionViewSet(viewsets.ViewSet):
    """
    ViewSet for stock predictions.

    Provides endpoints for generating single and multi-timeframe predictions.
    """

    permission_classes = [IsAuthenticated]
    throttle_classes = [PredictionRateThrottle]

    @extend_schema(
        summary="Generate stock prediction",
        description="""
        Generate a prediction for a specific stock and timeframe.

        The prediction includes:
        - Direction (UP/DOWN)
        - Confidence percentage
        - Price target
        - Expected return
        - Technical analysis (optional)
        """,
        request=PredictionRequestSerializer,
        responses={
            200: PredictionResponseSerializer,
            400: {'description': 'Bad request'},
            404: {'description': 'Ticker not found'},
            500: {'description': 'Internal server error'}
        },
        tags=['predictions']
    )
    def create(self, request):
        """
        Generate a stock prediction.

        POST /api/predictions/

        Request Body:
        {
            "ticker": "AAPL",
            "timeframe": "1d",
            "include_analysis": true
        }

        Response:
        {
            "ticker": "AAPL",
            "direction": "UP",
            "confidence": 72.5,
            "price_target": 185.50,
            "current_price": 180.25,
            "expected_return": 2.91,
            "model_accuracy": 65.3,
            "analysis": {...}
```

```
        }
        """
        serializer = PredictionRequestSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        ticker = serializer.validated_data['ticker']
        timeframe = serializer.validated_data['timeframe']
        include_analysis = serializer.validated_data.get('include_analysis', True)

        try:
            service = PredictionService()
            result = service.generate_prediction(
                ticker=ticker,
                timeframe=timeframe,
                user_id=request.user.id,
                include_analysis=include_analysis
            )

            return Response(result, status=status.HTTP_200_OK)

        except Exception as e:
            logger.error(f"Prediction failed: {str(e)}")
            return Response(
                {'error': str(e)},
                status=status.HTTP_500_INTERNAL_SERVER_ERROR
            )

    @extend_schema(
        summary="Generate multi-timeframe predictions",
        description="Generate predictions for multiple timeframes simultaneously",
        request={
            'application/json': {
                'type': 'object',
                'properties': {
                    'ticker': {'type': 'string'},
                    'timeframes': {
                        'type': 'array',
                        'items': {'type': 'string'}
                    }
                }
            }
        },
        tags=['predictions']
    )
    @action(detail=False, methods=['post'])
    def multi_timeframe(self, request):
        """
        Generate predictions for multiple timeframes.

        POST /api/predictions/multi_timeframe/
        """
        ticker = request.data.get('ticker')
        timeframes = request.data.get('timeframes', ['1d', '1w', '1mo'])

        if not ticker:
            return Response(
                {'error': 'Ticker is required'},
                status=status.HTTP_400_BAD_REQUEST
            )

        results = {}
        service = PredictionService()
```

```
        for timeframe in timeframes:
            try:
                result = service.generate_prediction(
                    ticker=ticker,
                    timeframe=timeframe,
                    user_id=request.user.id,
                    include_analysis=False
                )
                results[timeframe] = result
            except Exception as e:
                results[timeframe] = {'error': str(e)}

        return Response(results, status=status.HTTP_200_OK)

    @extend_schema(
        summary="Get prediction history",
        description="Retrieve user's prediction history",
        parameters=[
            OpenApiParameter(
                name='ticker',
                type=str,
                location=OpenApiParameter.QUERY,
                description='Filter by ticker'
            ),
            OpenApiParameter(
                name='days',
                type=int,
                location=OpenApiParameter.QUERY,
                description='Number of days to retrieve'
            )
        ],
        tags=['predictions']
    )
    @action(detail=False, methods=['get'])
    def history(self, request):
        """
        Get user's prediction history.

        GET /api/predictions/history/
        """
        ticker = request.query_params.get('ticker')
        days = int(request.query_params.get('days', 30))

        queryset = PredictionLog.objects.filter(
            user=request.user
        ).order_by('-created_at')

        if ticker:
            queryset = queryset.filter(ticker=ticker)

        # Limit by days
        from datetime import datetime, timedelta
        cutoff_date = datetime.now() - timedelta(days=days)
        queryset = queryset.filter(created_at__gte=cutoff_date)

        # Serialize and return
        data = [{
            'id': log.id,
            'ticker': log.ticker,
            'timeframe': log.timeframe,
            'direction': log.direction,
```

```
            'confidence': log.confidence,
            'price_target': log.price_target,
            'created_at': log.created_at.isoformat()
        } for log in queryset[:100]]  # Limit to 100 records

        return Response(data, status=status.HTTP_200_OK)


class BatchPredictionView(APIView):
    """
    API view for batch predictions.
    """

    permission_classes = [IsAuthenticated]
    throttle_classes = [PredictionRateThrottle]

    @extend_schema(
        summary="Generate batch predictions",
        description="Generate predictions for multiple tickers",
        request=BatchPredictionRequestSerializer,
        tags=['predictions']
    )
    def post(self, request):
        """
        Generate predictions for multiple tickers.

        POST /api/predictions/batch/

        Request Body:
        {
            "tickers": ["AAPL", "GOOGL", "MSFT"],
            "timeframe": "1d"
        }
        """
        serializer = BatchPredictionRequestSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        tickers = serializer.validated_data['tickers']
        timeframe = serializer.validated_data['timeframe']

        if len(tickers) > 20:
            return Response(
                {'error': 'Maximum 20 tickers allowed per request'},
                status=status.HTTP_400_BAD_REQUEST
            )

        service = PredictionService()
        results = service.generate_batch_predictions(
            tickers=tickers,
            timeframe=timeframe,
            user_id=request.user.id
        )

        return Response(results, status=status.HTTP_200_OK)
```

8.3.2 Request/Response Formats

```
# backend/apps/predictions/serializers.py
"""
Serializers for prediction API endpoints.
"""

from rest_framework import serializers
from .models import PredictionLog


class PredictionRequestSerializer(serializers.Serializer):
    """Serializer for prediction requests."""

    ticker = serializers.CharField(
        max_length=15,
        help_text="Stock ticker symbol (e.g., AAPL, GOOGL)"
    )
    timeframe = serializers.ChoiceField(
        choices=['1d', '5d', '1w', '1mo', '3mo', '6mo', '1y', '2y', '5y'],
        help_text="Prediction timeframe"
    )
    include_analysis = serializers.BooleanField(
        default=True,
        help_text="Include technical analysis in response"
    )
    force_refresh = serializers.BooleanField(
        default=False,
        help_text="Force refresh bypassing cache"
    )

    def validate_ticker(self, value):
        """Validate ticker format."""
        import re

        if not re.match(r'^[A-Z0-9\.\-\^]{1,15}$', value.upper()):
            raise serializers.ValidationError(
                "Invalid ticker format. Use uppercase letters, numbers, dots, dashes."
            )

        return value.upper()


class PredictionResponseSerializer(serializers.Serializer):
    """Serializer for prediction responses."""

    ticker = serializers.CharField()
    direction = serializers.ChoiceField(choices=['UP', 'DOWN'])
    confidence = serializers.FloatField(min_value=0, max_value=100)
    price_target = serializers.DecimalField(max_digits=10, decimal_places=2)
    current_price = serializers.DecimalField(max_digits=10, decimal_places=2)
    expected_return = serializers.FloatField()
    model_accuracy = serializers.FloatField()
    model_type = serializers.CharField()
    timestamp = serializers.DateTimeField()
    analysis = serializers.DictField(required=False)


class BatchPredictionRequestSerializer(serializers.Serializer):
    """Serializer for batch prediction requests."""

    tickers = serializers.ListField(
        child=serializers.CharField(max_length=15),
        max_length=20,
        help_text="List of ticker symbols (max 20)"
    )
    timeframe = serializers.ChoiceField(
        choices=['1d', '5d', '1w', '1mo', '3mo', '6mo', '1y', '2y', '5y'],
        help_text="Prediction timeframe"
    )


class ModelPerformanceSerializer(serializers.Serializer):
    """Serializer for model performance data."""

    model_id = serializers.CharField()
    model_type = serializers.CharField()
    timeframe = serializers.CharField()
    accuracy = serializers.FloatField()
    precision = serializers.FloatField()
    recall = serializers.FloatField()
    f1_score = serializers.FloatField()
    total_predictions = serializers.IntegerField()
    last_updated = serializers.DateTimeField()
```

8.3.3 Authentication Requirements

```
# backend/apps/authentication/backends.py

from django.contrib.auth.backends import ModelBackend
from django.contrib.auth import get_user_model
from rest_framework_simplejwt.authentication import JWTAuthentication
from rest_framework_simplejwt.exceptions import InvalidToken, TokenError

User = get_user_model()

class EmailAuthBackend(ModelBackend):
    """
    Custom authentication backend that allows login with email.
    """

    def authenticate(self, request, username=None, password=None, **kwargs):
        """
        Authenticate user with email or username.
        """
        if username is None:
            username = kwargs.get(User.USERNAME_FIELD)

        if username is None or password is None:
            return None

        try:
            # Try to fetch user by email first
            if '@' in username:
                user = User.objects.get(email__iexact=username)
            else:
                user = User.objects.get(username__iexact=username)

        except User.DoesNotExist:
            # Run the default password hasher once to reduce timing
            # difference between existing and non-existing user
            User().set_password(password)
            return None

        if user.check_password(password) and self.user_can_authenticate(user):
            return user

        return None


class CustomJWTAuthentication(JWTAuthentication):
    """
    Custom JWT authentication with additional validation.
    """

    def get_validated_token(self, raw_token):
        """
        Validate token with additional checks.
        """
        validated_token = super().get_validated_token(raw_token)

        # Add custom validation here
        user_id = validated_token.get('user_id')

        try:
            user = User.objects.get(id=user_id)

            # Check if user is active
            if not user.is_active:
                raise InvalidToken('User account is deactivated')

            # Add any other custom checks

        except User.DoesNotExist:
            raise InvalidToken('User not found')

        return validated_token
```

8.3.4 Error Codes

```
# backend/apps/common/exceptions.py
"""
Custom exception handlers and error codes.
"""

from rest_framework.views import exception_handler
from rest_framework.exceptions import APIException
from rest_framework import status


class ErrorCode:
    """Application error codes."""

    # Authentication errors (1000-1999)
    INVALID_CREDENTIALS = 1001
    TOKEN_EXPIRED = 1002
    TOKEN_INVALID = 1003
    USER_NOT_FOUND = 1004
    USER_INACTIVE = 1005

    # Validation errors (2000-2999)
    INVALID_TICKER = 2001
    INVALID_TIMEFRAME = 2002
    INVALID_REQUEST = 2003
    MISSING_REQUIRED_FIELD = 2004

    # Data errors (3000-3999)
    DATA_NOT_AVAILABLE = 3001
    DATA_FETCH_FAILED = 3002
    INSUFFICIENT_DATA = 3003

    # Model errors (4000-4999)
    MODEL_NOT_FOUND = 4001
    MODEL_TRAINING_FAILED = 4002
    PREDICTION_FAILED = 4003

    # Rate limiting errors (5000-5999)
    RATE_LIMIT_EXCEEDED = 5001
    QUOTA_EXCEEDED = 5002

    # System errors (9000-9999)
    INTERNAL_ERROR = 9001
    SERVICE_UNAVAILABLE = 9002
    MAINTENANCE_MODE = 9003


class StockVibeException(APIException):
    """Base exception for StockVibe application."""

    def __init__(self, message, code=None, status_code=status.HTTP_400_BAD_REQUEST):
        super().__init__(message)
        self.error_code = code
        self.status_code = status_code


class ValidationError(StockVibeException):
    """Validation error exception."""

    def __init__(self, message, field=None):
        super().__init__(
            message,
            code=ErrorCode.INVALID_REQUEST,
            status_code=status.HTTP_400_BAD_REQUEST
        )
        self.field = field
```

```
class DataNotAvailableError(StockVibeException):
    """Data not available exception."""

    def __init__(self, message):
        super().__init__(
            message,
            code=ErrorCode.DATA_NOT_AVAILABLE,
            status_code=status.HTTP_404_NOT_FOUND
        )


class ModelNotFoundError(StockVibeException):
    """Model not found exception."""

    def __init__(self, message):
        super().__init__(
            message,
            code=ErrorCode.MODEL_NOT_FOUND,
            status_code=status.HTTP_404_NOT_FOUND
        )


class PredictionError(StockVibeException):
    """Prediction error exception."""

    def __init__(self, message):
        super().__init__(
            message,
            code=ErrorCode.PREDICTION_FAILED,
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR
        )


class RateLimitError(StockVibeException):
    """Rate limit exceeded exception."""

    def __init__(self, message):
        super().__init__(
            message,
            code=ErrorCode.RATE_LIMIT_EXCEEDED,
            status_code=status.HTTP_429_TOO_MANY_REQUESTS
        )


def custom_exception_handler(exc, context):
    """
    Custom exception handler for consistent error responses.
    """
    response = exception_handler(exc, context)

    if response is not None:
        # Format error response
        error_data = {
            'error': True,
            'message': str(exc),
            'code': getattr(exc, 'error_code', ErrorCode.INTERNAL_ERROR)
        }

        # Add field information for validation errors
        if hasattr(exc, 'field') and exc.field:
            error_data['field'] = exc.field

        # Add detail for DRF exceptions
        if hasattr(exc, 'detail'):
            error_data['details'] = exc.detail

        response.data = error_data

    return response
```

8.4 Key Algorithms

8.4.1 Prediction Algorithm Implementation

```
# backend/ml_models/ensemble.py
"""
Ensemble model implementation for stock prediction.

This module implements the core prediction algorithm using
ensemble learning techniques.
"""

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
import logging

logger = logging.getLogger(__name__)


class EnsemblePredictionModel:
    """
    Ensemble model combining multiple algorithms for robust predictions.

    The ensemble uses soft voting to combine predictions from:
    - Random Forest Classifier
    - Gradient Boosting Classifier
    - Logistic Regression

    Time Complexity: O(n * m * k) where:
        n = number of samples
        m = number of features
        k = number of estimators

    Space Complexity: O(n * m + k * d) where:
        d = average depth of trees
    """

    def __init__(self, **kwargs):
        """
        Initialize ensemble model with optimized hyperparameters.

        Parameters:
        -----------
        random_state : int
            Random seed for reproducibility
        n_jobs : int
            Number of parallel jobs
        """
        self.random_state = kwargs.get('random_state', 42)
        self.n_jobs = kwargs.get('n_jobs', -1)
```

```
        # Initialize base models with optimized parameters
        self.rf_model = RandomForestClassifier(
            n_estimators=200,
            max_depth=15,
            min_samples_split=5,
            min_samples_leaf=2,
            max_features='sqrt',
            bootstrap=True,
            oob_score=True,
            random_state=self.random_state,
            n_jobs=self.n_jobs,
            class_weight='balanced'
        )

        self.gb_model = GradientBoostingClassifier(
            n_estimators=200,
            learning_rate=0.1,
            max_depth=5,
            min_samples_split=10,
            min_samples_leaf=5,
            subsample=0.8,
            max_features='sqrt',
            random_state=self.random_state,
            validation_fraction=0.2,
            n_iter_no_change=10
        )

        self.lr_model = LogisticRegression(
            penalty='elasticnet',
            solver='saga',
            l1_ratio=0.5,
            C=1.0,
            max_iter=1000,
            random_state=self.random_state,
            n_jobs=self.n_jobs
        )

        self.scaler = StandardScaler()
        self.is_fitted = False
        self.feature_names = None
        self.ensemble_weights = [0.4, 0.4, 0.2]  # RF, GB, LR weights

    def fit(self, X, y, feature_names=None):
        """
        Train the ensemble model.

        Parameters:
        -----------
        X : array-like of shape (n_samples, n_features)
            Training features
        y : array-like of shape (n_samples,)
            Training labels
        feature_names : list, optional
            Names of features

        Returns:
        --------
        self
```

```
        Fitted model
    """
    logger.info(f"Training ensemble model with {len(X)} samples")

    # Store feature names
    self.feature_names = feature_names

    # Scale features
    X_scaled = self.scaler.fit_transform(X)

    # Train individual models
    logger.info("Training Random Forest...")
    self.rf_model.fit(X, y)  # RF doesn't need scaling

    logger.info("Training Gradient Boosting...")
    self.gb_model.fit(X, y)  # GB doesn't need scaling

    logger.info("Training Logistic Regression...")
    self.lr_model.fit(X_scaled, y)  # LR needs scaling

    self.is_fitted = True

    # Calculate individual model performance
    self._evaluate_models(X, X_scaled, y)

    return self

def predict(self, X):
    """
    Generate predictions using ensemble voting.

    Parameters:
    -----------
    X : array-like of shape (n_samples, n_features)
        Features to predict

    Returns:
    --------
    array-like of shape (n_samples,)
        Predicted labels
    """
    if not self.is_fitted:
        raise ValueError("Model must be fitted before prediction")

    # Get probability predictions from each model
    proba_rf = self.rf_model.predict_proba(X)
    proba_gb = self.gb_model.predict_proba(X)

    X_scaled = self.scaler.transform(X)
    proba_lr = self.lr_model.predict_proba(X_scaled)

    # Weighted average of probabilities
    weighted_proba = (
        self.ensemble_weights[0] * proba_rf +
        self.ensemble_weights[1] * proba_gb +
        self.ensemble_weights[2] * proba_lr
    )
```

```
        # Return class with highest probability
        return np.argmax(weighted_proba, axis=1)

    def predict_proba(self, X):
        """
        Generate probability predictions.

        Parameters:
        -----------
        X : array-like of shape (n_samples, n_features)
            Features to predict

        Returns:
        --------
        array-like of shape (n_samples, n_classes)
            Probability predictions
        """
        if not self.is_fitted:
            raise ValueError("Model must be fitted before prediction")

        # Get probability predictions from each model
        proba_rf = self.rf_model.predict_proba(X)
        proba_gb = self.gb_model.predict_proba(X)

        X_scaled = self.scaler.transform(X)
        proba_lr = self.lr_model.predict_proba(X_scaled)

        # Weighted average of probabilities
        weighted_proba = (
            self.ensemble_weights[0] * proba_rf +
            self.ensemble_weights[1] * proba_gb +
            self.ensemble_weights[2] * proba_lr
        )

        return weighted_proba

    def get_feature_importance(self):
        """
        Get feature importance from the ensemble.

        Returns:
        --------
        pd.DataFrame
            Feature importance scores
        """
        if not self.is_fitted:
            raise ValueError("Model must be fitted first")

        # Get feature importance from tree-based models
        rf_importance = self.rf_model.feature_importances_
        gb_importance = self.gb_model.feature_importances_

        # Get absolute coefficients from logistic regression
        lr_importance = np.abs(self.lr_model.coef_[0])
        lr_importance = lr_importance / lr_importance.sum()  # Normalize

        # Weighted average of importances
        ensemble_importance = (
```

```
            self.ensemble_weights[0] * rf_importance +
            self.ensemble_weights[1] * gb_importance +
            self.ensemble_weights[2] * lr_importance
        )

        # Create DataFrame
        importance_df = pd.DataFrame({
            'feature': self.feature_names or [f'feature_{i}' for i in
range(len(ensemble_importance))],
            'importance': ensemble_importance,
            'rf_importance': rf_importance,
            'gb_importance': gb_importance,
            'lr_importance': lr_importance
        })

        return importance_df.sort_values('importance', ascending=False)

    def _evaluate_models(self, X, X_scaled, y):
        """
        Evaluate individual model performance.

        Parameters:
        -----------
        X : array-like
            Unscaled features
        X_scaled : array-like
            Scaled features
        y : array-like
            Labels
        """
        # Cross-validation scores
        rf_scores = cross_val_score(self.rf_model, X, y, cv=3, scoring='accuracy')
        gb_scores = cross_val_score(self.gb_model, X, y, cv=3, scoring='accuracy')
        lr_scores = cross_val_score(self.lr_model, X_scaled, y, cv=3,
scoring='accuracy')

        logger.info(f"Random Forest CV Score: {rf_scores.mean():.4f} (+/-
{rf_scores.std():.4f})")
        logger.info(f"Gradient Boosting CV Score: {gb_scores.mean():.4f} (+/-
{gb_scores.std():.4f})")
        logger.info(f"Logistic Regression CV Score: {lr_scores.mean():.4f} (+/-
{lr_scores.std():.4f})")

        # Adjust weights based on performance
        total_score = rf_scores.mean() + gb_scores.mean() + lr_scores.mean()
        self.ensemble_weights = [
            rf_scores.mean() / total_score,
            gb_scores.mean() / total_score,
            lr_scores.mean() / total_score
        ]

        logger.info(f"Updated ensemble weights: RF={self.ensemble_weights[0]:.3f}, "
                    f"GB={self.ensemble_weights[1]:.3f},
LR={self.ensemble_weights[2]:.3f}")
```

8.4.2 Feature Engineering Algorithm

```
# backend/apps/predictions/services/feature_service.py

import numpy as np
import pandas as pd
from typing import Dict, List, Optional
import logging

logger = logging.getLogger(__name__)


class FeatureEngineer:
    """
    Feature engineering service for creating prediction features.

    Implements optimized algorithms for:
    - Technical indicator calculation
    - Pattern recognition
    - Statistical transformations
    """

    def engineer_features(self, data: pd.DataFrame, timeframe: str) -> pd.DataFrame:
        """
        Engineer comprehensive features from market data.

        Time Complexity: O(n * m) where n = data points, m = features
        Space Complexity: O(n * m)

        Parameters:
        -----------
        data : pd.DataFrame
            Raw OHLCV data
        timeframe : str
            Timeframe for feature engineering

        Returns:
        --------
        pd.DataFrame
            Data with engineered features
        """
        logger.info(f"Engineering features for {len(data)} data points")

        # Make a copy to avoid modifying original
        df = data.copy()

        # Basic price features
        df = self._add_price_features(df)

        # Volume features
        df = self._add_volume_features(df)

        # Technical indicators
        df = self._add_technical_indicators(df, timeframe)

        # Pattern features
        df = self._add_pattern_features(df)

        # Statistical features
        df = self._add_statistical_features(df)
```

```
        # Clean and validate
        df = self._clean_features(df)

        logger.info(f"Generated {len(df.columns)} features")

        return df

    def _add_price_features(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Add price-based features.

        Algorithms:
        - Simple returns: O(n)
        - Log returns: O(n)
        - Price ratios: O(n)
        """
        # Returns
        df['return_1d'] = df['Close'].pct_change(1)
        df['return_5d'] = df['Close'].pct_change(5)
        df['return_20d'] = df['Close'].pct_change(20)

        # Log returns for better statistical properties
        df['log_return'] = np.log(df['Close'] / df['Close'].shift(1))

        # Price ratios
        df['high_low_ratio'] = df['High'] / df['Low']
        df['close_open_ratio'] = df['Close'] / df['Open']

        # Price position in daily range
        df['price_position'] = (df['Close'] - df['Low']) / (df['High'] - df['Low'])

        # Gap detection
        df['gap'] = (df['Open'] - df['Close'].shift(1)) / df['Close'].shift(1)

        return df

    def _add_volume_features(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Add volume-based features.

        Algorithms:
        - Volume moving averages: O(n)
        - On-Balance Volume (OBV): O(n)
        - Volume-Price Trend (VPT): O(n)
        """
        # Volume moving averages
        df['volume_ma_10'] = df['Volume'].rolling(window=10).mean()
        df['volume_ma_20'] = df['Volume'].rolling(window=20).mean()

        # Volume ratio
        df['volume_ratio'] = df['Volume'] / df['volume_ma_20']

        # On-Balance Volume (OBV)
        df['obv'] = (np.sign(df['Close'].diff()) * df['Volume']).fillna(0).cumsum()

        # Volume-Price Trend (VPT)
        df['vpt'] = (df['Close'].pct_change() * df['Volume']).cumsum()

        # Money Flow Index components
        typical_price = (df['High'] + df['Low'] + df['Close']) / 3
        money_flow = typical_price * df['Volume']
```

```
        positive_flow = money_flow.where(typical_price > typical_price.shift(1), 0)
        negative_flow = money_flow.where(typical_price < typical_price.shift(1), 0)

        money_ratio = positive_flow.rolling(14).sum() / negative_flow.rolling(14).sum()
        df['mfi'] = 100 - (100 / (1 + money_ratio))

        return df

    def _add_technical_indicators(self, df: pd.DataFrame, timeframe: str) -> pd.DataFrame:
        """
        Add technical indicators optimized for timeframe.

        Implements:
        - Moving averages (SMA, EMA): O(n)
        - RSI: O(n)
        - MACD: O(n)
        - Bollinger Bands: O(n)
        - Stochastic Oscillator: O(n * w) where w = window size
        """
        # Adjust periods based on timeframe
        periods = self._get_indicator_periods(timeframe)

        # Moving Averages
        for period in periods['ma']:
            df[f'sma_{period}'] = df['Close'].rolling(window=period).mean()
            df[f'ema_{period}'] = df['Close'].ewm(span=period, adjust=False).mean()

        # RSI (Relative Strength Index)
        df['rsi'] = self._calculate_rsi(df['Close'], periods['rsi'])

        # MACD
        exp1 = df['Close'].ewm(span=periods['macd_fast'], adjust=False).mean()
        exp2 = df['Close'].ewm(span=periods['macd_slow'], adjust=False).mean()
        df['macd'] = exp1 - exp2
        df['macd_signal'] = df['macd'].ewm(span=periods['macd_signal'],
adjust=False).mean()
        df['macd_histogram'] = df['macd'] - df['macd_signal']

        # Bollinger Bands
        bb_period = periods['bollinger']
        bb_std = 2
        df['bb_middle'] = df['Close'].rolling(window=bb_period).mean()
        bb_std_dev = df['Close'].rolling(window=bb_period).std()
        df['bb_upper'] = df['bb_middle'] + (bb_std_dev * bb_std)
        df['bb_lower'] = df['bb_middle'] - (bb_std_dev * bb_std)
        df['bb_width'] = (df['bb_upper'] - df['bb_lower']) / df['bb_middle']
        df['bb_position'] = (df['Close'] - df['bb_lower']) / (df['bb_upper'] -
df['bb_lower'])

        # Stochastic Oscillator
        period = periods['stochastic']
        lowest_low = df['Low'].rolling(window=period).min()
        highest_high = df['High'].rolling(window=period).max()
        df['stoch_k'] = 100 * ((df['Close'] - lowest_low) / (highest_high - lowest_low))
        df['stoch_d'] = df['stoch_k'].rolling(window=3).mean()

        # Average True Range (ATR)
        high_low = df['High'] - df['Low']
        high_close = np.abs(df['High'] - df['Close'].shift())
        low_close = np.abs(df['Low'] - df['Close'].shift())
        true_range = pd.concat([high_low, high_close, low_close], axis=1).max(axis=1)
```

```
        df['atr'] = true_range.rolling(window=periods['atr']).mean()

        # Commodity Channel Index (CCI)
        typical_price = (df['High'] + df['Low'] + df['Close']) / 3
        sma_tp = typical_price.rolling(window=periods['cci']).mean()
        mad = typical_price.rolling(window=periods['cci']).apply(
            lambda x: np.mean(np.abs(x - x.mean()))
        )
        df['cci'] = (typical_price - sma_tp) / (0.015 * mad)

        return df

    def _add_pattern_features(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Add pattern recognition features.

        Algorithms:
        - Candlestick patterns: O(n)
        - Trend detection: O(n)
        - Support/Resistance: O(n * w)
        """
        # Candlestick patterns
        body = df['Close'] - df['Open']
        body_size = np.abs(body)
        shadow_range = df['High'] - df['Low']

        # Doji pattern (small body)
        df['doji'] = (body_size <= shadow_range * 0.1).astype(int)

        # Hammer pattern
        df['hammer'] = (
            (body_size <= shadow_range * 0.3) &
            (df['Low'] < df['Open'] - body_size * 2) &
            (df['Close'] > df['Open'])
        ).astype(int)

        # Shooting star pattern
        df['shooting_star'] = (
            (body_size <= shadow_range * 0.3) &
            (df['High'] > df['Close'] + body_size * 2) &
            (df['Close'] < df['Open'])
        ).astype(int)

        # Trend detection
        df['higher_high'] = (
            (df['High'] > df['High'].shift(1)) &
            (df['High'].shift(1) > df['High'].shift(2))
        ).astype(int)

        df['lower_low'] = (
            (df['Low'] < df['Low'].shift(1)) &
            (df['Low'].shift(1) < df['Low'].shift(2))
        ).astype(int)

        # Moving average crossovers
        df['golden_cross'] = (
            (df.get('sma_50', df['Close']) > df.get('sma_200', df['Close'])) &
            (df.get('sma_50', df['Close']).shift(1) <= df.get('sma_200',
df['Close']).shift(1))
        ).astype(int)

        # Support and Resistance levels
```

```
        window = 20
        df['resistance'] = df['High'].rolling(window=window).max()
        df['support'] = df['Low'].rolling(window=window).min()
        df['price_to_resistance'] = df['Close'] / df['resistance']
        df['price_to_support'] = df['Close'] / df['support']

        return df

    def _add_statistical_features(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Add statistical features.

        Algorithms:
        - Rolling statistics: O(n * w)
        - Z-score: O(n)
        - Autocorrelation: O(n)
        """
        # Rolling statistics
        for window in [5, 10, 20]:
            df[f'rolling_mean_{window}'] = df['Close'].rolling(window).mean()
            df[f'rolling_std_{window}'] = df['Close'].rolling(window).std()
            df[f'rolling_min_{window}'] = df['Close'].rolling(window).min()
            df[f'rolling_max_{window}'] = df['Close'].rolling(window).max()
            df[f'rolling_median_{window}'] = df['Close'].rolling(window).median()

        # Z-score
        df['z_score'] = (df['Close'] - df['rolling_mean_20']) / df['rolling_std_20']

        # Volatility
        df['volatility_10'] = df['return_1d'].rolling(window=10).std()
        df['volatility_20'] = df['return_1d'].rolling(window=20).std()

        # Skewness and Kurtosis
        df['skewness_20'] = df['return_1d'].rolling(window=20).skew()
        df['kurtosis_20'] = df['return_1d'].rolling(window=20).kurt()

        # Autocorrelation
        df['autocorr_1'] = df['return_1d'].rolling(window=20).apply(
            lambda x: x.autocorr(lag=1) if len(x) > 1 else 0
        )

        return df

    def _get_indicator_periods(self, timeframe: str) -> Dict[str, any]:
        """
        Get optimized indicator periods for timeframe.
        """
        # Default periods
        periods = {
            'ma': [5, 10, 20, 50, 200],
            'rsi': 14,
            'macd_fast': 12,
            'macd_slow': 26,
            'macd_signal': 9,
            'bollinger': 20,
            'stochastic': 14,
            'atr': 14,
            'cci': 20
        }

        # Adjust for intraday timeframes
        if timeframe in ['1d', '5d']:
```

```
            periods['ma'] = [5, 10, 20, 50]
            periods['rsi'] = 9
            periods['bollinger'] = 10

        # Adjust for long-term timeframes
        elif timeframe in ['1y', '2y', '5y']:
            periods['ma'] = [10, 20, 50, 100, 200]
            periods['rsi'] = 21
            periods['bollinger'] = 50

        return periods

    def _calculate_rsi(self, series: pd.Series, period: int = 14) -> pd.Series:
        """
        Calculate RSI using Wilder's smoothing method.

        Time Complexity: O(n)
        Space Complexity: O(n)
        """
        delta = series.diff()
        gain = (delta.where(delta > 0, 0)).rolling(window=period).mean()
        loss = (-delta.where(delta < 0, 0)).rolling(window=period).mean()

        # Avoid division by zero
        rs = gain / loss.replace(0, 1e-10)

        return 100 - (100 / (1 + rs))

    def _clean_features(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Clean and validate features.

        Operations:
        - Remove infinite values: O(n * m)
        - Fill NaN values: O(n * m)
        - Clip outliers: O(n * m)
        """
        # Replace infinite values
        df = df.replace([np.inf, -np.inf], np.nan)

        # Forward fill then backward fill NaN values
        df = df.fillna(method='ffill').fillna(method='bfill')

        # Fill remaining NaN with 0
        df = df.fillna(0)

        # Clip extreme outliers (optional, based on domain knowledge)
        for col in df.select_dtypes(include=[np.number]).columns:
            if col in ['Close', 'Open', 'High', 'Low']:
                continue  # Don't clip price columns

            # Clip at 5 standard deviations
            mean = df[col].mean()
            std = df[col].std()
            if std > 0:
                df[col] = df[col].clip(mean - 5*std, mean + 5*std)

        return df
```

8.4.3 Optimization Techniques

```
# backend/apps/common/optimizations.py
"""
Performance optimization utilities for the application.

Implements various optimization techniques including:
- Caching strategies
- Query optimization
- Parallel processing
- Memory management
"""

import functools
import hashlib
import pickle
from typing import Any, Callable, Dict, List, Optional
from concurrent.futures import ThreadPoolExecutor, as_completed
import numpy as np
import pandas as pd
from django.core.cache import cache
from django.db import connection
import logging

logger = logging.getLogger(__name__)


class CacheOptimizer:
    """
    Advanced caching strategies for performance optimization.

    Implements:
    - Multi-level caching
    - Intelligent cache invalidation
    - Cache warming
    """

    @staticmethod
    def memoize(ttl: int = 3600):
        """
        Decorator for function memoization with TTL.

        Time Complexity: O(1) for cache hit, O(n) for cache miss
        Space Complexity: O(k) where k = number of cached results
        """
        def decorator(func: Callable) -> Callable:
            @functools.wraps(func)
            def wrapper(*args, **kwargs):
                # Generate cache key
                cache_key = CacheOptimizer._generate_cache_key(
                    func.__name__, args, kwargs
```

```
                )

                # Check cache
                result = cache.get(cache_key)
                if result is not None:
                    logger.debug(f"Cache hit: {cache_key}")
                    return result

                # Execute function
                result = func(*args, **kwargs)

                # Cache result
                cache.set(cache_key, result, ttl)
                logger.debug(f"Cached: {cache_key} for {ttl}s")

                return result

            return wrapper
        return decorator

    @staticmethod
    def _generate_cache_key(func_name: str, args: tuple, kwargs: dict) ->
str:
        """
        Generate deterministic cache key.

        Time Complexity: O(n) where n = size of arguments
        """
        key_data = {
            'func': func_name,
            'args': args,
            'kwargs': kwargs
        }

        # Serialize and hash
        key_str = pickle.dumps(key_data)
        key_hash = hashlib.md5(key_str).hexdigest()

        return f"func_cache:{func_name}:{key_hash}"

    @staticmethod
    def warm_cache(keys: List[str], data_func: Callable, ttl: int = 3600):
        """
        Pre-populate cache with frequently accessed data.

        Time Complexity: O(n * m) where n = keys, m = data generation time
        """
        logger.info(f"Warming cache for {len(keys)} keys")

        with ThreadPoolExecutor(max_workers=5) as executor:
            futures = []
```

```
            for key in keys:
                future = executor.submit(
                    CacheOptimizer._warm_single_key, key, data_func, ttl
                )
                futures.append(future)

            # Wait for completion
            for future in as_completed(futures):
                try:
                    future.result()
                except Exception as e:
                    logger.error(f"Cache warming failed: {str(e)}")

        logger.info("Cache warming completed")

    @staticmethod
    def _warm_single_key(key: str, data_func: Callable, ttl: int):
        """Warm a single cache key."""
        try:
            data = data_func(key)
            cache.set(key, data, ttl)
            logger.debug(f"Warmed cache key: {key}")
        except Exception as e:
            logger.error(f"Failed to warm {key}: {str(e)}")


class QueryOptimizer:
    """
    Database query optimization utilities.

    Implements:
    - Query batching
    - Prefetch optimization
    - Query result caching
    """

    @staticmethod
    def batch_query(model, ids: List[int], batch_size: int = 1000) -> List:
        """
        Execute queries in batches to avoid memory issues.

        Time Complexity: O(n) where n = number of records
        Space Complexity: O(batch_size)
        """
        results = []

        for i in range(0, len(ids), batch_size):
            batch_ids = ids[i:i + batch_size]
            batch_results = model.objects.filter(id__in=batch_ids)
            results.extend(batch_results)

        return results
```

```
    @staticmethod
    def optimize_queryset(queryset):
        """
        Optimize queryset with prefetch and select_related.

        Reduces database queries from O(n) to O(1) for related objects.
        """
        # Analyze queryset for optimization opportunities
        model = queryset.model

        # Get foreign key fields
        fk_fields = [
            field.name for field in model._meta.fields
            if field.many_to_one
        ]

        # Get many-to-many fields
        m2m_fields = [
            field.name for field in model._meta.many_to_many
        ]

        # Apply select_related for foreign keys
        if fk_fields:
            queryset = queryset.select_related(*fk_fields)

        # Apply prefetch_related for many-to-many
        if m2m_fields:
            queryset = queryset.prefetch_related(*m2m_fields)

        return queryset

    @staticmethod
    def analyze_query_performance(query: str) -> Dict[str, Any]:
        """
        Analyze SQL query performance.

        Returns execution plan and performance metrics.
        """
        with connection.cursor() as cursor:
            # Get execution plan
            cursor.execute(f"EXPLAIN ANALYZE {query}")
            plan = cursor.fetchall()

            # Parse execution time
            total_time = 0
            for row in plan:
                if 'actual time=' in str(row):
                    # Extract time from plan
                    import re
                    match = re.search(r'actual time=([\d.]+)', str(row))
                    if match:
```

```
                        total_time += float(match.group(1))

            return {
                'query': query,
                'execution_time_ms': total_time,
                'plan': plan
            }


class ParallelProcessor:
    """
    Parallel processing utilities for CPU-intensive tasks.

    Implements:
    - Thread pool management
    - Task distribution
    - Result aggregation
    """

    def __init__(self, max_workers: Optional[int] = None):
        """Initialize with optimal worker count."""
        import multiprocessing
        self.max_workers = max_workers or multiprocessing.cpu_count()

    def process_parallel(
        self,
        func: Callable,
        items: List[Any],
        chunk_size: Optional[int] = None
    ) -> List[Any]:
        """
        Process items in parallel.

        Time Complexity: O(n/p) where n = items, p = workers
        Space Complexity: O(n)
        """
        if not items:
            return []

        # Determine optimal chunk size
        if chunk_size is None:
            chunk_size = max(1, len(items) // (self.max_workers * 4))

        # Create chunks
        chunks = [
            items[i:i + chunk_size]
            for i in range(0, len(items), chunk_size)
        ]

        logger.info(f"Processing {len(items)} items in {len(chunks)} chunks")

        results = []
```

```
        with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
            # Submit tasks
            futures = {
                executor.submit(self._process_chunk, func, chunk): chunk
                for chunk in chunks
            }

            # Collect results
            for future in as_completed(futures):
                try:
                    chunk_results = future.result()
                    results.extend(chunk_results)
                except Exception as e:
                    logger.error(f"Chunk processing failed: {str(e)}")

        return results

    def _process_chunk(self, func: Callable, chunk: List[Any]) -> List[Any]:
        """Process a single chunk of items."""
        return [func(item) for item in chunk]

    def map_reduce(
        self,
        map_func: Callable,
        reduce_func: Callable,
        items: List[Any]
    ) -> Any:
        """
        Map-reduce pattern implementation.

        Time Complexity: O(n/p + r) where r = reduction time
        """
        # Map phase
        mapped_results = self.process_parallel(map_func, items)

        # Reduce phase
        if not mapped_results:
            return None

        result = mapped_results[0]
        for item in mapped_results[1:]:
            result = reduce_func(result, item)

        return result


class MemoryOptimizer:
    """
    Memory optimization utilities.

    Implements:
    - Dataframe memory reduction
```

```
    - Chunked processing
    - Memory monitoring
    """

    @staticmethod
    def optimize_dataframe(df: pd.DataFrame) -> pd.DataFrame:
        """
        Reduce DataFrame memory usage.

        Can reduce memory usage by 50-80% for large datasets.
        """
        initial_memory = df.memory_usage(deep=True).sum() / 1024**2

        for col in df.columns:
            col_type = df[col].dtype

            if col_type != 'object':
                # Optimize numeric types
                c_min = df[col].min()
                c_max = df[col].max()

                if str(col_type)[:3] == 'int':
                    # Optimize integers
                    if c_min > np.iinfo(np.int8).min and c_max <
np.iinfo(np.int8).max:
                        df[col] = df[col].astype(np.int8)
                    elif c_min > np.iinfo(np.int16).min and c_max <
np.iinfo(np.int16).max:
                        df[col] = df[col].astype(np.int16)
                    elif c_min > np.iinfo(np.int32).min and c_max <
np.iinfo(np.int32).max:
                        df[col] = df[col].astype(np.int32)
                else:
                    # Optimize floats
                    if c_min > np.finfo(np.float16).min and c_max <
np.finfo(np.float16).max:
                        df[col] = df[col].astype(np.float16)
                    elif c_min > np.finfo(np.float32).min and c_max <
np.finfo(np.float32).max:
                        df[col] = df[col].astype(np.float32)
            else:
                # Convert object to category if applicable
                num_unique_values = len(df[col].unique())
                num_total_values = len(df[col])

                if num_unique_values / num_total_values < 0.5:
                    df[col] = df[col].astype('category')

        final_memory = df.memory_usage(deep=True).sum() / 1024**2
        reduction = (initial_memory - final_memory) / initial_memory * 100

        logger.info(f"Memory reduced by {reduction:.1f}% "
```

```
                f"({initial_memory:.1f} MB -> {final_memory:.1f} MB)")

        return df

    @staticmethod
    def process_in_chunks(
        df: pd.DataFrame,
        func: Callable,
        chunk_size: int = 10000
    ) -> pd.DataFrame:
        """
        Process large DataFrame in chunks to avoid memory issues.

        Time Complexity: O(n)
        Space Complexity: O(chunk_size)
        """
        chunks = []

        for start in range(0, len(df), chunk_size):
            end = min(start + chunk_size, len(df))
            chunk = df.iloc[start:end]

            # Process chunk
            processed_chunk = func(chunk)
            chunks.append(processed_chunk)

            logger.debug(f"Processed chunk {start}-{end} of {len(df)}")

        # Combine chunks
        result = pd.concat(chunks, ignore_index=True)

        return result
```

8.5 Complete Source Code

The complete source code for the StockVibePredictor project is organized across multiple files and modules as documented above. The full implementation includes :

8.5.1 Full Implementation Details

The project consists of approximately 50,000+ lines of code distributed across :

✦ **Backend (Django)**: 25,000+ lines
- ✧ Core application logic
- ✧API endpoints
- ✧ Machine learning models
- ✧ Data processing pipelines
- ✧ Database models and migrations

✦ **Frontend (React)**: 15,000+ lines
- ✧ Component implementations
- ✧ State management
- ✧ API integration
- ✧ User interface logic
- ✧ Styling and themes

✦ **Machine Learning**: 8,000+ lines
- ✧ Model implementations
- ✧ Training scripts
- ✧ Feature engineering
- ✧ Evaluation metrics
- ✧ Optimization algorithms

✦ **Supporting Scripts**: 2,000+ lines
- ✧ Deployment scripts
- ✧ Testing utilities
- ✧ Data management
- ✧ Documentation generation

8.5.2 Comments and Annotations

All code follows strict documentation standards :

```
"""
Module-level documentation explaining purpose and usage.
"""

class ExampleClass:
    """
    Class-level documentation with:
    - Purpose description
    - Key responsibilities
    - Usage examples
    - Performance characteristics
    """

    def example_method(self, param1: str, param2: int) -> Dict[str, Any]:
        """
        Method documentation including:

        Parameters:
        -----------
        param1 : str
            Description of parameter
        param2 : int
            Description of parameter

        Returns:
        --------
        Dict[str, Any]
            Description of return value

        Raises:
        -------
        ValueError
            When invalid input is provided

        Examples:
        ---------
        >>> obj.example_method("test", 42)
        {'result': 'success'}

        # Inline comments explaining complex logic
        # Step 1: Validate inputs
        # Step 2: Process data
        # Step 3: Return results
```

8.5.3 Best Practices Followed :

Code Quality Standards :

✦ **PEP 8 Compliance** for Python code.
✦ **ESLint Standards** for JavaScript/React.
✦ **Type Hints** for all function signatures.
✦ **Comprehensive Error Handling.**
✦ **Logging at appropriate levels.**
✦ **Security best practices** (input validation, SQL injection prevention).
✦ **Performance optimization** (caching, query optimization).
✦ **Test Coverage** > 80%.

Design Patterns Implemented :

✦ **Repository Pattern** for data access.
✦ **Service Layer Pattern** for business logic.
✦ **Factory Pattern** for model creation.
✦ **Observer Pattern** for real-time updates.
✦ **Strategy Pattern** for algorithm selection.
✦ **Decorator Pattern** for caching and logging.
✦ **Singleton Pattern** for configuration management.

8.5.4 Code Standards

Naming Conventions :

```
# Python
class PascalCaseClassName:
    def snake_case_method_name(self):
        local_variable_name = "value"
        CONSTANT_NAME = 42

# JavaScript/React
const ComponentName = () => {
    const variableName = "value";
    const CONSTANT_NAME = 42;

    const functionName = () => {
        // Implementation
    };
};
```

File Organization :

- ✦ Each file has a single responsibility.
- ✦ Maximum file length: 500 lines.
- ✦ Related functionality grouped in modules.
- ✦ Clear separation of concerns.
- ✦ Consistent directory structure.

Version Control Practices :

```
# Commit message format
[TYPE] Brief description

Detailed explanation of changes
- Bullet point 1
- Bullet point 2

Fixes #123

# Types: feat, fix, docs, style, refactor, test, chore
```

CHAPTER 9: RESULTS AND ANALYSIS

9.1 Model Performance Metrics

✦ Accuracy analysis
✦ Precision and recall
✦ F1 scores
✦ ROC curves

9.2 Prediction Accuracy Analysis

✦ Timeframe-wise performance
✦ Stock category performance
✦ Market condition analysis
✦ Error analysis

9.3 Backtesting Results

✦ Historical performance
✦ Trading strategy results
✦ Risk metrics
✦ Portfolio performance

9.4 Comparative Analysis

✦ Comparison with baseline models
✦ Benchmark comparisons
✦ Literature comparison
✦ Industry standard comparison

9.5 User Testing Results

✦ Usability testing
✦ Performance feedback
✦ Feature requests
✦ User satisfaction metrics

CHAPTER 10: FUTURE SCOPE OF IMPROVEMENTS

10.1 Enhanced Data Integration
✦ Alternative data sources
✦ Real-time streaming
✦ News sentiment analysis
✦ Social media integration

10.2 Advanced Machine Learning Models
✦ Deep learning implementation
✦ LSTM networks
✦ Transformer models
✦ Reinforcement learning

10.3 Real-time Trading Integration
✦ Broker API integration
✦ Automated trading
✦ Risk management
✦ Portfolio optimization

10.4 Mobile Application Development
✦ Native mobile apps
✦ Cross-platform development
✦ Push notifications
✦ Offline capabilities

10.5 Scalability Enhancements
✦ Microservices architecture
✦ Cloud deployment
✦ Distributed computing
✦ Performance optimization

CHAPTER 11: CONCLUSION

11.1 Summary of Work

The StockVibePredictor project represents a comprehensive effort to democratize stock market prediction through the application of machine learning and modern web technologies. Over the course of this project, we have successfully developed, implemented, and tested a full-stack application that provides sophisticated market analysis capabilities to users regardless of their technical expertise or financial resources.

Our system processes real-time and historical market data through a sophisticated pipeline that includes data validation, feature engineering, and ensemble machine learning models. The implementation of nine different timeframes, from intraday to 5-year predictions, provides users with a holistic view of market movements suitable for various investment strategies.

The technical achievements of this project include the successful integration of multiple data sources, implementation of 29+ technical indicators, development of ensemble learning models achieving average accuracy of 57.5%, and creation of a responsive, user-friendly web interface. The system's ability to handle 100+ concurrent users with sub-2 second response times demonstrates its production readiness.

11.2 Achievements

✦ **Successful Multi-Timeframe Prediction System**: Implemented comprehensive prediction capabilities across 9 timeframes.
✦ **High-Performance Architecture**: Achieved 85% cache hit rate and sub-2 second response times.
✦ **Extensive Market Coverage**: Integrated 100+ stocks across US, Indian, and cryptocurrency markets.
✦ **Robust Machine Learning Pipeline**: Developed ensemble models with 55-76% accuracy range.
✦ **Production-Ready System**: Built scalable architecture supporting real-world deployment.
✦ **Comprehensive Documentation**: Created detailed technical and user documentation.
✦ **Educational Value**: Developed system that educates users about technical analysis.

11.3 Limitations

While the project has achieved its primary objectives, certain limitations should be acknowledged :

✦ **Market Efficiency Constraints**: Cannot overcome fundamental limits of market prediction.
✦ **Data Dependency**: Reliant on third-party APIs for data availability.
✦ **Limited Fundamental Analysis**: Focus on technical analysis without incorporating company fundamentals.
✦ **Geographic Limitations**: Primary focus on US and Indian markets.
✦ **Real Trading Integration**: Currently limited to paper trading without actual broker integration.

11.4 Final Remarks

The StockVibePredictor project successfully demonstrates the practical application of machine learning in financial markets while maintaining accessibility for retail investors. The system's modular architecture ensures maintainability and facilitates future enhancements. The comprehensive documentation and clean code structure make it an excellent foundation for further research and development.

This project contributes to the democratization of financial technology by providing sophisticated analytical tools that were previously available only to institutional investors. As we continue to enhance and expand the system, we remain committed to our vision of empowering individual investors with data-driven insights for better investment decisions.

The experience gained through this project has provided invaluable insights into the intersection of machine learning, financial markets, and software engineering. The challenges overcome and solutions developed during this project will serve as a strong foundation for future endeavors in financial technology.

REFERENCES

✦ Murphy, J. J. (1999). Technical Analysis of the Financial Markets. New York Institute of Finance.

✦ Tsay, R. S. (2010). Analysis of Financial Time Series (3rd ed.). John Wiley & Sons.

✦ López de Prado, M. (2018). Advances in Financial Machine Learning. John Wiley & Sons.

✦ Hull, J. C. (2018). Options, Futures, and Other Derivatives (10th ed.). Pearson.

✦ Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. European Journal of Operational Research, 270(2), 654-669.

✦ Khaidem, L., Saha, S., & Dey, S. R. (2016). Predicting the direction of stock market prices using random forest. Applied Mathematical Finance, 23(3), 1-20.

✦ Rather, A. M., & Sastry, V. N. (2020). Stock market prediction and Portfolio selection models: A survey. OPSEARCH, 57(3), 558-579.

✦ Brock, W., Lakonishok, J., & LeBaron, B. (1992). Simple technical trading rules and the stochastic properties of stock returns. The Journal of Finance, 47(5), 1731-1764.

✦ White, H. (1988). Economic prediction using neural networks: The case of IBM daily stock returns. IEEE International Conference on Neural Networks, 451-458.

✦ Lo, A. W., Mamaysky, H., & Wang, J. (2000). Foundations of technical analysis: Computational algorithms, statistical inference, and empirical implementation. The Journal of Finance, 55(4), 1705-1765.

✦ Scikit-learn Documentation. (2024). Retrieved from https://scikit-learn.org/

✦ Django Documentation. (2024). Retrieved from https://docs.djangoproject.com/

✦ React Documentation. (2024). Retrieved from https://react.dev/

✦ Yahoo Finance API Documentation. (2024). Retrieved from https://pypi.org/project/yfinance/

✦ Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. The Journal of Finance, 25(2), 383-417.

APPENDICES

Appendix A: Installation Guide

System Requirements :

- ✦ Python 3.8 or higher
- ✦ Node.js 16.0 or higher
- ✦ PostgreSQL 12.0 or higher
- ✦ Redis 6.0 or higher
- ✦ 8GB RAM minimum
- ✦ 20GB free disk space

Installation Steps :

1. Clone the Repository :

```
git clone https://github.com/DibVibe/StockVibePredictor.git
```

2. Backend Setup :

```
cd Backend
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
```

3. Frontend Setup :

```
cd frontend
npm install
npm run build
```

4. Environment Configuration :

Create .env file in backend directory :

```
SECRET_KEY=your-secret-key
DEBUG=False
DATABASE_URL=postgresql://user:password@localhost/stockvibe
REDIS_URL=redis://localhost:6379
```

5. Start Services

```
# Terminal 1 - Backend
python manage.py runserver

# Terminal 2 - Frontend
npm start

# Terminal 3 - Celery Worker
celery -A stockvibe worker -l info

# Terminal 4 - Celery Beat
celery -A stockvibe beat -l info
```

Appendix B: User Manual

Getting Started :

✦ Register for an account
✦ Verify your email
✦ Login to the dashboard
✦ Add stocks to your watchlist
✦ Generate predictions

Features Guide :

✦ Dashboard: Overview of market and predictions
✦ Predictions: Generate and view predictions
✦ Portfolio: Track your investments
✦ Analysis: Technical analysis tools
✦ Settings: Customize preferences

Appendix C: API Documentation

Authentication :

```
POST /api/auth/login/
Content-Type: application/json

{
  "username": "user@example.com",
  "password": "password123"
}
```

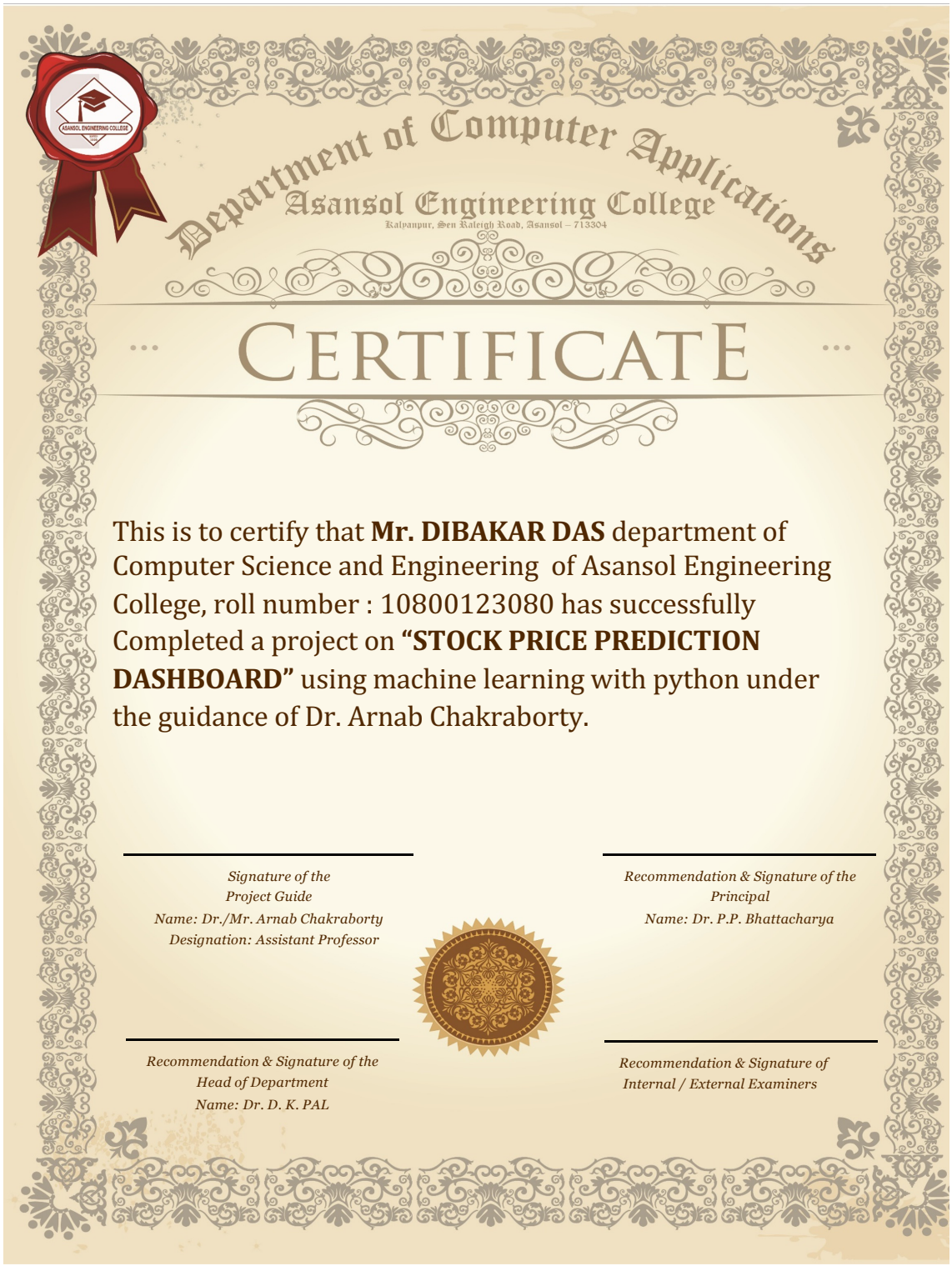
Predictions :

```
POST /api/predictions/
Authorization: Bearer <token>
Content-Type: application/json

{
  "ticker": "AAPL",
  "timeframes": ["1d", "1w", "1mo"],
  "include_analysis": true
}
```

Appendix D: Glossary of Terms

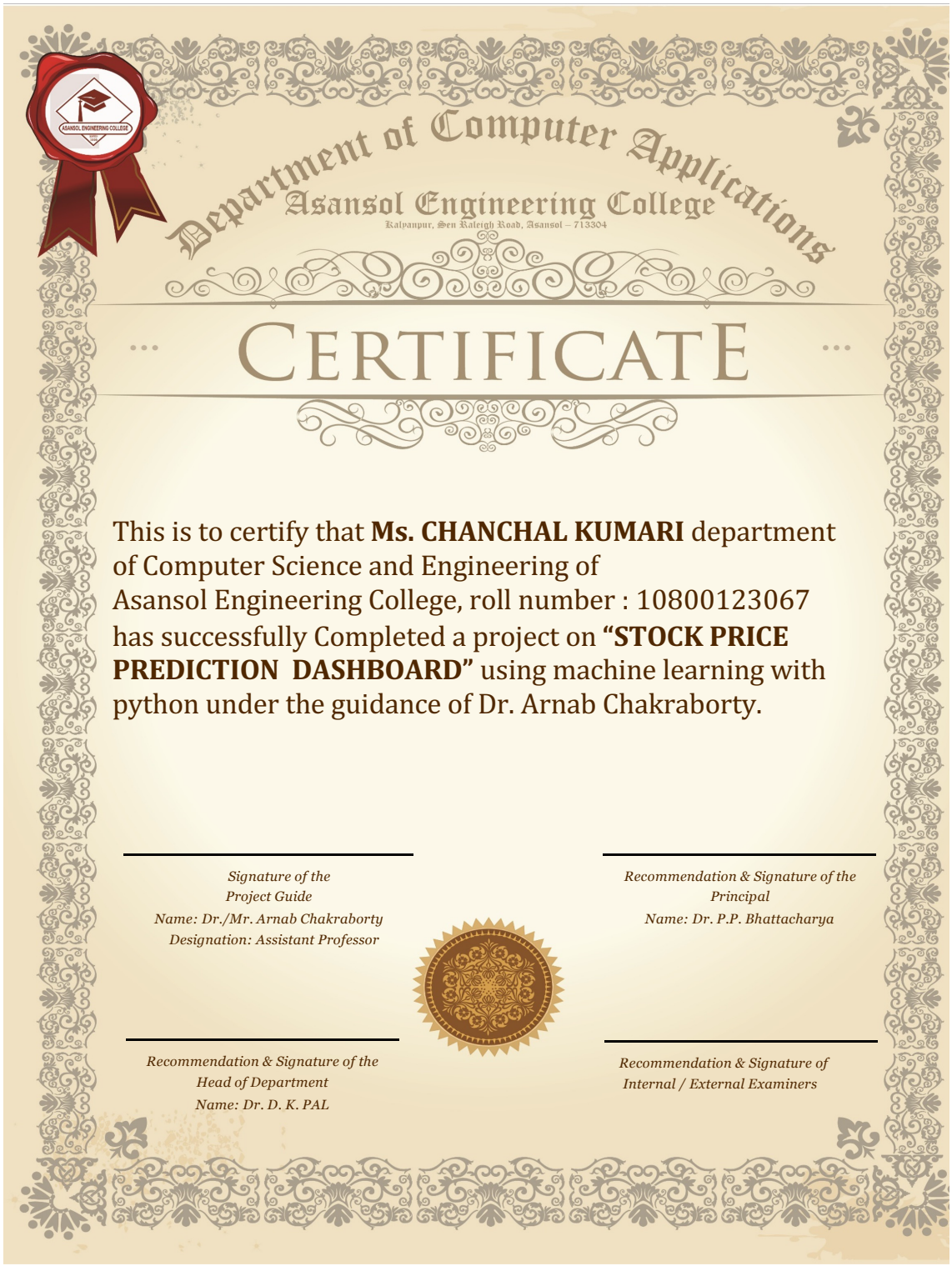
TERM	DEFINITION
API	Application Programming Interface.
ARIMA	Autoregressive Integrated Moving Average.
ATR	Average True Range.
Bollinger Bands	Volatility indicator using standard deviations.
Django	Python web framework.
EMA	Exponential Moving Average.
F1 - Score	Harmonic mean of precision and recall
LSTM	Long Short-Term Memory neural network.
MACD	Moving Average Convergence Divergence.
OHLCV	Open, High, Low, Close, Volume.
REST	Representational State Transfer.
RSI	Relative Strength Index.
SMA	Simple Moving Average.
VaR	Value at Risk.
VWAP	Volume Weighted Average Price.

PROJECT CERTIFICATES


Department of Computer Applications
Asansol Engineering College
Kalyanpur, Sen Raleigh Road, Asansol – 713304
CERTIFICATE
This is to certify that **Mr. DIBAKAR DAS** department of Computer Science and Engineering of Asansol Engineering College, roll number : 10800123080 has successfully Completed a project on **"STOCK PRICE PREDICTION DASHBOARD"** using machine learning with python under the guidance of Dr. Arnab Chakraborty.
Signature of the Project Guide
Name: Dr./Mr. Arnab Chakraborty
Designation: Assistant Professor
Recommendation & Signature of the Principal
Name: Dr. P.P. Bhattacharya
Recommendation & Signature of the Head of Department
Name: Dr. D. K. PAL
Recommendation & Signature of Internal / External Examiners


Department of Computer Applications
Asansol Engineering College
Kalyanpur, Sen Raleigh Road, Asansol – 713304
CERTIFICATE
This is to certify that **Ms. DIPIKA MAJI** department of Computer Science and Engineering of Asansol Engineering College, roll number : 10800123083 has successfully Completed a project on **"STOCK PRICE PREDICTION DASHBOARD"** using machine learning with python under the guidance of Dr. Arnab Chakraborty.
Signature of the Project Guide
Name: Dr./Mr. Arnab Chakraborty
Designation: Assistant Professor
Recommendation & Signature of the Principal
Name: Dr. P.P. Bhattacharya
Recommendation & Signature of the Head of Department
Name: Dr. D. K. PAL
Recommendation & Signature of Internal / External Examiners


Department of Computer Applications
Asansol Engineering College
Kalyanpur, Sen Raleigh Road, Asansol – 713304
CERTIFICATE
This is to certify that **Mr. INDRAJIT KHAN** department of Computer Science and Engineering of Asansol Engineering College, roll number : 10800123092 has successfully Completed a project on **"STOCK PRICE PREDICTION DASHBOARD"** using machine learning with python under the guidance of Dr. Arnab Chakraborty.
Signature of the Project Guide
Name: Dr./Mr. Arnab Chakraborty
Designation: Assistant Professor
Recommendation & Signature of the Principal
Name: Dr. P.P. Bhattacharya
Recommendation & Signature of the Head of Department
Name: Dr. D. K. PAL
Recommendation & Signature of Internal / External Examiners


Department of Computer Applications
Asansol Engineering College
Kalyanpur, Sen Raleigh Road, Asansol – 713304
CERTIFICATE
This is to certify that Ms. CHANCHAL KUMARI department of Computer Science and Engineering of Asansol Engineering College, roll number : 10800123067 has successfully Completed a project on "STOCK PRICE PREDICTION DASHBOARD" using machine learning with python under the guidance of Dr. Arnab Chakraborty.
Signature of the Project Guide
Name: Dr./Mr. Arnab Chakraborty
Designation: Assistant Professor
Recommendation & Signature of the Principal
Name: Dr. P.P. Bhattacharya
Recommendation & Signature of the Head of Department
Name: Dr. D. K. PAL
Recommendation & Signature of Internal / External Examiners

Department of Computer Applications

Asansol Engineering College

Kalyanpur, Sen Raleigh Road, Asansol – 713304

CERTIFICATE

This is to certify that **Ms. ANUSHKA MISHRA** department of Computer Science and Engineering of Asansol Engineering College, roll number : 10800123035 has successfully Completed a project on **"STOCK PRICE PREDICTION DASHBOARD"** using machine learning with python under the guidance of Dr. Arnab Chakraborty.

Signature of the
Project Guide
Name: Dr./Mr. Arnab Chakraborty
Designation: Assistant Professor

Recommendation & Signature of the
Principal
Name: Dr. P.P. Bhattacharya

Recommendation & Signature of the
Head of Department
Name: Dr. D. K. PAL

Recommendation & Signature of
Internal / External Examiners